1. Merge Sorted Array

Sıralı Dizileri Birleştir | Seviye: Kolay | LeetCode #88
Orijinal soru: leetcode.com/problems/merge-sorted-array/

Soru

Sana **nums1** ve **nums2** adında, azalmayan (non-decreasing) sırada sıralanmış iki tamsayı dizisi ve bu dizilerdeki eleman sayılarını gösteren **m** ile **n** adlı iki tamsayı veriliyor.

nums1 ve **nums2** dizilerini birleştirip azalmayan sırada tek bir dizi elde et.

Sonuç dizisi fonksiyondan döndürülmemeli; bunun yerine **nums1** dizisinin içinde saklanmalı. Bunun için **nums1**'in uzunluğu **m + n**'dir: ilk **m** eleman birleştirilmesi gereken elemanlardır, son **n** eleman ise 0'dır ve yok sayılmalıdır. **nums2**'nin uzunluğu **n**'dir.

Örnekler

Örnek 1

```
Girdi : nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3
Çıktı : [1,2,2,3,5,6]
```

Birleştirilecek diziler [1,2,3] ve [2,5,6]. Sonuç [1,2,2,3,5,6] olur.

Örnek 2

```
Girdi : nums1 = [1], m = 1, nums2 = [], n = 0
Çıktı : [1]
```

Birleştirilecek diziler [1] ve []. Sonuç [1] olur.

Örnek 3

```
Girdi : nums1 = [0], m = 0, nums2 = [1], n = 1
Çıktı : [1]
```

Birleştirilecek diziler [] ve [1]. Sonuç [1] olur. m = 0 olduğu için nums1'de yok sayılacak 0 sadece birleştirme sonucunun sığması için oradadır.

Kısıtlar

• nums1.length == m + n
• nums2.length == n
• 0 <= m, n <= 200
• 1 <= m + n <= 200
• -10^9 <= nums1[i], nums2[j] <= 10^9

Ek soru (Follow up)

O(m + n) zamanda çalışan bir algoritma bulabilir misin?

2. Remove Element

Elemanı Sil | Seviye: Kolay | LeetCode #27
Orijinal soru: leetcode.com/problems/remove-element/

Soru

Sana bir tamsayı dizisi **nums** ve bir tamsayı **val** veriliyor. **nums** içindeki **val** değerine eşit olan tüm elemanları yerinde (in-place) sil. Elemanların sırası değişebilir. Ardından, **nums** içinde **val**'e eşit olmayan eleman sayısını döndür.

val'e eşit olmayan eleman sayısına **k** diyelim. Sorunun kabul edilmesi için şunları yapman gerekir:

• **nums** dizisini öyle değiştir ki ilk **k** elemanı **val**'e eşit olmayan elemanlar olsun. Dizinin geri kalan elemanları ve dizinin boyutu önemli değildir.
• **k** değerini döndür.

Örnekler

Örnek 1

```
Girdi : nums = [3,2,2,3], val = 3
Çıktı : 2, nums = [2,2,_,_]
```

Fonksiyon k = 2 döndürmeli ve nums'ın ilk iki elemanı 2 olmalı. k'dan sonra ne bıraktığın önemli değil (alt çizgiler bunu gösterir).

Örnek 2

```
Girdi : nums = [0,1,2,2,3,0,4,2], val = 2
Çıktı : 5, nums = [0,1,4,0,3,_,_,_]
```

Fonksiyon k = 5 döndürmeli ve nums'ın ilk beş elemanı 0, 0, 1, 3, 4 olmalı. Bu beş eleman herhangi bir sırada olabilir. k'dan sonra ne bıraktığın önemli değil.

Kısıtlar

• 0 <= nums.length <= 100
• 0 <= nums[i] <= 50
• 0 <= val <= 100

3. Remove Duplicates from Sorted Array

Sıralı Diziden Tekrarları Sil | Seviye: Kolay | LeetCode #26
Orijinal soru: leetcode.com/problems/remove-duplicates-from-sorted-array/

Soru

Azalmayan (non-decreasing) sırada sıralanmış bir tamsayı dizisi **nums** veriliyor. Her benzersiz eleman yalnızca bir kez görünecek şekilde tekrarları yerinde (in-place) sil. Elemanların göreli sırası aynı kalmalı. Ardından **nums** içindeki benzersiz eleman sayısını döndür.

Benzersiz eleman sayısına **k** diyelim. Sorunun kabul edilmesi için şunları yapman gerekir:

• **nums** dizisini öyle değiştir ki ilk **k** elemanı, benzersiz elemanları başlangıçtaki sırasıyla içersin. Dizinin geri kalan elemanları ve dizinin boyutu önemli değildir.
• **k** değerini döndür.

Örnekler

Örnek 1

```
Girdi : nums = [1,1,2]
Çıktı : 2, nums = [1,2,_]
```

Fonksiyon k = 2 döndürmeli ve nums'ın ilk iki elemanı sırasıyla 1 ve 2 olmalı. k'dan sonra ne bıraktığın önemli değil.

Örnek 2

```
Girdi : nums = [0,0,1,1,1,2,2,3,3,4]
Çıktı : 5, nums = [0,1,2,3,4,_,_,_,_,_]
```

Fonksiyon k = 5 döndürmeli ve nums'ın ilk beş elemanı sırasıyla 0, 1, 2, 3 ve 4 olmalı. k'dan sonra ne bıraktığın önemli değil.

Kısıtlar

• 1 <= nums.length <= $3 * 10^4$
• -100 <= nums[i] <= 100
• nums azalmayan sırada sıralıdır.

4. Remove Duplicates from Sorted Array II

Sıralı Diziden Tekrarları Sil II | Seviye: Orta | LeetCode #80
Orijinal soru: leetcode.com/problems/remove-duplicates-from-sorted-array-ii/

Soru

Azalmayan (non-decreasing) sırada sıralanmış bir tamsayı dizisi **nums** veriliyor. Her benzersiz eleman **en fazla iki kez** görünecek şekilde bazı tekrarları yerinde (in-place) sil. Elemanların göreli sırası aynı kalmalı.

Bazı dillerde dizinin uzunluğunu değiştirmek mümkün olmadığı için sonucu **nums** dizisinin ilk kısmına yerleştirmelisin. Daha açık söylemek gerekirse, tekrarlar silindikten sonra **k** eleman kaldıysa, **nums**'ın ilk **k** elemanı nihai sonucu tutmalıdır. İlk **k** elemandan sonra ne bıraktığın önemli değildir.

Nihai sonucu **nums**'ın ilk **k** yuvasına yerleştirdikten sonra **k** değerini döndür.

Başka bir dizi için ekstra yer ayırma. Bunu, girdi dizisini yerinde değiştirerek ve **O(1)** ekstra bellekle yapmalısın.

Örnekler

Örnek 1

```
Girdi : nums = [1,1,1,2,2,3]
Çıktı : 5, nums = [1,1,2,2,3,_]
```

Fonksiyon k = 5 döndürmeli ve nums'ın ilk beş elemanı sırasıyla 1, 1, 2, 2 ve 3 olmalı. k'dan sonra ne bıraktığın önemli değil.

Örnek 2

```
Girdi : nums = [0,0,1,1,1,1,2,3,3]
Çıktı : 7, nums = [0,0,1,1,2,3,3,_,_]
```

Fonksiyon k = 7 döndürmeli ve nums'ın ilk yedi elemanı sırasıyla 0, 0, 1, 1, 2, 3 ve 3 olmalı. k'dan sonra ne bıraktığın önemli değil.

Kısıtlar

- $1 <= nums.length <= 3 * 10^4$
- $-10^4 <= nums[i] <= 10^4$
- nums azalmayan sırada sıralıdır.

5. Majority Element

Çoğunluk Elemanı | Seviye: Kolay | LeetCode #169
Orijinal soru: leetcode.com/problems/majority-element/

Soru

Boyutu **n** olan bir **nums** dizisi veriliyor. Çoğunluk elemanını döndür.

Çoğunluk elemanı, **⌊n / 2⌋** değerinden (n/2'nin aşağı yuvarlanmış hali) daha fazla kez görünen elemandır. Çoğunluk elemanının dizide her zaman var olduğunu varsayabilirsin.

Örnekler

Örnek 1

```
Girdi : nums = [3,2,3]
Çıktı : 3
```

Örnek 2

```
Girdi : nums = [2,2,1,1,1,2,2]
Çıktı : 2
```

Kısıtlar

- n == nums.length
- $1 <= n <= 5 * 10^4$
- $-10^9 <= nums[i] <= 10^9$

Ek soru (Follow up)

Problemi doğrusal zamanda (linear time) ve **O(1)** ekstra bellekle çözebilir misin?

6. Rotate Array

Diziyi Döndür | Seviye: Orta | LeetCode #189
Orijinal soru: leetcode.com/problems/rotate-array/

Soru

Bir tamsayı dizisi **nums** veriliyor. Diziyi **k** adım sağa döndür; burada **k** negatif olmayan bir sayıdır.

Örnekler

Örnek 1

```
Girdi : nums = [1,2,3,4,5,6,7], k = 3
Çıktı : [5,6,7,1,2,3,4]
```

1 adım sağa döndür: [7,1,2,3,4,5,6]
2 adım sağa döndür: [6,7,1,2,3,4,5]
3 adım sağa döndür: [5,6,7,1,2,3,4]

Örnek 2

```
Girdi : nums = [-1,-100,3,99], k = 2
Çıktı : [3,99,-1,-100]
```

1 adım sağa döndür: [99,-1,-100,3]
2 adım sağa döndür: [3,99,-1,-100]

Kısıtlar

- $1 <= nums.length <= 10^5$
- $-2^{31} <= nums[i] <= 2^{31} - 1$
- $0 <= k <= 10^5$

Ek soru (Follow up)

Aklına gelen mümkün olduğunca çok çözüm bulmaya çalış. Bu problemi çözmenin en az üç farklı yolu var. Bunu yerinde (in-place) ve **O(1)** ekstra bellekle yapabilir misin?

7. Best Time to Buy and Sell Stock

Hisse Senedini Al-Sat İçin En İyi Zaman | Seviye: Kolay | LeetCode #121
Orijinal soru: leetcode.com/problems/best-time-to-buy-and-sell-stock/

Soru

Sana bir **prices** dizisi veriliyor; **prices[i]**, bir hisse senedinin **i**. gündeki fiyatıdır.

Bir hisse senedini almak için tek bir gün, satmak için ise gelecekte farklı bir gün seçerek kârını en üst düzeye çıkarmak istiyorsun.

Bu işlemden elde edebileceğin **maksimum kârı** döndür. Hiç kâr elde edemiyorsan **0** döndür.

Örnekler

Örnek 1

```
Girdi : prices = [7,1,5,3,6,4]
Çıktı : 5
```

2. gün al (fiyat = 1), 5. gün sat (fiyat = 6), kâr = 6 - 1 = 5. Not: 2. gün alıp 1. gün satmak yasaktır, çünkü satmadan önce almalısın.

Örnek 2

```
Girdi : prices = [7,6,4,3,1]
Çıktı : 0
```

Bu durumda hiçbir işlem yapılmaz ve maksimum kâr 0'dır.

Kısıtlar

• $1 <= prices.length <= 10^5$
• $0 <= prices[i] <= 10^4$

8. Best Time to Buy and Sell Stock II

Hisse Senedini Al-Sat İçin En İyi Zaman II | Seviye: Orta | LeetCode #122
Orijinal soru: leetcode.com/problems/best-time-to-buy-and-sell-stock-ii/

Soru

Sana bir **prices** tamsayı dizisi veriliyor; **prices[i]**, bir hisse senedinin **i**. gündeki fiyatıdır.

Her gün hisse senedini almaya ve/veya satmaya karar verebilirsin. Aynı anda en fazla **bir** hisse tutabilirsin. Ancak aynı gün içinde alıp hemen satabilirsin.

Elde edebileceğin **maksimum kârı** bul ve döndür.

Örnekler

Örnek 1

```
Girdi : prices = [7,1,5,3,6,4]
Çıktı : 7
```

2. gün al (fiyat = 1), 3. gün sat (fiyat = 5), kâr = 5 - 1 = 4. Sonra 4. gün al (fiyat = 3), 5. gün sat (fiyat = 6), kâr = 6 - 3 = 3. Toplam kâr 4 + 3 = 7.

Örnek 2

```
Girdi : prices = [1,2,3,4,5]
Çıktı : 4
```

1. gün al (fiyat = 1), 5. gün sat (fiyat = 5), kâr = 5 - 1 = 4. Toplam kâr 4.

Örnek 3

```
Girdi : prices = [7,6,4,3,1]
Çıktı : 0
```

Pozitif kâr elde etmenin bir yolu yoktur; bu yüzden hiç almayız ve maksimum kâr 0 olur.

Kısıtlar

• $1 <= prices.length <= 3 * 10^4$
• $0 <= prices[i] <= 10^4$

9. Jump Game

Zıplama Oyunu | Seviye: Orta | LeetCode #55
Orijinal soru: leetcode.com/problems/jump-game/

Soru

Sana bir tamsayı dizisi **nums** veriliyor. Başlangıçta dizinin ilk indeksindesin ve dizideki her eleman, o konumdaki **maksimum zıplama uzunluğunu** gösteriyor.

Son indekse ulaşabiliyorsan **true**, ulaşamıyorsan **false** döndür.

Örnekler

Örnek 1

```
Girdi : nums = [2,3,1,1,4]
Çıktı : true
```

0. indeksten 1. indekse 1 adım zıpla, oradan son indekse 3 adım zıpla.

Örnek 2

```
Girdi : nums = [3,2,1,0,4]
Çıktı : false
```

Ne yaparsan yap her zaman 3. indekse varırsın. Bu indeksin maksimum zıplama uzunluğu 0 olduğu için son indekse ulaşmak imkânsızdır.

Kısıtlar

- $1 <= nums.length <= 10^4$
- $0 <= nums[i] <= 10^5$

10. Jump Game II

Zıplama Oyunu II | Seviye: Orta | LeetCode #45
Orijinal soru: leetcode.com/problems/jump-game-ii/

Soru

Uzunluğu **n** olan, 0'dan başlayan indeksli bir tamsayı dizisi **nums** veriliyor. Başlangıçta 0. indeksteysin. Her **nums[i]** elemanı, **i** indeksinden yapılabilecek ileri yönlü zıplamanın maksimum uzunluğunu gösterir. Yani **i** indeksindeysen, şu koşulları sağlayan herhangi bir **(i + j)** indeksine zıplayabilirsin:

• **0 <= j <= nums[i]**
• **i + j < n**

n - 1 indeksine ulaşmak için gereken **minimum zıplama sayısını** döndür. Test durumları, **n - 1** indeksine ulaşılabilecek şekilde oluşturulmuştur.

Örnekler

Örnek 1

```
Girdi : nums = [2,3,1,1,4]
Çıktı : 2
```

Son indekse ulaşmak için gereken minimum zıplama sayısı 2'dir. 0. indeksten 1. indekse 1 adım, oradan son indekse 3 adım zıpla.

Örnek 2

```
Girdi : nums = [2,3,0,1,4]
Çıktı : 2
```

Kısıtlar

• 1 <= nums.length <= 10^4
• 0 <= nums[i] <= 1000
• nums[n - 1] indeksine ulaşabileceğin garanti edilmiştir.

11. H-Index

H-İndeksi | Seviye: Orta | LeetCode #274
Orijinal soru: leetcode.com/problems/h-index/

Soru

Bir **citations** tamsayı dizisi veriliyor; **citations[i]**, bir araştırmacının **i**. makalesinin aldığı atıf sayısıdır. Araştırmacının **h-indeksini** döndür.

Vikipedi'deki tanıma göre: h-indeksi, araştırmacının her biri en az **h** kez atıf almış en az **h** makale yayımladığı **en büyük h değeridir**.

Örnekler

Örnek 1

```
Girdi : citations = [3,0,6,1,5]
Çıktı : 3
```

[3,0,6,1,5], araştırmacının toplam 5 makalesi olduğunu ve bunların sırasıyla 3, 0, 6, 1, 5 atıf aldığını gösterir. Araştırmacının en az 3 atıf almış 3 makalesi ve en fazla 3 atıf almış diğer iki makalesi olduğu için h-indeksi 3'tür.

Örnek 2

```
Girdi : citations = [1,3,1]
Çıktı : 1
```

Kısıtlar

• n == citations.length
• 1 <= n <= 5000
• 0 <= citations[i] <= 1000

12. Insert Delete GetRandom O(1)

Ekle Sil Rastgele Getir O(1) | Seviye: Orta | LeetCode #380
Orijinal soru: leetcode.com/problems/insert-delete-getrandom-o1/

Soru

RandomizedSet sınıfını gerçekle (implement et):

• **RandomizedSet()**: RandomizedSet nesnesini başlatır.
• **bool insert(int val)**: **val** öğesi kümede yoksa kümeye ekler. Öğe daha önce yoksa **true**, varsa **false** döndürür.
• **bool remove(int val)**: **val** öğesi kümede varsa kümeden siler. Öğe varsa **true**, yoksa **false** döndürür.
• **int getRandom()**: Kümedeki mevcut elemanlardan rastgele birini döndürür (bu metot çağrıldığında kümede en az bir elemanın olduğu garanti edilir). Her elemanın döndürülme olasılığı **eşit** olmalıdır.

Sınıfın fonksiyonlarını, her biri **ortalama O(1)** zaman karmaşıklığında çalışacak şekilde gerçekleştirmelisin.

Örnek

```
Girdi :
["RandomizedSet", "insert", "remove", "insert",
 "getRandom", "remove", "insert", "getRandom"]
[[], [1], [2], [2], [], [1], [2], []]
Çıktı :
[null, true, false, true, 2, true, false, 2]
```

Açıklama
RandomizedSet randomizedSet = new RandomizedSet();

• **insert(1)**: 1 kümeye başarıyla eklendi, **true** döner.
• **remove(2)**: 2 kümede yok, **false** döner.
• **insert(2)**: 2 kümeye eklenir, **true** döner. Küme şimdi [1, 2] içeriyor.
• **getRandom()**: 1 veya 2'yi rastgele döndürmeli.
• **remove(1)**: 1 kümeden silinir, **true** döner. Küme şimdi [2] içeriyor.
• **insert(2)**: 2 zaten kümede, **false** döner.
• **getRandom()**: Kümedeki tek sayı 2 olduğu için her zaman 2 döner.

Kısıtlar

• -2^{31} <= val <= 2^{31} - 1
• **insert**, **remove** ve **getRandom** için en fazla 2 * 10^5 çağrı yapılacaktır.
• **getRandom** çağrıldığında veri yapısında en az bir eleman olacaktır.

13. Product of Array Except Self

Kendisi Hariç Dizinin Çarpımı | Seviye: Orta | LeetCode #238
Orijinal soru: leetcode.com/problems/product-of-array-except-self/

Soru

Bir tamsayı dizisi **nums** veriliyor. **answer[i]** değeri, **nums[i]** hariç **nums**'ın tüm elemanlarının çarpımına eşit olan bir **answer** dizisi döndür.

nums'ın herhangi bir ön ekinin (prefix) veya son ekinin (suffix) çarpımının 32 bitlik bir tamsayıya sığacağı garanti edilmiştir.

O(n) zamanda çalışan ve **bölme işlemi kullanmayan** bir algoritma yazmalısın.

Örnekler

Örnek 1

```
Girdi : nums = [1,2,3,4]
Çıktı : [24,12,8,6]
```

Örnek 2

```
Girdi : nums = [-1,1,0,-3,3]
Çıktı : [0,0,9,0,0]
```

Kısıtlar

• 2 <= nums.length <= 10^5
• -30 <= nums[i] <= 30
• nums'ın herhangi bir ön ekinin veya son ekinin çarpımı 32 bitlik bir tamsayıya sığar.

Ek soru (Follow up)

Problemi **O(1)** ekstra bellek karmaşıklığıyla çözebilir misin? (Çıktı dizisi, bellek karmaşıklığı analizinde ekstra yer sayılmaz.)

14. Gas Station

Benzin İstasyonu | Seviye: Orta | LeetCode #134
Orijinal soru: leetcode.com/problems/gas-station/

Soru

Dairesel bir güzergâh üzerinde **n** tane benzin istasyonu var. **i**. istasyondaki benzin miktarı **gas[i]**'dir.

Depo kapasitesi sınırsız bir arabaya sahipsin. **i**. istasyondan bir sonraki **(i + 1)**. istasyona gitmek **cost[i]** kadar benzine mal olur. Yolculuğa, istasyonlardan birinde boş depoyla başlıyorsun.

gas ve **cost** tamsayı dizileri veriliyor. Devri saat yönünde bir kez dolaşabiliyorsan başlangıç istasyonunun indeksini, dolaşamıyorsan **-1** döndür. Bir çözüm varsa bunun **benzersiz** olduğu garanti edilmiştir.

Örnekler

Örnek 1

```
Girdi : gas = [1,2,3,4,5], cost = [3,4,5,1,2]
Çıktı : 3
```

3. istasyondan (indeks 3) başla ve 4 birim benzin doldur. Depo = 0 + 4 = 4. 4. istasyona git. Depo = 4 - 1 + 5 = 8. 0. istasyona git. Depo = 8 - 2 + 1 = 7. 1. istasyona git. Depo = 7 - 3 + 2 = 6. 2. istasyona git. Depo = 6 - 4 + 3 = 5. 3. istasyona git. Maliyet 5; benzinin 3. istasyona dönmeye tam yetiyor. Bu yüzden başlangıç indeksi olarak 3 döndürülür.

Örnek 2

```
Girdi : gas = [2,3,4], cost = [3,4,3]
Çıktı : -1
```

0. veya 1. istasyondan başlayamazsın, çünkü bir sonraki istasyona gitmeye yetecek benzin yok. 2. istasyondan başlayıp 4 birim benzin dolduralım. Depo = 0 + 4 = 4. 0. istasyona git. Depo = 4 - 3 + 2 = 3. 1. istasyona git. Depo = 3 - 3 + 3 = 3. 2. istasyona geri dönemezsin; bunun için 4 birim benzin gerekir ama elinde sadece 3 var. Bu yüzden nereden başlarsan başla devri bir kez dolaşamazsın.

Kısıtlar

- n == gas.length == cost.length
- 1 <= n <= 10^5
- 0 <= gas[i], cost[i] <= 10^4
- Girdi, cevap benzersiz olacak şekilde oluşturulmuştur.

15. Candy

Şeker Dağıtımı | Seviye: Zor | LeetCode #135
Orijinal soru: leetcode.com/problems/candy/

Soru

Bir sırada duran **n** çocuk var. Her çocuğa, **ratings** tamsayı dizisinde verilen bir puan (rating) atanmış.

Bu çocuklara aşağıdaki koşullara uyarak şeker dağıtıyorsun:

• Her çocuk en az bir şeker almalı.
• Daha yüksek puanlı çocuk, yanındaki komşularından daha fazla şeker almalı.

Şekerleri çocuklara dağıtmak için gereken **minimum şeker sayısını** döndür.

Örnekler

Örnek 1

```
Girdi : ratings = [1,0,2]
Çıktı : 5
```
Birinci, ikinci ve üçüncü çocuğa sırasıyla 2, 1, 2 şeker verebilirsin.

Örnek 2

```
Girdi : ratings = [1,2,2]
Çıktı : 4
```
Birinci, ikinci ve üçüncü çocuğa sırasıyla 1, 2, 1 şeker verebilirsin. Üçüncü çocuk 1 şeker alır, çünkü yukarıdaki iki koşulu da sağlar.

Kısıtlar

• n == ratings.length
• $1 <= n <= 2 * 10^4$
• $0 <= ratings[i] <= 2 * 10^4$

16. Trapping Rain Water

Yağmur Suyunu Hapsetme | Seviye: Zor | LeetCode #42
Orijinal soru: leetcode.com/problems/trapping-rain-water/

Soru

Her çubuğun genişliği 1 olan bir yükseklik haritasını temsil eden **n** adet negatif olmayan tamsayı veriliyor. Yağmurdan sonra bu haritanın ne kadar su **hapsedebileceğini** hesapla.

Orijinal soruda bir görsel var; görmek için yukarıdaki LeetCode bağlantısına bak.

Örnekler

Örnek 1

```
Girdi : height = [0,1,0,2,1,0,1,3,2,1,2,1]
Çıktı : 6
```

Yukarıdaki yükseklik haritası [0,1,0,2,1,0,1,3,2,1,2,1] dizisiyle temsil edilir. Bu durumda 6 birim yağmur suyu hapsolur.

Örnek 2

```
Girdi : height = [4,2,0,3,2,5]
Çıktı : 9
```

Kısıtlar

- n == height.length
- $1 <= n <= 2 * 10^4$
- $0 <= height[i] <= 10^5$

17. Roman to Integer

Romen Rakamını Sayıya Çevir | Seviye: Kolay | LeetCode #13
Orijinal soru: leetcode.com/problems/roman-to-integer/

Soru

Romen rakamları yedi farklı sembolle gösterilir: **I, V, X, L, C, D** ve **M**.

```
I = 1    V = 5    X = 10    L = 50    C = 100    D = 500    M = 1000
```

Örneğin 2, Romen rakamıyla II olarak yazılır; yani iki tane bir toplanmıştır. 12, XII olarak yazılır, bu da X + II demektir. 27 sayısı XXVII olarak yazılır, bu da XX + V + II'dir.

Romen rakamları genellikle soldan sağa doğru büyükten küçüğe yazılır. Ancak dört sayısı IIII olarak yazılmaz; bunun yerine IV yazılır. Çünkü bir, beşin önünde olduğu için beşten çıkarılır ve dört elde edilir. Aynı ilke dokuz için de geçerlidir; dokuz IX olarak yazılır. Çıkarma kuralının kullanıldığı altı durum vardır:

• **I**, 4 ve 9'u yapmak için **V** (5) ve **X**'in (10) önüne konabilir.
• **X**, 40 ve 90'ı yapmak için **L** (50) ve **C**'nin (100) önüne konabilir.
• **C**, 400 ve 900'ü yapmak için **D** (500) ve **M**'nin (1000) önüne konabilir.

Bir Romen rakamı verildiğinde onu tamsayıya çevir.

Örnekler

Örnek 1

```
Girdi : s = "III"
Çıktı : 3
```
III = 3.

Örnek 2

```
Girdi : s = "LVIII"
Çıktı : 58
```
L = 50, V = 5, III = 3.

Örnek 3

```
Girdi : s = "MCMXCIV"
Çıktı : 1994
```
M = 1000, CM = 900, XC = 90 ve IV = 4.

Kısıtlar

• 1 <= s.length <= 15
• s yalnızca ('I', 'V', 'X', 'L', 'C', 'D', 'M') karakterlerini içerir.
• s'nin [1, 3999] aralığında geçerli bir Romen rakamı olduğu garanti edilmiştir.

18. Integer to Roman

Sayıyı Romen Rakamına Çevir | Seviye: Orta | LeetCode #12
Orijinal soru: leetcode.com/problems/integer-to-roman/

Soru

Romen rakamları şu değerlere sahip yedi farklı sembolle gösterilir:

```
I = 1    V = 5    X = 10    L = 50    C = 100    D = 500    M = 1000
```

Romen rakamları, onluk basamak değerlerinin dönüşümleri en yüksekten en düşüğe doğru art arda eklenerek oluşturulur. Bir onluk basamak değerini Romen rakamına çevirmenin kuralları şunlardır:

• Değer 4 veya 9 ile başlamıyorsa, girdiden çıkarılabilecek en büyük değerli sembolü seç, bu sembolü sonuca ekle, değerini çıkar ve kalanı Romen rakamına çevir.
• Değer 4 veya 9 ile başlıyorsa, bir sembolün bir sonraki sembolden çıkarıldığı çıkarmalı biçimi kullan. Örneğin 4, 5'ten (V) 1 (I) eksiktir: IV; 9 ise 10'dan (X) 1 (I) eksiktir: IX. Yalnızca şu çıkarmalı biçimler kullanılır: 4 (IV), 9 (IX), 40 (XL), 90 (XC), 400 (CD) ve 900 (CM).
• Yalnızca 10'un kuvvetleri (I, X, C, M) en fazla 3 kez art arda eklenerek 10'un katları gösterilebilir. 5 (V), 50 (L) veya 500 (D) birden fazla kez eklenemez. Bir sembolü 4 kez eklemen gerekiyorsa çıkarmalı biçimi kullan.

Bir tamsayı verildiğinde onu Romen rakamına çevir.

Örnekler

Örnek 1

```
Girdi : num = 3749
Çıktı : "MMMDCCXLIX"
```

3000 = MMM (1000 (M) + 1000 (M) + 1000 (M))
700 = DCC (500 (D) + 100 (C) + 100 (C))
40 = XL (50'den (L) 10 (X) eksik)
9 = IX (10'dan (X) 1 (I) eksik)
Not: 49, 50'den (L) 1 (I) eksik değildir; çünkü dönüşüm onluk basamaklara göre yapılır.

Örnek 2

```
Girdi : num = 58
Çıktı : "LVIII"
```

50 = L, 8 = VIII.

Örnek 3

```
Girdi : num = 1994
Çıktı : "MCMXCIV"
```

1000 = M, 900 = CM, 90 = XC, 4 = IV.

Kısıtlar

• 1 <= num <= 3999

19. Length of Last Word

Son Kelimenin Uzunluğu | Seviye: Kolay | LeetCode #58
Orijinal soru: leetcode.com/problems/length-of-last-word/

Soru

Kelimelerden ve boşluklardan oluşan bir **s** dizgisi (string) veriliyor. Dizgideki **son kelimenin** uzunluğunu döndür.

Kelime, yalnızca boşluk olmayan karakterlerden oluşan en uzun (maksimal) alt dizgidir.

Örnekler

Örnek 1

```
Girdi : s = "Hello World"
Çıktı : 5
```

Son kelime, uzunluğu 5 olan “World”dür.

Örnek 2

```
Girdi : s = "   fly me   to   the moon  "
Çıktı : 4
```

Son kelime, uzunluğu 4 olan “moon”dur.

Örnek 3

```
Girdi : s = "luffy is still joyboy"
Çıktı : 6
```

Son kelime, uzunluğu 6 olan “joyboy”dur.

Kısıtlar

• $1 <= s.length <= 10^4$
• s yalnızca İngilizce harflerden ve boşluk karakterinden ' ' oluşur.
• s içinde en az bir kelime vardır.

20. Longest Common Prefix

En Uzun Ortak Önek | Seviye: Kolay | LeetCode #14
Orijinal soru: leetcode.com/problems/longest-common-prefix/

Soru

Bir dizgi dizisi içindeki dizgilerin **en uzun ortak önekini** (longest common prefix) bulan bir fonksiyon yaz.

Ortak önek yoksa boş dizgi **""** döndür.

Örnekler

Örnek 1

```
Girdi : strs = ["flower","flow","flight"]
Çıktı : "fl"
```

Örnek 2

```
Girdi : strs = ["dog","racecar","car"]
Çıktı : ""
```

Girdi dizgileri arasında ortak önek yoktur.

Kısıtlar

- 1 <= strs.length <= 200
- 0 <= strs[i].length <= 200
- strs[i] boş değilse yalnızca küçük İngilizce harflerden oluşur.

21. Reverse Words in a String

Dizgideki Kelimeleri Ters Çevir | Seviye: Orta | LeetCode #151
Orijinal soru: leetcode.com/problems/reverse-words-in-a-string/

Soru

Bir **s** dizgisi veriliyor. Kelimelerin sırasını ters çevir.

Kelime, boşluk olmayan karakterlerden oluşan bir dizidir. **s** içindeki kelimeler en az bir boşlukla ayrılmıştır.

Kelimelerin ters sırada ve aralarında **tek bir boşluk** olacak şekilde birleştirildiği bir dizgi döndür.

Dikkat: **s** başında veya sonunda boşluk içerebilir ya da iki kelime arasında birden fazla boşluk olabilir. Döndürülen dizgide kelimeleri yalnızca tek bir boşluk ayırmalı; fazladan boşluk olmamalı.

Örnekler

Örnek 1

```
Girdi : s = "the sky is blue"
Çıktı : "blue is sky the"
```

Örnek 2

```
Girdi : s = "  hello world  "
Çıktı : "world hello"
```

Ters çevrilmiş dizgi başta veya sonda boşluk içermemeli.

Örnek 3

```
Girdi : s = "a good   example"
Çıktı : "example good a"
```

Ters çevrilmiş dizgide iki kelime arasındaki birden fazla boşluğu tek boşluğa indirmelisin.

Kısıtlar

• $1 <= s.length <= 10^4$
• s İngilizce harflerden (büyük ve küçük), rakamlardan ve boşluk karakterinden ' ' oluşur.
• s içinde en az bir kelime vardır.

Ek soru (Follow up)

Kullandığın dilde dizgi veri tipi değiştirilebilir (mutable) ise, bunu yerinde ve **O(1)** ekstra bellekle çözebilir misin?

22. Zigzag Conversion

Zikzak Dönüşümü | Seviye: Orta | LeetCode #6
Orijinal soru: leetcode.com/problems/zigzag-conversion/

Soru

“PAYPALISHIRING” dizgisi, verilen sayıda satır üzerinde şu şekilde zikzak desende yazılır:

```
P   A   H   N
A P L S I I G
Y   I   R
```

(Okunabilirlik için bu deseni sabit genişlikli bir yazı tipiyle görüntülemek isteyebilirsin.) Sonra satır satır okunur: “PAHNAPLSIIGYIR”.

Bir dizgiyi ve satır sayısını alıp bu dönüşümü yapan kodu yaz:

```
string convert(string s, int numRows);
```

Örnekler

Örnek 1

```
Girdi : s = "PAYPALISHIRING", numRows = 3
Çıktı : "PAHNAPLSIIGYIR"
```

Örnek 2

```
Girdi : s = "PAYPALISHIRING", numRows = 4
Çıktı : "PINALSIGYAHRPI"
```

Açıklama:

```
P     I    N
A   L S  I G
Y A   H R
P     I
```

Örnek 3

```
Girdi : s = "A", numRows = 1
Çıktı : "A"
```

Kısıtlar

• 1 <= s.length <= 1000
• s İngilizce harflerden (küçük ve büyük), ',' ve '.' karakterlerinden oluşur.
• 1 <= numRows <= 1000

23. Find the Index of the First Occurrence in a String

Dizgide İlk Eşleşmenin İndeksini Bul | Seviye: Kolay | LeetCode #28
Orijinal soru: leetcode.com/problems/find-the-index-of-the-first-occurrence-in-a-string/

Soru

İki dizgi **needle** ve **haystack** veriliyor. **needle**'ın **haystack** içinde ilk geçtiği yerin indeksini döndür. **needle**, **haystack**'in bir parçası değilse **-1** döndür.

Örnekler

Örnek 1

```
Girdi : haystack = "sadbutsad", needle = "sad"
Çıktı : 0
```

“sad”, 0 ve 6. indekslerde geçer. İlk geçtiği yer 0. indeks olduğu için 0 döndürürüz.

Örnek 2

```
Girdi : haystack = "leetcode", needle = "leeto"
Çıktı : -1
```

“leeto”, “leetcode” içinde geçmez; bu yüzden -1 döndürürüz.

Kısıtlar

• 1 <= haystack.length, needle.length <= 10^4
• haystack ve needle yalnızca küçük İngilizce harflerden oluşur.

24. Text Justification

Metin Hizalama | Seviye: Zor | LeetCode #68
Orijinal soru: leetcode.com/problems/text-justification/

Soru

Bir **words** dizgi dizisi ve bir **maxWidth** genişliği veriliyor. Metni öyle biçimlendir ki her satır tam olarak **maxWidth** karakter olsun ve iki yana yaslı (hem sol hem sağ hizalı) olsun.

Kelimeleri **açgözlü (greedy)** bir yaklaşımla yerleştirmelisin; yani her satıra sığabildiği kadar çok kelime koy. Her satırın tam olarak **maxWidth** karakter olması için gerektiğinde fazladan boşluk (' ') ekle.

Kelimeler arasındaki fazladan boşluklar mümkün olduğunca eşit dağıtılmalıdır. Bir satırdaki boşluk sayısı kelimeler arasında eşit bölünmüyorsa, soldaki boş aralıklara sağdakilerden daha fazla boşluk verilir.

Metnin **son satırı** sola yaslı olmalı ve kelimeler arasına fazladan boşluk eklenmemelidir.

Not:
• **Kelime**, yalnızca boşluk olmayan karakterlerden oluşan bir karakter dizisidir.
• Her kelimenin uzunluğunun 0'dan büyük olduğu ve **maxWidth**'i aşmadığı garanti edilmiştir.
• Girdi dizisi **words** en az bir kelime içerir.

Örnekler

Örnek 1

```
Girdi : words = ["This", "is", "an", "example", "of", "text", "justification."],
        maxWidth = 16
Çıktı : ["This    is    an",
         "example  of text",
         "justification.  "]
```

Örnek 2

```
Girdi : words = ["What","must","be","acknowledgment","shall","be"],
        maxWidth = 16
Çıktı : ["What   must   be",
         "acknowledgment  ",
         "shall be        "]
```

Son satırın “shall be ” şeklinde olduğuna dikkat et, “shall be” değil; çünkü son satır iki yana yaslı değil sola yaslı olmalıdır. İkinci satırın da sola yaslı olduğuna dikkat et; çünkü yalnızca bir kelime içerir.

Örnek 3

```
Girdi : words = ["Science","is","what","we","understand","well","enough","to",
         "explain","to","a","computer.","Art","is","everything","else",
         "we","do"],
        maxWidth = 20
Çıktı : ["Science  is  what we",
         "understand      well",
         "enough to explain to",
         "a  computer.  Art is",
         "everything  else  we",
         "do                  "]
```

Kısıtlar

• 1 <= words.length <= 300
• 1 <= words[i].length <= 20
• words[i] yalnızca İngilizce harflerden ve sembollerden oluşur.
• 1 <= maxWidth <= 100
• words[i].length <= maxWidth

25. Valid Palindrome

Geçerli Palindrom | Seviye: Kolay | LeetCode #125
Orijinal soru: leetcode.com/problems/valid-palindrome/

Soru

Bir ifade, tüm büyük harfler küçük harfe çevrildikten ve harf-rakam olmayan (alfanümerik olmayan) tüm karakterler silindikten sonra, baştan ve sondan okunduğunda aynı oluyorsa **palindromdur**. Alfanümerik karakterler harfleri ve rakamları kapsar.

Bir **s** dizgisi veriliyor. Palindromsa **true**, değilse **false** döndür.

Örnekler

Örnek 1

```
Girdi : s = "A man, a plan, a canal: Panama"
Çıktı : true
```

“amanaplanacanalpanama” bir palindromdur.

Örnek 2

```
Girdi : s = "race a car"
Çıktı : false
```

“raceacar” bir palindrom değildir.

Örnek 3

```
Girdi : s = " "
Çıktı : true
```

s, alfanümerik olmayan karakterler silindikten sonra boş dizgi (“”) olur. Boş dizgi baştan ve sondan okunduğunda aynı olduğu için bir palindromdur.

Kısıtlar

• $1 <= s.length <= 2 * 10^5$
• s yalnızca yazdırılabilir ASCII karakterlerinden oluşur.

26. Is Subsequence

Alt Dizi mi? | Seviye: Kolay | LeetCode #392
Orijinal soru: leetcode.com/problems/is-subsequence/

Soru

İki dizgi **s** ve **t** veriliyor. **s**, **t**'nin bir alt dizisiyse (subsequence) **true**, değilse **false** döndür.

Bir dizginin **alt dizisi**, orijinal dizgiden bazı karakterlerin (hiçbiri de olabilir) silinmesiyle, kalan karakterlerin göreli konumları bozulmadan oluşturulan yeni dizgidir. (Örneğin “ace”, “abcde”nin bir alt dizisidir; “aec” değildir.)

Örnekler

Örnek 1

```
Girdi : s = "abc", t = "ahbgdc"
Çıktı : true
```

Örnek 2

```
Girdi : s = "axc", t = "ahbgdc"
Çıktı : false
```

Kısıtlar

• 0 <= s.length <= 100
• 0 <= t.length <= 10^4
• s ve t yalnızca küçük İngilizce harflerden oluşur.

Ek soru (Follow up)

Çok sayıda gelen **s** dizgisi olduğunu düşün: s_1, s_2, ..., s_k ve **k >= 10^9**. Bunları tek tek kontrol edip **t**'nin bu dizgilerin alt dizisi olup olmadığına bakmak istiyorsun. Bu senaryoda kodunu nasıl değiştirirdin?

27. Two Sum II - Input Array Is Sorted

İki Toplam II - Girdi Dizisi Sıralı | Seviye: Orta | LeetCode #167
Orijinal soru: leetcode.com/problems/two-sum-ii-input-array-is-sorted/

Soru

1'den başlayan indeksli (1-indexed) ve azalmayan sırada sıralanmış bir **numbers** tamsayı dizisi veriliyor. Toplamları belirli bir **target** sayısına eşit olan iki sayı bul. Bu iki sayı **numbers[index1]** ve **numbers[index2]** olsun; burada **1 <= index1 < index2 <= numbers.length**.

İki sayının indekslerini, **index1** ve **index2**'yi, uzunluğu 2 olan **[index1, index2]** tamsayı dizisi olarak döndür.

Test durumları tam olarak **bir** çözüm olacak şekilde oluşturulmuştur. Aynı elemanı iki kez kullanamazsın.

Çözümün yalnızca **sabit (constant) ekstra bellek** kullanmalıdır.

Orijinal metinde indekslerin “bir eklenmiş olarak” döndürüleceği yazar; dizi zaten 1'den başlayan indeksli olduğu için örneklerde indeksler 1'den başlar.

Örnekler

Örnek 1

```
Girdi : numbers = [2,7,11,15], target = 9
Çıktı : [1,2]
```

2 ile 7'nin toplamı 9'dur. Bu yüzden index1 = 1, index2 = 2. [1, 2] döndürürüz.

Örnek 2

```
Girdi : numbers = [2,3,4], target = 6
Çıktı : [1,3]
```

2 ile 4'ün toplamı 6'dır. Bu yüzden index1 = 1, index2 = 3. [1, 3] döndürürüz.

Örnek 3

```
Girdi : numbers = [-1,0], target = -1
Çıktı : [1,2]
```

-1 ile 0'ın toplamı -1'dir. Bu yüzden index1 = 1, index2 = 2. [1, 2] döndürürüz.

Kısıtlar

- 2 <= numbers.length <= $3 * 10^4$
- -1000 <= numbers[i] <= 1000
- numbers azalmayan sırada sıralıdır.
- -1000 <= target <= 1000
- Test durumları tam olarak bir çözüm olacak şekilde oluşturulmuştur.

28. Container With Most Water

En Çok Su Alan Kap | Seviye: Orta | LeetCode #11
Orijinal soru: leetcode.com/problems/container-with-most-water/

Soru

Uzunluğu **n** olan bir **height** tamsayı dizisi veriliyor. **n** tane dikey çizgi çizilmiş; **i**. çizginin iki uç noktası **(i, 0)** ve **(i, height[i])**'dir.

x ekseniyle birlikte bir kap oluşturan öyle iki çizgi bul ki kap en çok suyu içersin.

Bir kabın depolayabileceği **maksimum su miktarını** döndür.

Dikkat: Kabı eğemezsin.

Orijinal soruda bir görsel var; görmek için yukarıdaki LeetCode bağlantısına bak.

Örnekler

Örnek 1

```
Girdi : height = [1,8,6,2,5,4,8,3,7]
Çıktı : 49
```

Dikey çizgiler [1,8,6,2,5,4,8,3,7] dizisiyle temsil edilir. Bu durumda kabın alabileceği maksimum su alanı 49'dur.

Örnek 2

```
Girdi : height = [1,1]
Çıktı : 1
```

Kısıtlar

- n == height.length
- 2 <= n <= 10^5
- 0 <= height[i] <= 10^4

29. 3Sum

Üç Toplam | Seviye: Orta | LeetCode #15
Orijinal soru: leetcode.com/problems/3sum/

Soru

Bir **nums** tamsayı dizisi veriliyor. **i != j**, **i != k** ve **j != k** olmak üzere **nums[i] + nums[j] + nums[k] == 0** koşulunu sağlayan tüm **[nums[i], nums[j], nums[k]]** üçlülerini döndür.

Dikkat: Çözüm kümesi **tekrar eden üçlüler** içermemelidir.

Örnekler

Örnek 1

```
Girdi : nums = [-1,0,1,2,-1,-4]
Çıktı : [[-1,-1,2],[-1,0,1]]
```

nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
Farklı üçlüler [-1,0,1] ve [-1,-1,2]'dir.
Çıktının sırasının ve üçlülerin sırasının önemli olmadığına dikkat et.

Örnek 2

```
Girdi : nums = [0,1,1]
Çıktı : []
```

Mümkün olan tek üçlünün toplamı 0 değildir.

Örnek 3

```
Girdi : nums = [0,0,0]
Çıktı : [[0,0,0]]
```

Mümkün olan tek üçlünün toplamı 0'dır.

Kısıtlar

• 3 <= nums.length <= 3000
• -10^5 <= nums[i] <= 10^5

30. Minimum Size Subarray Sum

Minimum Boyutlu Alt Dizi Toplamı | Seviye: Orta | LeetCode #209
Orijinal soru: leetcode.com/problems/minimum-size-subarray-sum/

Soru

Pozitif tamsayılardan oluşan bir **nums** dizisi ve pozitif bir tamsayı **target** veriliyor. Toplamı **target**'a eşit veya ondan büyük olan bir alt dizinin (subarray) **minimum uzunluğunu** döndür. Böyle bir alt dizi yoksa bunun yerine **0** döndür.

Örnekler

Örnek 1

```
Girdi : target = 7, nums = [2,3,1,2,4,3]
Çıktı : 2
```

[4,3] alt dizisi, problem koşulu altında minimum uzunluğa sahiptir.

Örnek 2

```
Girdi : target = 4, nums = [1,4,4]
Çıktı : 1
```

Örnek 3

```
Girdi : target = 11, nums = [1,1,1,1,1,1,1,1]
Çıktı : 0
```

Kısıtlar

- $1 <= target <= 10^9$
- $1 <= nums.length <= 10^5$
- $1 <= nums[i] <= 10^4$

Ek soru (Follow up)

O(n) çözümü bulduysan, zaman karmaşıklığı **O(n log(n))** olan başka bir çözüm yazmaya çalış.

31. Longest Substring Without Repeating Characters

Tekrarsız En Uzun Alt Dizgi | Seviye: Orta | LeetCode #3
Orijinal soru: leetcode.com/problems/longest-substring-without-repeating-characters/

Soru

Bir **s** dizgisi veriliyor. Tekrar eden karakter içermeyen en uzun **alt dizginin** (substring) uzunluğunu bul.

Örnekler

Örnek 1

```
Girdi : s = "abcabcbb"
Çıktı : 3
```

Cevap, uzunluğu 3 olan “abc”dir.

Örnek 2

```
Girdi : s = "bbbbb"
Çıktı : 1
```

Cevap, uzunluğu 1 olan “b”dir.

Örnek 3

```
Girdi : s = "pwwkew"
Çıktı : 3
```

Cevap, uzunluğu 3 olan “wke”dir. Cevabın bir alt dizgi olması gerektiğine dikkat et; “pwke” bir alt dizidir (subsequence), alt dizgi (substring) değildir.

Kısıtlar

• $0 <= s.length <= 5 * 10^4$
• s İngilizce harflerden, rakamlardan, sembollerden ve boşluklardan oluşur.

32. Substring with Concatenation of All Words

Tüm Kelimelerin Birleşimini İçeren Alt Dizgi | Seviye: Zor | LeetCode #30
Orijinal soru: leetcode.com/problems/substring-with-concatenation-of-all-words/

Soru

Bir **s** dizgisi ve bir **words** dizgi dizisi veriliyor. **words** içindeki tüm dizgilerin uzunluğu aynıdır.

Birleştirilmiş dizgi, **words**'ün herhangi bir permütasyonundaki tüm dizgileri art arda ekleyerek elde edilen ve bunların hepsini tam olarak içeren dizgidir.

Örneğin **words = ["ab","cd","ef"]** ise “abcdef”, “abefcd”, “cdabef”, “cdefab”, “efabcd” ve “efcdab” birleştirilmiş dizgilerdir. “acdbef” ise **words**'ün hiçbir permütasyonunun birleşimi olmadığı için birleştirilmiş dizgi değildir.

s içindeki tüm birleştirilmiş alt dizgilerin başlangıç indekslerinden oluşan bir dizi döndür. Cevabı herhangi bir sırada döndürebilirsin.

Örnekler

Örnek 1

```
Girdi : s = "barfoothefoobarman", words = ["foo","bar"]
Çıktı : [0,9]
```

0. indeksten başlayan alt dizgi “barfoo”dur. Bu, words'ün bir permütasyonu olan [“bar”,“foo”]'nun birleşimidir.
9. indeksten başlayan alt dizgi “foobar”dır. Bu, words'ün bir permütasyonu olan [“foo”,“bar”]'ın birleşimidir.

Örnek 2

```
Girdi : s = "wordgoodgoodgoodbestword", words = ["word","good","best","word"]
Çıktı : []
```

Birleştirilmiş alt dizgi yoktur.

Örnek 3

```
Girdi : s = "barfoofoobarthefoobarman", words = ["bar","foo","the"]
Çıktı : [6,9,12]
```

6. indeksten başlayan alt dizgi “foobarthe”dir. Bu, [“foo”,“bar”,“the”]'nin birleşimidir.
9. indeksten başlayan alt dizgi “barthefoo”dur. Bu, [“bar”,“the”,“foo”]'nun birleşimidir.
12. indeksten başlayan alt dizgi “thefoobar”dır. Bu, [“the”,“foo”,“bar”]'ın birleşimidir.

Kısıtlar

- 1 <= s.length <= 10^4
- 1 <= words.length <= 5000
- 1 <= words[i].length <= 30
- s ve words[i] küçük İngilizce harflerden oluşur.

33. Minimum Window Substring

Minimum Pencere Alt Dizgisi | Seviye: Zor | LeetCode #76
Orijinal soru: leetcode.com/problems/minimum-window-substring/

Soru

Uzunlukları sırasıyla **m** ve **n** olan iki dizgi **s** ve **t** veriliyor. **t** içindeki her karakteri (tekrarlar dahil) içeren **s**'in **minimum pencere alt dizgisini** döndür. Böyle bir alt dizgi yoksa boş dizgi **""** döndür.

Test durumları cevabın benzersiz olacağı şekilde oluşturulmuştur.

Örnekler

Örnek 1

```
Girdi : s = "ADOBECODEBANC", t = "ABC"
Çıktı : "BANC"
```

Minimum pencere alt dizgisi “BANC”, t dizgisindeki 'A', 'B' ve 'C' karakterlerini içerir.

Örnek 2

```
Girdi : s = "a", t = "a"
Çıktı : "a"
```

Dizginin tamamı minimum penceredir.

Örnek 3

```
Girdi : s = "a", t = "aa"
Çıktı : ""
```

t içindeki iki 'a'nın ikisi de pencerede bulunmalı. s'in en büyük penceresinde yalnızca bir 'a' olduğu için boş dizgi döndürülür.

Kısıtlar

• m == s.length
• n == t.length
• 1 <= m, n <= 10^5
• s ve t büyük ve küçük İngilizce harflerden oluşur.

Ek soru (Follow up)

O(m + n) zamanda çalışan bir algoritma bulabilir misin?

34. Valid Sudoku

Geçerli Sudoku | Seviye: Orta | LeetCode #36
Orijinal soru: leetcode.com/problems/valid-sudoku/

Soru

9 x 9 bir Sudoku tahtasının geçerli olup olmadığını belirle. Yalnızca dolu hücrelerin aşağıdaki kurallara göre doğrulanması gerekir:

1. Her satır, 1-9 rakamlarını tekrarsız içermelidir.
2. Her sütun, 1-9 rakamlarını tekrarsız içermelidir.
3. Izgaranın dokuz adet **3 x 3** alt kutusunun her biri, 1-9 rakamlarını tekrarsız içermelidir.

Not: Kısmen doldurulmuş bir Sudoku tahtası geçerli olabilir ama çözülebilir olmak zorunda değildir. Yalnızca dolu hücrelerin belirtilen kurallara göre doğrulanması gerekir.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : board =
        [["5","3",".",".","7",".",".",".","."],
         ["6",".",".","1","9","5",".",".","."],
         [".","9","8",".",".",".",".","6","."],
         ["8",".",".",".","6",".",".",".","3"],
         ["4",".",".","8",".","3",".",".","1"],
         ["7",".",".",".","2",".",".",".","6"],
         [".","6",".",".",".",".","2","8","."],
         [".",".",".","4","1","9",".",".","5"],
         [".",".",".",".","8",".",".","7","9"]]
Çıktı : true
```

Örnek 2

```
Girdi : board =
        [["8","3",".",".","7",".",".",".","."],
         ["6",".",".","1","9","5",".",".","."],
         [".","9","8",".",".",".",".","6","."],
         ["8",".",".",".","6",".",".",".","3"],
         ["4",".",".","8",".","3",".",".","1"],
         ["7",".",".",".","2",".",".",".","6"],
         [".","6",".",".",".",".","2","8","."],
         [".",".",".","4","1","9",".",".","5"],
         [".",".",".",".","8",".",".","7","9"]]
Çıktı : false
```

Örnek 1 ile aynı, ancak sol üst köşedeki 5 değeri 8 yapılmış. Sol üstteki 3x3 alt kutuda iki tane 8 olduğu için tahta geçersizdir.

Kısıtlar

- board.length == 9
- board[i].length == 9
- board[i][j] ya 1-9 arası bir rakamdır ya da '.' karakteridir.

35. Spiral Matrix

Spiral Matris | Seviye: Orta | LeetCode #54
Orijinal soru: leetcode.com/problems/spiral-matrix/

Soru

m x n boyutunda bir **matrix** veriliyor. Matrisin tüm elemanlarını **spiral (sarmal) sırayla** döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : matrix = [[1,2,3],[4,5,6],[7,8,9]]
Çıktı : [1,2,3,6,9,8,7,4,5]
```

Örnek 2

```
Girdi : matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
Çıktı : [1,2,3,4,8,12,11,10,9,5,6,7]
```

Kısıtlar

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 10
- -100 <= matrix[i][j] <= 100

36. Rotate Image

Resmi Döndür | Seviye: Orta | LeetCode #48
Orijinal soru: leetcode.com/problems/rotate-image/

Soru

Bir görüntüyü temsil eden **n x n** boyutunda 2 boyutlu bir **matrix** veriliyor. Görüntüyü **90 derece (saat yönünde)** döndür.

Görüntüyü **yerinde (in-place)** döndürmelisin; yani girdi olan 2 boyutlu matrisi doğrudan değiştirmelisin. Döndürme için başka bir 2 boyutlu matris **oluşturma**.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : matrix = [[1,2,3],[4,5,6],[7,8,9]]
Çıktı : [[7,4,1],[8,5,2],[9,6,3]]
```

Örnek 2

```
Girdi : matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
Çıktı : [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]
```

Kısıtlar

- n == matrix.length == matrix[i].length
- 1 <= n <= 20
- -1000 <= matrix[i][j] <= 1000

37. Set Matrix Zeroes

Matrisi Sıfırla | Seviye: Orta | LeetCode #73
Orijinal soru: leetcode.com/problems/set-matrix-zeroes/

Soru

m x n boyutunda bir tamsayı matrisi **matrix** veriliyor. Bir eleman 0 ise, o elemanın tüm satırını ve sütununu 0 yap.

Bunu **yerinde (in-place)** yapmalısın.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : matrix = [[1,1,1],[1,0,1],[1,1,1]]
Çıktı : [[1,0,1],[0,0,0],[1,0,1]]
```

Örnek 2

```
Girdi : matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]
Çıktı : [[0,0,0,0],[0,4,5,0],[0,3,1,0]]
```

Kısıtlar

- m == matrix.length
- n == matrix[0].length
- 1 <= m, n <= 200
- -2^{31} <= matrix[i][j] <= $2^{31} - 1$

Ek soru (Follow up)

O(mn) bellek kullanan basit bir çözüm muhtemelen kötü bir fikirdir. Basit bir iyileştirme **O(m + n)** bellek kullanır ama bu da en iyi çözüm değildir. **Sabit (constant) bellekli** bir çözüm bulabilir misin?

38. Game of Life

Yaşam Oyunu | Seviye: Orta | LeetCode #289
Orijinal soru: leetcode.com/problems/game-of-life/

Soru

Vikipedi'ye göre: “Yaşam Oyunu (Game of Life), kısaca Life olarak da bilinir, İngiliz matematikçi John Horton Conway tarafından 1970'te geliştirilmiş bir hücresel otomattır.”

Tahta, **m x n** boyutunda bir hücre ızgarasından oluşur ve her hücrenin başlangıç durumu **canlı** (1 ile gösterilir) ya da **ölü** (0 ile gösterilir). Her hücre, sekiz komşusuyla (yatay, dikey, çapraz) aşağıdaki dört kurala göre etkileşir:

1. İkiden az canlı komşusu olan canlı hücre, nüfus azlığından ölür.
2. İki ya da üç canlı komşusu olan canlı hücre bir sonraki nesle yaşamaya devam eder.
3. Üçten fazla canlı komşusu olan canlı hücre, aşırı nüfustan ölür.
4. Tam olarak üç canlı komşusu olan ölü hücre, üreme yoluyla canlı hücre olur.

Tahtanın bir sonraki durumu, yukarıdaki kuralların **aynı anda** mevcut durumdaki her hücreye uygulanmasıyla belirlenir. Bu süreçte doğumlar ve ölümler aynı anda gerçekleşir.

Tahtanın mevcut durumu verildiğinde, tahtayı bir sonraki durumunu yansıtacak şekilde güncelle. Hiçbir şey döndürmen gerekmediğine dikkat et.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : board = [[0,1,0],[0,0,1],[1,1,1],[0,0,0]]
Çıktı : [[0,0,0],[1,0,1],[0,1,1],[0,1,0]]
```

Örnek 2

```
Girdi : board = [[1,1],[1,0]]
Çıktı : [[1,1],[1,1]]
```

Kısıtlar

- m == board.length
- n == board[i].length
- 1 <= m, n <= 25
- board[i][j] 0 veya 1'dir.

Ek soru (Follow up)

Bunu yerinde çözebilir misin? Tahtanın aynı anda güncellenmesi gerektiğini unutma: bazı hücreleri önce güncelleyip sonra güncellenmiş değerleri başka hücreleri güncellemek için kullanamazsın. Bu soruda tahtayı 2 boyutlu bir dizi ile temsil ediyoruz. Prensipte tahta sonsuzdur; bu da aktif alan dizinin sınırına dayandığında (yani canlı hücreler kenara ulaştığında) sorun yaratır. Bu sorunları nasıl çözerdin?

39. Ransom Note

Fidye Notu | Seviye: Kolay | LeetCode #383
Orijinal soru: leetcode.com/problems/ransom-note/

Soru

İki dizgi **ransomNote** ve **magazine** veriliyor. **ransomNote**, **magazine**'deki harfler kullanılarak oluşturulabiliyorsa **true**, oluşturulamıyorsa **false** döndür.

magazine'deki her harf, **ransomNote** içinde yalnızca bir kez kullanılabilir.

Örnekler

Örnek 1

```
Girdi : ransomNote = "a", magazine = "b"
Çıktı : false
```

Örnek 2

```
Girdi : ransomNote = "aa", magazine = "ab"
Çıktı : false
```

Örnek 3

```
Girdi : ransomNote = "aa", magazine = "aab"
Çıktı : true
```

Kısıtlar

• 1 <= ransomNote.length, magazine.length <= 10^5
• ransomNote ve magazine küçük İngilizce harflerden oluşur.

40. Isomorphic Strings

İzomorfik Dizgiler | Seviye: Kolay | LeetCode #205
Orijinal soru: leetcode.com/problems/isomorphic-strings/

Soru

İki dizgi **s** ve **t** veriliyor. Bunların **izomorfik** olup olmadığını belirle.

s'deki karakterler değiştirilerek **t** elde edilebiliyorsa **s** ve **t** izomorfiktir. Bir karakterin tüm geçtiği yerler, karakterlerin sırası korunarak başka bir karakterle değiştirilmelidir. İki farklı karakter aynı karaktere eşlenemez, ancak bir karakter kendisine eşlenebilir.

Örnekler

Örnek 1

```
Girdi : s = "egg", t = "add"
Çıktı : true
```

s ve t şu şekilde aynı yapılabilir: 'e' karakteri 'a'ya eşlenir. 'g' karakteri 'd'ye eşlenir.

Örnek 2

```
Girdi : s = "foo", t = "bar"
Çıktı : false
```

s ve t aynı yapılamaz, çünkü 'o' karakterinin hem 'a'ya hem de 'r'ye eşlenmesi gerekir.

Örnek 3

```
Girdi : s = "paper", t = "title"
Çıktı : true
```

Kısıtlar

• $1 <= s.length <= 5 * 10^4$
• t.length == s.length
• s ve t herhangi geçerli ASCII karakterlerinden oluşur.

41. Word Pattern

Kelime Deseni | Seviye: Kolay | LeetCode #290
Orijinal soru: leetcode.com/problems/word-pattern/

Soru

Bir **pattern** (desen) ve bir **s** dizgisi veriliyor. **s**'in aynı desene uyup uymadığını bul.

Burada “uymak”, **pattern** içindeki bir harf ile **s** içindeki boş olmayan bir kelime arasında bir **birebir eşleme (bijection)** bulunması demektir. Yani:

• **pattern** içindeki her harf, **s** içindeki tam olarak bir benzersiz kelimeye eşlenir.
• **s** içindeki her benzersiz kelime, **pattern** içindeki tam olarak bir harfe eşlenir.
• Hiçbir iki harf aynı kelimeye, hiçbir iki kelime de aynı harfe eşlenmez.

Örnekler

Örnek 1

```
Girdi : pattern = "abba", s = "dog cat cat dog"
Çıktı : true
```

Eşleme şu şekilde kurulabilir: 'a' harfi “dog” kelimesine eşlenir. 'b' harfi “cat” kelimesine eşlenir.

Örnek 2

```
Girdi : pattern = "abba", s = "dog cat cat fish"
Çıktı : false
```

Örnek 3

```
Girdi : pattern = "aaaa", s = "dog cat cat dog"
Çıktı : false
```

Kısıtlar

• 1 <= pattern.length <= 300
• pattern yalnızca küçük İngilizce harflerden oluşur.
• 1 <= s.length <= 3000
• s yalnızca küçük İngilizce harflerden ve boşluk karakterinden ' ' oluşur.
• s başta veya sonda boşluk içermez.
• s içindeki tüm kelimeler tek bir boşlukla ayrılmıştır.

42. Valid Anagram

Geçerli Anagram | Seviye: Kolay | LeetCode #242
Orijinal soru: leetcode.com/problems/valid-anagram/

Soru

İki dizgi **s** ve **t** veriliyor. **t**, **s**'nin bir anagramıysa **true**, değilse **false** döndür.

Anagram, başka bir kelimenin veya ifadenin harflerinin yeniden düzenlenmesiyle oluşan ve genellikle tüm orijinal harflerin tam olarak bir kez kullanıldığı kelime ya da ifadedir.

Örnekler

Örnek 1

```
Girdi : s = "anagram", t = "nagaram"
Çıktı : true
```

Örnek 2

```
Girdi : s = "rat", t = "car"
Çıktı : false
```

Kısıtlar

• 1 <= s.length, t.length <= $5 * 10^4$
• s ve t küçük İngilizce harflerden oluşur.

Ek soru (Follow up)

Girdiler Unicode karakterler içeriyorsa ne olur? Çözümünü böyle bir duruma nasıl uyarlarsın?

43. Group Anagrams

Anagramları Grupla | Seviye: Orta | LeetCode #49
Orijinal soru: leetcode.com/problems/group-anagrams/

Soru

Bir **strs** dizgi dizisi veriliyor. Anagramları bir araya **grupla**. Cevabı herhangi bir sırada döndürebilirsin.

Örnekler

Örnek 1

```
Girdi : strs = ["eat","tea","tan","ate","nat","bat"]
Çıktı : [["bat"],["nat","tan"],["ate","eat","tea"]]
```

strs içinde “bat” oluşturacak şekilde yeniden düzenlenebilecek başka dizgi yoktur.
“nat” ve “tan” birbirinin anagramıdır, çünkü yeniden düzenlenerek birbirlerine dönüştürülebilirler.
“ate”, “eat” ve “tea” birbirinin anagramıdır, çünkü yeniden düzenlenerek birbirlerine dönüştürülebilirler.

Örnek 2

```
Girdi : strs = [""]
Çıktı : [[""]]
```

Örnek 3

```
Girdi : strs = ["a"]
Çıktı : [["a"]]
```

Kısıtlar

- 1 <= strs.length <= 10^4
- 0 <= strs[i].length <= 100
- strs[i] küçük İngilizce harflerden oluşur.

44. Two Sum

İki Toplam | Seviye: Kolay | LeetCode #1
Orijinal soru: leetcode.com/problems/two-sum/

Soru

Bir tamsayı dizisi **nums** ve bir tamsayı **target** veriliyor. Toplamı **target**'a eşit olan iki sayının **indekslerini** döndür.

Her girdinin tam olarak **bir** çözümü olduğunu varsayabilirsin ve aynı elemanı iki kez kullanamazsın.

Cevabı herhangi bir sırada döndürebilirsin.

Örnekler

Örnek 1

```
Girdi : nums = [2,7,11,15], target = 9
Çıktı : [0,1]
```

nums[0] + nums[1] == 9 olduğu için [0, 1] döndürürüz.

Örnek 2

```
Girdi : nums = [3,2,4], target = 6
Çıktı : [1,2]
```

Örnek 3

```
Girdi : nums = [3,3], target = 6
Çıktı : [0,1]
```

Kısıtlar

• 2 <= nums.length <= 10^4
• -10^9 <= nums[i] <= 10^9
• -10^9 <= target <= 10^9
• Yalnızca bir geçerli cevap vardır.

Ek soru (Follow up)

Zaman karmaşıklığı **O(n^2)**'den daha iyi bir algoritma bulabilir misin?

45. Happy Number

Mutlu Sayı | Seviye: Kolay | LeetCode #202
Orijinal soru: leetcode.com/problems/happy-number/

Soru

Bir **n** sayısının **mutlu** olup olmadığını belirleyen bir algoritma yaz.

Mutlu sayı, şu süreçle tanımlanan sayıdır:

• Herhangi bir pozitif tamsayıyla başla ve sayıyı, rakamlarının karelerinin toplamıyla değiştir.
• Sayı 1'e eşit olana kadar (1'de kalacaktır) ya da 1'i içermeyen bir döngüde sonsuza dek dönene kadar bu süreci tekrarla.
• Bu sürecin 1 ile bittiği sayılar mutludur.

n mutlu bir sayıysa **true**, değilse **false** döndür.

Örnekler

Örnek 1

```
Girdi : n = 19
Çıktı : true
```

$1^2 + 9^2 = 82$
$8^2 + 2^2 = 68$
$6^2 + 8^2 = 100$
$1^2 + 0^2 + 0^2 = 1$

Örnek 2

```
Girdi : n = 2
Çıktı : false
```

Kısıtlar

• $1 <= n <= 2^{31} - 1$

46. Contains Duplicate II

Tekrar Eden Eleman II | Seviye: Kolay | LeetCode #219
Orijinal soru: leetcode.com/problems/contains-duplicate-ii/

Soru

Bir tamsayı dizisi **nums** ve bir tamsayı **k** veriliyor. Dizide **nums[i] == nums[j]** ve **abs(i - j) <= k** olacak şekilde iki **farklı** indeks **i** ve **j** varsa **true**, yoksa **false** döndür.

Örnekler

Örnek 1

```
Girdi : nums = [1,2,3,1], k = 3
Çıktı : true
```

Örnek 2

```
Girdi : nums = [1,0,1,1], k = 1
Çıktı : true
```

Örnek 3

```
Girdi : nums = [1,2,3,1,2,3], k = 2
Çıktı : false
```

Kısıtlar

• $1 <= nums.length <= 10^5$
• $-10^9 <= nums[i] <= 10^9$
• $0 <= k <= 10^5$

47. Longest Consecutive Sequence

En Uzun Ardışık Dizi | Seviye: Orta | LeetCode #128
Orijinal soru: leetcode.com/problems/longest-consecutive-sequence/

Soru

Sırasız bir tamsayı dizisi **nums** veriliyor. En uzun **ardışık elemanlar dizisinin** uzunluğunu döndür.

O(n) zamanda çalışan bir algoritma yazmalısın.

Örnekler

Örnek 1

```
Girdi : nums = [100,4,200,1,3,2]
Çıktı : 4
```

En uzun ardışık eleman dizisi [1, 2, 3, 4]'tür. Bu yüzden uzunluğu 4'tür.

Örnek 2

```
Girdi : nums = [0,3,7,2,5,8,4,6,0,1]
Çıktı : 9
```

Örnek 3

```
Girdi : nums = [1,0,1,2]
Çıktı : 3
```

Kısıtlar

• 0 <= nums.length <= 10^5
• -10^9 <= nums[i] <= 10^9

48. Summary Ranges

Aralıkları Özetle | Seviye: Kolay | LeetCode #228
Orijinal soru: leetcode.com/problems/summary-ranges/

Soru

Sıralı ve benzersiz tamsayılardan oluşan bir **nums** dizisi veriliyor.

[a,b] aralığı, **a**'dan **b**'ye kadar (ikisi dahil) tüm tamsayıların kümesidir.

Dizideki tüm sayıları **tam olarak** kapsayan en küçük sıralı aralık listesini döndür. Yani **nums**'ın her elemanı aralıklardan tam olarak biri tarafından kapsanmalı ve aralıklardan birinde olup **nums**'ta olmayan hiçbir **x** tamsayısı bulunmamalıdır.

Listedeki her **[a,b]** aralığı şu şekilde yazılmalıdır:

• **a != b** ise **“a->b”**
• **a == b** ise **“a”**

Örnekler

Örnek 1

```
Girdi : nums = [0,1,2,4,5,7]
Çıktı : ["0->2","4->5","7"]
```

Aralıklar şunlardır:
[0,2] --> “0->2”
[4,5] --> “4->5”
[7,7] --> “7”

Örnek 2

```
Girdi : nums = [0,2,3,4,6,8,9]
Çıktı : ["0","2->4","6","8->9"]
```

Aralıklar şunlardır:
[0,0] --> “0”
[2,4] --> “2->4”
[6,6] --> “6”
[8,9] --> “8->9”

Kısıtlar

• 0 <= nums.length <= 20
• -2^{31} <= nums[i] <= 2^{31} - 1
• nums'ın tüm değerleri benzersizdir.
• nums artan sırada sıralıdır.

49. Merge Intervals

Aralıkları Birleştir | Seviye: Orta | LeetCode #56
Orijinal soru: leetcode.com/problems/merge-intervals/

Soru

intervals[i] = [start$_i$, end$_i$] olan bir **intervals** aralık dizisi veriliyor. Üst üste binen (overlapping) tüm aralıkları birleştir ve girdideki tüm aralıkları kapsayan, birbiriyle çakışmayan aralıklardan oluşan bir dizi döndür.

Örnekler

Örnek 1

```
Girdi : intervals = [[1,3],[2,6],[8,10],[15,18]]
Çıktı : [[1,6],[8,10],[15,18]]
```

[1,3] ve [2,6] aralıkları üst üste bindiği için bunlar [1,6] olarak birleştirilir.

Örnek 2

```
Girdi : intervals = [[1,4],[4,5]]
Çıktı : [[1,5]]
```

[1,4] ve [4,5] aralıkları üst üste binen aralıklar olarak kabul edilir.

Örnek 3

```
Girdi : intervals = [[4,7],[1,4]]
Çıktı : [[1,7]]
```

[1,4] ve [4,7] aralıkları üst üste binen aralıklar olarak kabul edilir.

Kısıtlar

- 1 <= intervals.length <= 10^4
- intervals[i].length == 2
- 0 <= start$_i$ <= end$_i$ <= 10^4

50. Insert Interval

Aralık Ekle | Seviye: Orta | LeetCode #57
Orijinal soru: leetcode.com/problems/insert-interval/

Soru

Birbiriyle çakışmayan aralıklardan oluşan bir **intervals** dizisi veriliyor; **intervals[i] = [start$_i$, end$_i$]**, **i**. aralığın başlangıcını ve bitişini gösterir ve **intervals**, **start$_i$** değerine göre artan sırada sıralıdır. Ayrıca başka bir aralığın başlangıcını ve bitişini temsil eden **newInterval = [start, end]** aralığı da veriliyor.

newInterval'ı **intervals** içine öyle ekle ki **intervals** hâlâ **start$_i$** değerine göre artan sırada sıralı olsun ve hâlâ üst üste binen aralık içermesin (gerekirse üst üste binen aralıkları birleştir).

Ekleme sonrası **intervals**'ı döndür.

Not: **intervals**'ı yerinde değiştirmen gerekmez. Yeni bir dizi oluşturup onu döndürebilirsin.

Örnekler

Örnek 1

```
Girdi : intervals = [[1,3],[6,9]], newInterval = [2,5]
Çıktı : [[1,5],[6,9]]
```

Örnek 2

```
Girdi : intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
Çıktı : [[1,2],[3,10],[12,16]]
```

Yeni aralık [4,8], [3,5], [6,7] ve [8,10] aralıklarıyla üst üste biner.

Kısıtlar

- 0 <= intervals.length <= 10^4
- intervals[i].length == 2
- 0 <= start$_i$ <= end$_i$ <= 10^5
- intervals, start$_i$ değerine göre artan sırada sıralıdır.
- newInterval.length == 2
- 0 <= start <= end <= 10^5

51. Minimum Number of Arrows to Burst Balloons

Balonları Patlatmak İçin Minimum Ok Sayısı | Seviye: Orta | LeetCode #452
Orijinal soru: leetcode.com/problems/minimum-number-of-arrows-to-burst-balloons/

Soru

XY düzlemini temsil eden düz bir duvara bantlanmış bazı küre şeklinde balonlar var. Balonlar, **points** adlı 2 boyutlu bir tamsayı dizisiyle temsil edilir; **points[i] = [x_{start}, x_{end}]**, yatay çapı x_{start} ile x_{end} arasında uzanan bir balonu gösterir. Balonların tam y koordinatlarını bilmiyorsun.

Oklar, x ekseni üzerindeki farklı noktalardan dikey olarak (pozitif y yönünde) yukarı doğru atılabilir. x_{start} ve x_{end} sınırlarına sahip bir balon, **x** noktasından atılan bir okla, **x_{start} <= x <= x_{end}** ise patlar. Atılabilecek ok sayısında bir sınır yoktur. Atılan ok sonsuza dek yukarı gitmeye devam eder ve yolundaki tüm balonları patlatır.

points dizisi verildiğinde, tüm balonları patlatmak için atılması gereken **minimum ok sayısını** döndür.

Örnekler

Örnek 1

```
Girdi : points = [[10,16],[2,8],[1,6],[7,12]]
Çıktı : 2
```

Balonlar 2 okla patlatılabilir:

• x = 6 noktasına bir ok at; [2,8] ve [1,6] balonlarını patlatır.
• x = 11 noktasına bir ok at; [10,16] ve [7,12] balonlarını patlatır.

Örnek 2

```
Girdi : points = [[1,2],[3,4],[5,6],[7,8]]
Çıktı : 4
```

Her balon için bir ok atılmalı; toplam 4 ok.

Örnek 3

```
Girdi : points = [[1,2],[2,3],[3,4],[4,5]]
Çıktı : 2
```

• x = 2 noktasına bir ok at; [1,2] ve [2,3] balonlarını patlatır.
• x = 4 noktasına bir ok at; [3,4] ve [4,5] balonlarını patlatır.

Kısıtlar

• 1 <= points.length <= 10^5
• points[i].length == 2
• -2^{31} <= x_{start} < x_{end} <= 2^{31} - 1

52. Valid Parentheses

Geçerli Parantezler | Seviye: Kolay | LeetCode #20
Orijinal soru: leetcode.com/problems/valid-parentheses/

Soru

Yalnızca '**(**', '**)**', '**{**', '**}**', '**[**' ve '**]**' karakterlerini içeren bir **s** dizgisi veriliyor. Girdi dizgisinin geçerli olup olmadığını belirle.

Bir girdi dizgisi şu durumlarda **geçerlidir**:

1. Açılan parantezler aynı türdeki parantezlerle kapatılmalıdır.
2. Açılan parantezler doğru sırayla kapatılmalıdır.
3. Her kapanan parantezin aynı türde karşılık gelen bir açılan paranteze sahip olması gerekir.

Örnekler

Örnek 1

```
Girdi : s = "()"
Çıktı : true
```

Örnek 2

```
Girdi : s = "()[]{}"
Çıktı : true
```

Örnek 3

```
Girdi : s = "(]"
Çıktı : false
```

Örnek 4

```
Girdi : s = "([])"
Çıktı : true
```

Örnek 5

```
Girdi : s = "([)]"
Çıktı : false
```

Kısıtlar

• 1 <= s.length <= 10^4
• s yalnızca '()[]{}' parantezlerinden oluşur.

53. Simplify Path

Yolu Sadeleştir | Seviye: Orta | LeetCode #71
Orijinal soru: leetcode.com/problems/simplify-path/

Soru

Unix tarzı bir dosya sisteminin, her zaman bir eğik çizgiyle **'/'** başlayan mutlak (absolute) yolu veriliyor. Görevin, bu mutlak yolu sadeleştirilmiş **kanonik yoluna** dönüştürmek.

Unix tarzı bir dosya sisteminin kuralları şunlardır:

• Tek nokta **'.'** geçerli dizini temsil eder.
• Çift nokta **'..'** bir önceki/üst dizini temsil eder.
• **'//'** ve **'///'** gibi art arda gelen birden fazla eğik çizgi tek bir eğik çizgi **'/'** olarak kabul edilir.
• Yukarıdaki kurallara uymayan herhangi bir nokta dizisi geçerli bir dizin veya dosya adı olarak ele alınır. Örneğin **'...'** ve **'....'** geçerli dizin veya dosya adlarıdır.

Sadeleştirilmiş kanonik yol şu kurallara uymalıdır:

• Yol tek bir eğik çizgiyle **'/'** başlamalıdır.
• Yoldaki dizinler tam olarak bir eğik çizgiyle **'/'** ayrılmalıdır.
• Kök dizin olmadıkça yol bir eğik çizgiyle **'/'** bitmemelidir.
• Yol, geçerli veya üst dizini belirtmek için kullanılan tek veya çift nokta (**'.'** ve **'..'**) içermemelidir.

Sadeleştirilmiş kanonik yolu döndür.

Örnekler

Örnek 1

```
Girdi : path = "/home/"
Çıktı : "/home"
```

Sondaki eğik çizgi kaldırılmalıdır.

Örnek 2

```
Girdi : path = "/home//foo/"
Çıktı : "/home/foo"
```

Art arda gelen birden fazla eğik çizgi tek eğik çizgiyle değiştirilir.

Örnek 3

```
Girdi : path = "/home/user/Documents/../Pictures"
Çıktı : "/home/user/Pictures"
```

Çift nokta “..” bir üst seviyedeki dizini (üst dizin) belirtir.

Örnek 4

```
Girdi : path = "/../"
Çıktı : "/"
```

Kök dizinden bir seviye yukarı çıkmak mümkün değildir.

Örnek 5

```
Girdi : path = "/.../a/../b/c/../d/./"
Çıktı : "/.../b/d"
```

Bu problemde “...” geçerli bir dizin adıdır.

Kısıtlar

• 1 <= path.length <= 3000
• path İngilizce harflerden, rakamlardan, nokta '.', eğik çizgi '/' veya '_' karakterlerinden oluşur.
• path geçerli bir mutlak Unix yoludur.

54. Min Stack

Minimum Yığın | Seviye: Orta | LeetCode #155
Orijinal soru: leetcode.com/problems/min-stack/

Soru

push, **pop**, **top** işlemlerini destekleyen ve **minimum elemanı sabit zamanda** getirebilen bir yığın (stack) tasarla.

MinStack sınıfını gerçekle:

- **MinStack()**: yığın nesnesini başlatır.
- **void push(int val)**: **val** elemanını yığına ekler.
- **void pop()**: yığının en üstündeki elemanı siler.
- **int top()**: yığının en üstündeki elemanı getirir.
- **int getMin()**: yığındaki minimum elemanı getirir.

Her fonksiyon için **O(1)** zaman karmaşıklığına sahip bir çözüm gerçekleştirmelisin.

Örnekler

Örnek 1

```
Girdi :
["MinStack","push","push","push","getMin","pop","top","getMin"]
        [[],[-2],[0],[-3],[],[],[],[]]
Çıktı :
        [null,null,null,null,-3,null,0,-2]
```

- **MinStack()** ile yığın oluşturulur; sırasıyla **push(-2)**, **push(0)**, **push(-3)** çağrılır.
- **getMin()**: -3 döner.
- **pop()**: -3 silinir.
- **top()**: 0 döner.
- **getMin()**: -2 döner.

Kısıtlar

- $-2^{31} <= val <= 2^{31} - 1$
- **pop**, **top** ve **getMin** işlemleri her zaman boş olmayan yığınlar üzerinde çağrılacaktır.
- **push**, **pop**, **top** ve **getMin** için en fazla $3 * 10^4$ çağrı yapılacaktır.

55. Evaluate Reverse Polish Notation

Ters Lehçe Notasyonunu Değerlendir | Seviye: Orta | LeetCode #150
Orijinal soru: leetcode.com/problems/evaluate-reverse-polish-notation/

Soru

Ters Lehçe Notasyonu'nda (Reverse Polish Notation) bir aritmetik ifadeyi temsil eden bir **tokens** dizgi dizisi veriliyor. İfadeyi değerlendir ve ifadenin değerini temsil eden bir tamsayı döndür.

Dikkat:

• Geçerli operatörler **'+'**, **'-'**, **'*'** ve **'/'**'dir.
• Her işlenen (operand) bir tamsayı ya da başka bir ifade olabilir.
• İki tamsayı arasındaki bölme her zaman **sıfıra doğru** keser (truncate toward zero).
• Sıfıra bölme olmayacaktır.
• Girdi, ters Lehçe notasyonunda geçerli bir aritmetik ifadeyi temsil eder.
• Cevap ve tüm ara hesaplamalar 32 bitlik bir tamsayıyla gösterilebilir.

Örnekler

Örnek 1

```
Girdi : tokens = ["2","1","+","3","*"]
Çıktı : 9
```
((2 + 1) * 3) = 9

Örnek 2

```
Girdi : tokens = ["4","13","5","/","+"]
Çıktı : 6
```
(4 + (13 / 5)) = 6

Örnek 3

```
Girdi : tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]
Çıktı : 22
```
((10 * (6 / ((9 + 3) * -11))) + 17) + 5
= ((10 * (6 / (12 * -11))) + 17) + 5
= ((10 * (6 / -132)) + 17) + 5
= ((10 * 0) + 17) + 5
= (0 + 17) + 5
= 17 + 5
= 22

Kısıtlar

• 1 <= tokens.length <= 10^4
• tokens[i] ya bir operatördür: “+”, “-”, “*” veya “/”; ya da [-200, 200] aralığında bir tamsayıdır.

56. Basic Calculator

Temel Hesap Makinesi | Seviye: Zor | LeetCode #224
Orijinal soru: leetcode.com/problems/basic-calculator/

Soru

Geçerli bir ifadeyi temsil eden bir **s** dizgisi veriliyor. İfadeyi değerlendiren temel bir hesap makinesi gerçekleştir ve değerlendirmenin sonucunu döndür.

Not: eval() gibi, dizgileri matematiksel ifade olarak değerlendiren hiçbir yerleşik fonksiyonu kullanamazsın.

Örnekler

Örnek 1

```
Girdi : s = "1 + 1"
Çıktı : 2
```

Örnek 2

```
Girdi : s = " 2-1 + 2 "
Çıktı : 3
```

Örnek 3

```
Girdi : s = "(1+(4+5+2)-3)+(6+8)"
Çıktı : 23
```

Kısıtlar

• 1 <= s.length <= $3 * 10^5$
• s rakamlardan, '+', '-', '(', ')' ve ' ' karakterlerinden oluşur.
• s geçerli bir ifadeyi temsil eder.
• '+' tekli (unary) operatör olarak kullanılmaz (yani “+1” ve “+(2 + 3)” geçersizdir).
• '-' tekli operatör olarak kullanılabilir (yani “-1” ve “-(2 + 3)” geçerlidir).
• Girdide art arda iki operatör olmayacaktır.
• Her sayı ve ara hesaplama işaretli 32 bitlik bir tamsayıya sığar.

57. Linked List Cycle

Bağlı Listede Döngü | Seviye: Kolay | LeetCode #141
Orijinal soru: leetcode.com/problems/linked-list-cycle/

Soru

Bir bağlı listenin başı **head** veriliyor. Bağlı listede **döngü (cycle)** olup olmadığını belirle.

Bir bağlı listede, **next** işaretçisi sürekli takip edilerek tekrar ulaşılabilen bir düğüm varsa listede döngü vardır. Dahili olarak **pos**, kuyruğun (tail) **next** işaretçisinin bağlandığı düğümün indeksini göstermek için kullanılır. **pos**'un bir parametre olarak **verilmediğine** dikkat et.

Bağlı listede döngü varsa **true**, yoksa **false** döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : head = [3,2,0,-4], pos = 1
Çıktı : true
```

Bağlı listede, kuyruğun 1. düğüme (0 tabanlı indeks) bağlandığı bir döngü var.

Örnek 2

```
Girdi : head = [1,2], pos = 0
Çıktı : true
```

Bağlı listede, kuyruğun 0. düğüme bağlandığı bir döngü var.

Örnek 3

```
Girdi : head = [1], pos = -1
Çıktı : false
```

Bağlı listede döngü yok.

Kısıtlar

- Listedeki düğüm sayısı $[0, 10^4]$ aralığındadır.
- $-10^5 <= Node.val <= 10^5$
- pos, -1'dir ya da bağlı listede geçerli bir indekstir.

Ek soru (Follow up)

Bunu **O(1)** (yani sabit) bellekle çözebilir misin?

58. Add Two Numbers

İki Sayıyı Topla | Seviye: Orta | LeetCode #2
Orijinal soru: leetcode.com/problems/add-two-numbers/

Soru

Negatif olmayan iki tamsayıyı temsil eden, boş olmayan iki bağlı liste veriliyor. Rakamlar **ters sırada** saklanır ve her düğüm tek bir rakam içerir.

İki sayıyı topla ve toplamı bir bağlı liste olarak döndür.

İki sayının, 0 sayısının kendisi dışında, başında sıfır içermediğini varsayabilirsin.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : l1 = [2,4,3], l2 = [5,6,4]
Çıktı : [7,0,8]
```

342 + 465 = 807.

Örnek 2

```
Girdi : l1 = [0], l2 = [0]
Çıktı : [0]
```

Örnek 3

```
Girdi : l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]
Çıktı : [8,9,9,9,0,0,0,1]
```

Kısıtlar

- Her bağlı listedeki düğüm sayısı [1, 100] aralığındadır.
- 0 <= Node.val <= 9
- Listenin, başında sıfır olmayan bir sayıyı temsil ettiği garanti edilmiştir.

59. Merge Two Sorted Lists

Sıralı İki Listeyi Birleştir | Seviye: Kolay | LeetCode #21
Orijinal soru: leetcode.com/problems/merge-two-sorted-lists/

Soru

Sıralı iki bağlı listenin başları **list1** ve **list2** veriliyor.

İki listeyi tek bir sıralı listede birleştir. Liste, ilk iki listenin düğümleri birbirine eklenerek (splicing) oluşturulmalıdır.

Birleştirilmiş bağlı listenin başını döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : list1 = [1,2,4], list2 = [1,3,4]
Çıktı : [1,1,2,3,4,4]
```

Örnek 2

```
Girdi : list1 = [], list2 = []
Çıktı : []
```

Örnek 3

```
Girdi : list1 = [], list2 = [0]
Çıktı : [0]
```

Kısıtlar

• İki listedeki düğüm sayısı [0, 50] aralığındadır.
• -100 <= Node.val <= 100
• list1 ve list2 azalmayan sırada sıralıdır.

60. Copy List with Random Pointer

Rastgele İşaretçili Listeyi Kopyala | Seviye: Orta | LeetCode #138
Orijinal soru: leetcode.com/problems/copy-list-with-random-pointer/

Soru

Uzunluğu **n** olan bir bağlı liste veriliyor; her düğüm, listedeki herhangi bir düğümü ya da **null**'ı gösterebilen ek bir **random** işaretçisi içeriyor.

Listenin bir **derin kopyasını (deep copy)** oluştur. Derin kopya tam olarak **n** adet yepyeni düğümden oluşmalı; her yeni düğümün değeri, karşılık gelen orijinal düğümün değerine eşit olmalıdır. Yeni düğümlerin hem **next** hem **random** işaretçileri, orijinal listedeki ve kopya listedeki işaretçiler aynı liste durumunu temsil edecek şekilde kopya listedeki yeni düğümleri göstermelidir. Yeni listedeki hiçbir işaretçi orijinal listedeki düğümleri göstermemelidir.

Örneğin orijinal listede **X.random --> Y** olan iki düğüm X ve Y varsa, kopya listedeki karşılık gelen x ve y düğümleri için **x.random --> y** olmalıdır.

Kopyalanmış bağlı listenin başını döndür.

Bağlı liste girdi/çıktıda **n** düğümlük bir liste olarak gösterilir. Her düğüm bir **[val, random_index]** çiftiyle temsil edilir:

• **val**: **Node.val** değerini gösteren bir tamsayı.
• **random_index**: **random** işaretçisinin gösterdiği düğümün indeksi (0 ile n-1 arası) ya da hiçbir düğümü göstermiyorsa **null**.

Kodun yalnızca orijinal bağlı listenin başı olan **head**'i alacaktır.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : head = [[7,null],[13,0],[11,4],[10,2],[1,0]]
Çıktı : [[7,null],[13,0],[11,4],[10,2],[1,0]]
```

Örnek 2

```
Girdi : head = [[1,1],[2,1]]
Çıktı : [[1,1],[2,1]]
```

Örnek 3

```
Girdi : head = [[3,null],[3,0],[3,null]]
Çıktı : [[3,null],[3,0],[3,null]]
```

Kısıtlar

• 0 <= n <= 1000
• -10^4 <= Node.val <= 10^4
• Node.random ya null'dır ya da bağlı listedeki bir düğümü gösterir.

61. Reverse Linked List II

Bağlı Listeyi Ters Çevir II | Seviye: Orta | LeetCode #92
Orijinal soru: leetcode.com/problems/reverse-linked-list-ii/

Soru

Tek yönlü bir bağlı listenin başı **head** ve **left <= right** olan iki tamsayı **left** ve **right** veriliyor. Listenin **left** konumundan **right** konumuna kadar olan düğümlerini ters çevir ve ters çevrilmiş listeyi döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : head = [1,2,3,4,5], left = 2, right = 4
Çıktı : [1,4,3,2,5]
```

Örnek 2

```
Girdi : head = [5], left = 1, right = 1
Çıktı : [5]
```

Kısıtlar

• Listedeki düğüm sayısı n'dir.
• 1 <= n <= 500
• -500 <= Node.val <= 500
• 1 <= left <= right <= n

Ek soru (Follow up)

Bunu tek geçişte (one pass) yapabilir misin?

62. Reverse Nodes in k-Group

k'lı Gruplarda Düğümleri Ters Çevir | Seviye: Zor | LeetCode #25
Orijinal soru: leetcode.com/problems/reverse-nodes-in-k-group/

Soru

Bir bağlı listenin başı **head** veriliyor. Listenin düğümlerini **k** düğümlük gruplar halinde ters çevir ve değiştirilmiş listeyi döndür.

k, listenin uzunluğundan küçük veya ona eşit pozitif bir tamsayıdır. Düğüm sayısı **k**'nın katı değilse, sonda kalan düğümler olduğu gibi kalmalıdır.

Listenin düğümlerinin değerlerini değiştiremezsin; yalnızca düğümlerin kendisi değiştirilebilir.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : head = [1,2,3,4,5], k = 2
Çıktı : [2,1,4,3,5]
```

Örnek 2

```
Girdi : head = [1,2,3,4,5], k = 3
Çıktı : [3,2,1,4,5]
```

Kısıtlar

• Listedeki düğüm sayısı n'dir.
• 1 <= k <= n <= 5000
• 0 <= Node.val <= 1000

Ek soru (Follow up)

Problemi **O(1)** ekstra bellekle çözebilir misin?

63. Remove Nth Node From End of List

Listenin Sonundan N. Düğümü Sil | Seviye: Orta | LeetCode #19
Orijinal soru: leetcode.com/problems/remove-nth-node-from-end-of-list/

Soru

Bir bağlı listenin başı **head** veriliyor. Listenin sondan **n**. düğümünü sil ve listenin başını döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : head = [1,2,3,4,5], n = 2
Çıktı : [1,2,3,5]
```

Örnek 2

```
Girdi : head = [1], n = 1
Çıktı : []
```

Örnek 3

```
Girdi : head = [1,2], n = 1
Çıktı : [1]
```

Kısıtlar

• Listedeki düğüm sayısı sz'dir.
• 1 <= sz <= 30
• 0 <= Node.val <= 100
• 1 <= n <= sz

Ek soru (Follow up)

Bunu tek geçişte yapabilir misin?

64. Remove Duplicates from Sorted List II

Sıralı Listeden Tekrarları Sil II | Seviye: Orta | LeetCode #82
Orijinal soru: leetcode.com/problems/remove-duplicates-from-sorted-list-ii/

Soru

Sıralı bir bağlı listenin başı **head** veriliyor. Tekrar eden sayılara sahip **tüm düğümleri** sil ve orijinal listeden yalnızca benzersiz (distinct) sayıları bırak. Bağlı listeyi yine sıralı olarak döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : head = [1,2,3,3,4,4,5]
Çıktı : [1,2,5]
```

Örnek 2

```
Girdi : head = [1,1,1,2,3]
Çıktı : [2,3]
```

Kısıtlar

- Listedeki düğüm sayısı [0, 300] aralığındadır.
- -100 <= Node.val <= 100
- Listenin artan sırada sıralı olduğu garanti edilmiştir.

65. Rotate List

Listeyi Döndür | Seviye: Orta | LeetCode #61
Orijinal soru: leetcode.com/problems/rotate-list/

Soru

Bir bağlı listenin başı **head** veriliyor. Listeyi **k** yer sağa döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : head = [1,2,3,4,5], k = 2
Çıktı : [4,5,1,2,3]
```

Örnek 2

```
Girdi : head = [0,1,2], k = 4
Çıktı : [2,0,1]
```

Kısıtlar

- Listedeki düğüm sayısı [0, 500] aralığındadır.
- -100 <= Node.val <= 100
- $0 <= k <= 2 * 10^9$

66. Partition List

Listeyi Bölümle | Seviye: Orta | LeetCode #86
Orijinal soru: leetcode.com/problems/partition-list/

Soru

Bir bağlı listenin başı **head** ve bir **x** değeri veriliyor. Listeyi öyle bölümle (partition) ki **x**'ten küçük tüm düğümler, **x**'e eşit veya ondan büyük düğümlerden önce gelsin.

İki bölümün her birinde düğümlerin orijinal göreli sırasını korumalısın.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : head = [1,4,3,2,5,2], x = 3
Çıktı : [1,2,2,4,3,5]
```

Örnek 2

```
Girdi : head = [2,1], x = 2
Çıktı : [1,2]
```

Kısıtlar

• Listedeki düğüm sayısı [0, 200] aralığındadır.
• -100 <= Node.val <= 100
• -200 <= x <= 200

67. LRU Cache

LRU Önbellek | Seviye: Orta | LeetCode #146
Orijinal soru: leetcode.com/problems/lru-cache/

Soru

En Az Yakın Zamanda Kullanılan (**LRU**, Least Recently Used) önbelleğin kısıtlarına uyan bir veri yapısı tasarla.

LRUCache sınıfını gerçekle:

• **LRUCache(int capacity)**: LRU önbelleğini pozitif bir **capacity** boyutuyla başlatır.
• **int get(int key)**: **key** varsa değerini, yoksa **-1** döndürür.
• **void put(int key, int value)**: **key** varsa değerini günceller. Yoksa anahtar-değer çiftini önbelleğe ekler. Bu işlem sonucunda anahtar sayısı **capacity**'yi aşarsa, en az yakın zamanda kullanılan anahtarı **çıkarır (evict)**.

get ve **put** fonksiyonlarının her biri **ortalama O(1)** zaman karmaşıklığında çalışmalıdır.

Örnekler

Örnek 1

```
Girdi :
["LRUCache","put","put","get","put","get","put","get","get","get"]
        [[2],[1,1],[2,2],[1],[3,3],[2],[4,4],[1],[3],[4]]
Çıktı :
        [null,null,null,1,null,-1,null,-1,3,4]
```

• **LRUCache(2)**: önbellek oluşturulur.
• **put(1, 1)**: önbellek {1=1}
• **put(2, 2)**: önbellek {1=1, 2=2}
• **get(1)**: 1 döner.
• **put(3, 3)**: LRU anahtar 2'ydi, 2 çıkarılır; önbellek {1=1, 3=3}
• **get(2)**: -1 döner (bulunamadı).
• **put(4, 4)**: LRU anahtar 1'di, 1 çıkarılır; önbellek {4=4, 3=3}
• **get(1)**: -1 döner (bulunamadı).
• **get(3)**: 3 döner.
• **get(4)**: 4 döner.

Kısıtlar

• 1 <= capacity <= 3000
• 0 <= key <= 10^4
• 0 <= value <= 10^5
• **get** ve **put** için en fazla 2 * 10^5 çağrı yapılacaktır.

68. Maximum Depth of Binary Tree

İkili Ağacın Maksimum Derinliği | Seviye: Kolay | LeetCode #104
Orijinal soru: leetcode.com/problems/maximum-depth-of-binary-tree/

Soru

Bir ikili ağacın kökü **root** veriliyor. Ağacın **maksimum derinliğini** döndür.

Bir ikili ağacın maksimum derinliği, kök düğümden en uzak yaprak düğüme kadar olan en uzun yol boyunca bulunan düğüm sayısıdır.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [3,9,20,null,null,15,7]
Çıktı : 3
```

Örnek 2

```
Girdi : root = [1,null,2]
Çıktı : 2
```

Kısıtlar

• Ağaçtaki düğüm sayısı [0, 10^4] aralığındadır.
• -100 <= Node.val <= 100

69. Same Tree

Aynı Ağaç | Seviye: Kolay | LeetCode #100
Orijinal soru: leetcode.com/problems/same-tree/

Soru

İki ikili ağacın kökleri **p** ve **q** veriliyor. Bunların aynı olup olmadığını kontrol eden bir fonksiyon yaz.

İki ikili ağaç, yapısal olarak özdeşse ve düğümleri aynı değerlere sahipse **aynı** kabul edilir.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : p = [1,2,3], q = [1,2,3]
Çıktı : true
```

Örnek 2

```
Girdi : p = [1,2], q = [1,null,2]
Çıktı : false
```

Örnek 3

```
Girdi : p = [1,2,1], q = [1,1,2]
Çıktı : false
```

Kısıtlar

• İki ağaçtaki düğüm sayısı [0, 100] aralığındadır.
• -10^4 <= Node.val <= 10^4

70. Invert Binary Tree

İkili Ağacı Ters Çevir | Seviye: Kolay | LeetCode #226
Orijinal soru: leetcode.com/problems/invert-binary-tree/

Soru

Bir ikili ağacın kökü **root** veriliyor. Ağacı **ters çevir** (invert) ve kökünü döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [4,2,7,1,3,6,9]
Çıktı : [4,7,2,9,6,3,1]
```

Örnek 2

```
Girdi : root = [2,1,3]
Çıktı : [2,3,1]
```

Örnek 3

```
Girdi : root = []
Çıktı : []
```

Kısıtlar

• Ağaçtaki düğüm sayısı [0, 100] aralığındadır.
• -100 <= Node.val <= 100

71. Symmetric Tree

Simetrik Ağaç | Seviye: Kolay | LeetCode #101
Orijinal soru: leetcode.com/problems/symmetric-tree/

Soru

Bir ikili ağacın kökü **root** veriliyor. Ağacın kendisinin ayna görüntüsü olup olmadığını (yani merkezine göre simetrik olup olmadığını) kontrol et.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [1,2,2,3,4,4,3]
Çıktı : true
```

Örnek 2

```
Girdi : root = [1,2,2,null,3,null,3]
Çıktı : false
```

Kısıtlar

• Ağaçtaki düğüm sayısı [1, 1000] aralığındadır.
• -100 <= Node.val <= 100

Ek soru (Follow up)

Problemi hem özyinelemeli (recursive) hem de yinelemeli (iterative) olarak çözebilir misin?

72. Construct Binary Tree from Preorder and Inorder Traversal

Preorder ve Inorder Dolaşımından İkili Ağaç Oluştur | Seviye: Orta | LeetCode #105
Orijinal soru: leetcode.com/problems/construct-binary-tree-from-preorder-and-inorder-traversal/

Soru

İki tamsayı dizisi **preorder** ve **inorder** veriliyor; **preorder**, bir ikili ağacın preorder (ön sıra) dolaşımı, **inorder** ise aynı ağacın inorder (ara sıra) dolaşımıdır. İkili ağacı oluştur ve döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
Çıktı : [3,9,20,null,null,15,7]
```

Örnek 2

```
Girdi : preorder = [-1], inorder = [-1]
Çıktı : [-1]
```

Kısıtlar

- 1 <= preorder.length <= 3000
- inorder.length == preorder.length
- -3000 <= preorder[i], inorder[i] <= 3000
- preorder ve inorder benzersiz değerlerden oluşur.
- inorder'daki her değer preorder'da da bulunur.
- preorder'ın ağacın preorder dolaşımı olduğu garanti edilmiştir.
- inorder'ın ağacın inorder dolaşımı olduğu garanti edilmiştir.

73. Construct Binary Tree from Inorder and Postorder Traversal

Inorder ve Postorder Dolaşımından İkili Ağaç Oluştur | Seviye: Orta | LeetCode #106
Orijinal soru: leetcode.com/problems/construct-binary-tree-from-inorder-and-postorder-traversal/

Soru

İki tamsayı dizisi **inorder** ve **postorder** veriliyor; **inorder**, bir ikili ağacın inorder (ara sıra) dolaşımı, **postorder** ise aynı ağacın postorder (son sıra) dolaşımıdır. İkili ağacı oluştur ve döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]
Çıktı : [3,9,20,null,null,15,7]
```

Örnek 2

```
Girdi : inorder = [-1], postorder = [-1]
Çıktı : [-1]
```

Kısıtlar

- 1 <= inorder.length <= 3000
- postorder.length == inorder.length
- -3000 <= inorder[i], postorder[i] <= 3000
- inorder ve postorder benzersiz değerlerden oluşur.
- postorder'daki her değer inorder'da da bulunur.
- inorder'ın ağacın inorder dolaşımı olduğu garanti edilmiştir.
- postorder'ın ağacın postorder dolaşımı olduğu garanti edilmiştir.

74. Populating Next Right Pointers in Each Node II

Her Düğümdeki Sağ İşaretçileri Doldur II | Seviye: Orta | LeetCode #117
Orijinal soru: leetcode.com/problems/populating-next-right-pointers-in-each-node-ii/

Soru

Bir ikili ağaç veriliyor:

```
struct Node {
  int val;
  Node *left;
  Node *right;
  Node *next;
}
```

Her **next** işaretçisini, bir sonraki sağ düğümü gösterecek şekilde doldur. Sağda bir sonraki düğüm yoksa **next** işaretçisi **NULL** olarak ayarlanmalıdır.

Başlangıçta tüm **next** işaretçileri **NULL**'dır.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [1,2,3,4,5,null,7]
Çıktı : [1,#,2,3,#,4,5,7,#]
```

Yukarıdaki ikili ağaç (Şekil A) verildiğinde, fonksiyonun her **next** işaretçisini Şekil B'deki gibi bir sonraki sağ düğümü gösterecek şekilde doldurması gerekir. Serileştirilmiş çıktı, **next** işaretçileriyle bağlı düğümlerin seviye sırasına göre listesidir; '#' her seviyenin sonunu gösterir.

Örnek 2

```
Girdi : root = []
Çıktı : []
```

Kısıtlar

• Ağaçtaki düğüm sayısı [0, 6000] aralığındadır.
• -100 <= Node.val <= 100

Ek soru (Follow up)

Yalnızca sabit (constant) ekstra bellek kullanabilirsin. Özyinelemeli (recursive) yaklaşım kabul edilir; bu problemde örtük yığın (stack) belleğinin ekstra bellek sayılmadığını varsayabilirsin.

75. Flatten Binary Tree to Linked List

İkili Ağacı Bağlı Listeye Düzleştir | Seviye: Orta | LeetCode #114
Orijinal soru: leetcode.com/problems/flatten-binary-tree-to-linked-list/

Soru

Bir ikili ağacın kökü **root** veriliyor. Ağacı bir "bağlı listeye" düzleştir (flatten):

• "Bağlı liste" aynı **TreeNode** sınıfını kullanmalı; **right** alt işaretçisi listedeki bir sonraki düğümü göstermeli ve **left** alt işaretçisi her zaman **null** olmalıdır.
• "Bağlı liste", ikili ağacın **pre-order** (ön sıra) dolaşımıyla aynı sırada olmalıdır.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [1,2,5,3,4,null,6]
Çıktı : [1,null,2,null,3,null,4,null,5,null,6]
```

Örnek 2

```
Girdi : root = []
Çıktı : []
```

Örnek 3

```
Girdi : root = [0]
Çıktı : [0]
```

Kısıtlar

• Ağaçtaki düğüm sayısı [0, 2000] aralığındadır.
• -100 <= Node.val <= 100

Ek soru (Follow up)

Ağacı yerinde (**O(1)** ekstra bellekle) düzleştirebilir misin?

76. Path Sum

Yol Toplamı | Seviye: Kolay | LeetCode #112
Orijinal soru: leetcode.com/problems/path-sum/

Soru

Bir ikili ağacın kökü **root** ve bir tamsayı **targetSum** veriliyor. Ağaçta, yol boyunca tüm değerlerin toplamı **targetSum**'a eşit olan bir **kökten yaprağa** yol varsa **true**, yoksa **false** döndür.

Yaprak, çocuğu olmayan düğümdür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22
Çıktı : true
```

Hedef toplama sahip kökten yaprağa yol görselde gösterilmiştir.

Örnek 2

```
Girdi : root = [1,2,3], targetSum = 5
Çıktı : false
```

Ağaçta iki kökten yaprağa yol var:
(1 --> 2): toplam 3.
(1 --> 3): toplam 4.
Toplamı 5 olan kökten yaprağa yol yoktur.

Örnek 3

```
Girdi : root = [], targetSum = 0
Çıktı : false
```

Ağaç boş olduğu için kökten yaprağa yol yoktur.

Kısıtlar

• Ağaçtaki düğüm sayısı [0, 5000] aralığındadır.
• -1000 <= Node.val <= 1000
• -1000 <= targetSum <= 1000

77. Sum Root to Leaf Numbers

Kökten Yaprağa Sayıların Toplamı | Seviye: Orta | LeetCode #129
Orijinal soru: leetcode.com/problems/sum-root-to-leaf-numbers/

Soru

Yalnızca 0-9 arası rakamlar içeren bir ikili ağacın kökü **root** veriliyor.

Ağaçtaki her kökten yaprağa yol bir sayıyı temsil eder.

Örneğin **1 -> 2 -> 3** kökten yaprağa yolu **123** sayısını temsil eder.

Tüm kökten yaprağa sayıların **toplamını** döndür. Test durumları, cevabın 32 bitlik bir tamsayıya sığacağı şekilde oluşturulmuştur.

Yaprak düğüm, çocuğu olmayan düğümdür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [1,2,3]
Çıktı : 25
```

1->2 kökten yaprağa yolu 12 sayısını temsil eder.
1->3 kökten yaprağa yolu 13 sayısını temsil eder.
Bu yüzden toplam = 12 + 13 = 25.

Örnek 2

```
Girdi : root = [4,9,0,5,1]
Çıktı : 1026
```

4->9->5 yolu 495 sayısını temsil eder.
4->9->1 yolu 491 sayısını temsil eder.
4->0 yolu 40 sayısını temsil eder.
Bu yüzden toplam = 495 + 491 + 40 = 1026.

Kısıtlar

• Ağaçtaki düğüm sayısı [1, 1000] aralığındadır.
• 0 <= Node.val <= 9
• Ağacın derinliği 10'u aşmayacaktır.

78. Binary Tree Maximum Path Sum

İkili Ağaç Maksimum Yol Toplamı | Seviye: Zor | LeetCode #124
Orijinal soru: leetcode.com/problems/binary-tree-maximum-path-sum/

Soru

Bir ikili ağaçtaki **yol (path)**, ardışık her düğüm çiftinin arasında bir kenar bulunan düğüm dizisidir. Bir düğüm dizide en fazla bir kez yer alabilir. Yolun kökten geçmesi **gerekmediğine** dikkat et.

Bir yolun **yol toplamı**, yoldaki düğümlerin değerlerinin toplamıdır.

Bir ikili ağacın kökü **root** veriliyor. Boş olmayan herhangi bir yolun **maksimum yol toplamını** döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [1,2,3]
Çıktı : 6
```

En uygun yol 2 -> 1 -> 3'tür; yol toplamı 2 + 1 + 3 = 6.

Örnek 2

```
Girdi : root = [-10,9,20,null,null,15,7]
Çıktı : 42
```

En uygun yol 15 -> 20 -> 7'dir; yol toplamı 15 + 20 + 7 = 42.

Kısıtlar

• Ağaçtaki düğüm sayısı $[1, 3 * 10^4]$ aralığındadır.
• -1000 <= Node.val <= 1000

79. Binary Search Tree Iterator

İkili Arama Ağacı Yineleyicisi | Seviye: Orta | LeetCode #173
Orijinal soru: leetcode.com/problems/binary-search-tree-iterator/

Soru

Bir ikili arama ağacının (BST) **in-order** dolaşımı üzerinde bir yineleyiciyi (iterator) temsil eden **BSTIterator** sınıfını gerçekle:

• **BSTIterator(TreeNode root)**: **BSTIterator** sınıfından bir nesneyi başlatır. BST'nin kökü yapıcıya (constructor) verilir. Gösterici, BST'deki herhangi bir elemandan küçük, var olmayan bir sayıya başlatılmalıdır.
• **boolean hasNext()**: Dolaşımda göstericinin sağında bir sayı varsa **true**, yoksa **false** döndürür.
• **int next()**: Göstericiyi sağa taşır, sonra göstericideki sayıyı döndürür.

Göstericiyi var olmayan en küçük bir sayıya başlatmak sayesinde, **next()**'in ilk çağrısının BST'deki en küçük elemanı döndüreceğine dikkat et.

next() çağrılarının her zaman geçerli olacağını varsayabilirsin. Yani **next()** çağrıldığında in-order dolaşımda en az bir sonraki sayı bulunacaktır.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi :
["BSTIterator","next","next","hasNext","next",
         "hasNext","next","hasNext","next","hasNext"]
        [[[7,3,15,null,null,9,20]],[],[],[],[],[],[],[],[],[]]
Çıktı :
        [null,3,7,true,9,true,15,true,20,false]
```

BSTIterator nesnesi [7, 3, 15, null, null, 9, 20] ağacıyla oluşturulur. Çağrılar sırasıyla şunları döndürür: **next()** → 3, **next()** → 7, **hasNext()** → true, **next()** → 9, **hasNext()** → true, **next()** → 15, **hasNext()** → true, **next()** → 20, **hasNext()** → false.

Kısıtlar

• Ağaçtaki düğüm sayısı [1, 10^5] aralığındadır.
• 0 <= Node.val <= 10^6
• **hasNext** ve **next** için en fazla 10^5 çağrı yapılacaktır.

Ek soru (Follow up)

next() ve **hasNext()**'i ortalama **O(1)** zamanda çalışacak ve **O(h)** bellek kullanacak şekilde gerçekleştirebilir misin? (Burada h, ağacın yüksekliğidir.)

80. Count Complete Tree Nodes

Tam Ağaçtaki Düğümleri Say | Seviye: Kolay | LeetCode #222
Orijinal soru: leetcode.com/problems/count-complete-tree-nodes/

Soru

Tam (complete) bir ikili ağacın kökü **root** veriliyor. Ağaçtaki düğüm sayısını döndür.

Vikipedi'ye göre, tam bir ikili ağaçta son seviye hariç her seviye tamamen doludur ve son seviyedeki tüm düğümler mümkün olduğunca soldadır. Son seviye **h**'de 1 ile 2^h (ikisi dahil) arasında düğüm bulunabilir.

Zaman karmaşıklığı **O(n)**'den **daha az** olan bir algoritma tasarla.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [1,2,3,4,5,6]
Çıktı : 6
```

Örnek 2

```
Girdi : root = []
Çıktı : 0
```

Örnek 3

```
Girdi : root = [1]
Çıktı : 1
```

Kısıtlar

- Ağaçtaki düğüm sayısı [0, $5 * 10^4$] aralığındadır.
- 0 <= Node.val <= $5 * 10^4$
- Ağacın tam olduğu garanti edilmiştir.

81. Lowest Common Ancestor of a Binary Tree

İkili Ağaçta En Düşük Ortak Ata | Seviye: Orta | LeetCode #236
Orijinal soru: leetcode.com/problems/lowest-common-ancestor-of-a-binary-tree/

Soru

Bir ikili ağaç ve ağaçtaki iki düğüm veriliyor. İki düğümün **en düşük ortak atasını** (LCA, lowest common ancestor) bul.

Vikipedi'deki LCA tanımına göre: “En düşük ortak ata, T ağacında hem **p** hem **q**'yu torun (descendant) olarak içeren en düşük düğüm olarak tanımlanır (bir düğümün kendisinin de torunu olmasına izin verilir).”

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Çıktı : 3
```

5 ve 1 düğümlerinin LCA'sı 3'tür.

Örnek 2

```
Girdi : root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
Çıktı : 5
```

5 ve 4 düğümlerinin LCA'sı 5'tir; çünkü LCA tanımına göre bir düğüm kendisinin torunu olabilir.

Örnek 3

```
Girdi : root = [1,2], p = 1, q = 2
Çıktı : 1
```

Kısıtlar

- Ağaçtaki düğüm sayısı [2, 10^5] aralığındadır.
- -10^9 <= Node.val <= 10^9
- Tüm Node.val değerleri benzersizdir.
- p != q
- p ve q ağaçta bulunur.

82. Binary Tree Right Side View

İkili Ağacın Sağdan Görünümü | Seviye: Orta | LeetCode #199
Orijinal soru: leetcode.com/problems/binary-tree-right-side-view/

Soru

Bir ikili ağacın kökü **root** veriliyor. Kendini ağacın sağ tarafında duruyormuş gibi hayal et; yukarıdan aşağıya sırayla, görebildiğin düğümlerin değerlerini döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [1,2,3,null,5,null,4]
Çıktı : [1,3,4]
```

Örnek 2

```
Girdi : root = [1,2,3,4,null,null,null,5]
Çıktı : [1,3,4,5]
```

Örnek 3

```
Girdi : root = [1,null,3]
Çıktı : [1,3]
```

Örnek 4

```
Girdi : root = []
Çıktı : []
```

Kısıtlar

• Ağaçtaki düğüm sayısı [0, 100] aralığındadır.
• -100 <= Node.val <= 100

83. Average of Levels in Binary Tree

İkili Ağaçta Seviye Ortalamaları | Seviye: Kolay | LeetCode #637
Orijinal soru: leetcode.com/problems/average-of-levels-in-binary-tree/

Soru

Bir ikili ağacın kökü **root** veriliyor. Her seviyedeki düğümlerin ortalama değerini bir dizi olarak döndür. Gerçek cevabın 10^{-5} yakınındaki cevaplar kabul edilecektir.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [3,9,20,null,null,15,7]
Çıktı : [3.00000,14.50000,11.00000]
```

0. seviyedeki düğümlerin ortalaması 3, 1. seviyede 14.5, 2. seviyede 11'dir. Bu yüzden [3, 14.5, 11] döndürülür.

Örnek 2

```
Girdi : root = [3,9,20,15,7]
Çıktı : [3.00000,14.50000,11.00000]
```

Kısıtlar

• Ağaçtaki düğüm sayısı [1, 10^4] aralığındadır.
• -2^{31} <= Node.val <= $2^{31} - 1$

84. Binary Tree Level Order Traversal

İkili Ağaç Seviye Sırası Dolaşımı | Seviye: Orta | LeetCode #102
Orijinal soru: leetcode.com/problems/binary-tree-level-order-traversal/

Soru

Bir ikili ağacın kökü **root** veriliyor. Düğümlerinin değerlerinin **seviye sırası (level order)** dolaşımını döndür (yani soldan sağa, seviye seviye).

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [3,9,20,null,null,15,7]
Çıktı : [[3],[9,20],[15,7]]
```

Örnek 2

```
Girdi : root = [1]
Çıktı : [[1]]
```

Örnek 3

```
Girdi : root = []
Çıktı : []
```

Kısıtlar

• Ağaçtaki düğüm sayısı [0, 2000] aralığındadır.
• -1000 <= Node.val <= 1000

85. Binary Tree Zigzag Level Order Traversal

İkili Ağaç Zikzak Seviye Sırası Dolaşımı | Seviye: Orta | LeetCode #103
Orijinal soru: leetcode.com/problems/binary-tree-zigzag-level-order-traversal/

Soru

Bir ikili ağacın kökü **root** veriliyor. Düğümlerinin değerlerinin **zikzak seviye sırası (zigzag level order)** dolaşımını döndür (yani soldan sağa, sonraki seviyede sağdan sola ve böyle dönüşümlü olarak).

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [3,9,20,null,null,15,7]
Çıktı : [[3],[20,9],[15,7]]
```

Örnek 2

```
Girdi : root = [1]
Çıktı : [[1]]
```

Örnek 3

```
Girdi : root = []
Çıktı : []
```

Kısıtlar

• Ağaçtaki düğüm sayısı [0, 2000] aralığındadır.
• -100 <= Node.val <= 100

86. Minimum Absolute Difference in BST

İkili Arama Ağacında Minimum Mutlak Fark | Seviye: Kolay | LeetCode #530
Orijinal soru: leetcode.com/problems/minimum-absolute-difference-in-bst/

Soru

Bir İkili Arama Ağacının (BST) kökü **root** veriliyor. Ağaçtaki herhangi iki **farklı** düğümün değerleri arasındaki **minimum mutlak farkı** döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [4,2,6,1,3]
Çıktı : 1
```

Örnek 2

```
Girdi : root = [1,0,48,null,null,12,49]
Çıktı : 1
```

Kısıtlar

• Ağaçtaki düğüm sayısı [2, 10^4] aralığındadır.
• 0 <= Node.val <= 10^5

87. Kth Smallest Element in a BST

İkili Arama Ağacında K. En Küçük Eleman | Seviye: Orta | LeetCode #230
Orijinal soru: leetcode.com/problems/kth-smallest-element-in-a-bst/

Soru

Bir ikili arama ağacının kökü **root** ve bir tamsayı **k** veriliyor. Ağaçtaki tüm düğüm değerleri arasında **k**. en küçük değeri (1'den başlayan sayımla) döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [3,1,4,null,2], k = 1
Çıktı : 1
```

Örnek 2

```
Girdi : root = [5,3,6,2,4,null,null,1], k = 3
Çıktı : 3
```

Kısıtlar

• Ağaçtaki düğüm sayısı n'dir.
• $1 <= k <= n <= 10^4$
• $0 <= Node.val <= 10^4$

Ek soru (Follow up)

BST sık sık değiştiriliyorsa (yani ekleme ve silme işlemleri yapabiliyorsak) ve **k**. en küçüğü sık sık bulman gerekiyorsa, bunu nasıl optimize ederdin?

88. Validate Binary Search Tree

Geçerli İkili Arama Ağacı | Seviye: Orta | LeetCode #98
Orijinal soru: leetcode.com/problems/validate-binary-search-tree/

Soru

Bir ikili ağacın kökü **root** veriliyor. Ağacın geçerli bir **ikili arama ağacı** (BST) olup olmadığını belirle.

Geçerli BST şöyle tanımlanır:

• Bir düğümün sol alt ağacı yalnızca düğümün anahtarından **küçük** anahtarlara sahip düğümler içerir.
• Bir düğümün sağ alt ağacı yalnızca düğümün anahtarından **büyük** anahtarlara sahip düğümler içerir.
• Hem sol hem sağ alt ağaçlar da ikili arama ağacı olmalıdır.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : root = [2,1,3]
Çıktı : true
```

Örnek 2

```
Girdi : root = [5,1,4,null,null,3,6]
Çıktı : false
```

Kök düğümün değeri 5 ama sağ çocuğunun değeri 4.

Kısıtlar

• Ağaçtaki düğüm sayısı $[1, 10^4]$ aralığındadır.
• -2^{31} <= Node.val <= $2^{31} - 1$

89. Number of Islands

Ada Sayısı | Seviye: Orta | LeetCode #200
Orijinal soru: leetcode.com/problems/number-of-islands/

Soru

'1'ler (kara) ve '0'lar (su) içeren bir haritayı temsil eden **m x n** boyutunda 2 boyutlu ikili bir **grid** ızgarası veriliyor. **Ada sayısını** döndür.

Ada, suyla çevrilidir ve yatay veya dikey olarak bitişik karaların birleşmesiyle oluşur. Izgaranın dört kenarının da suyla çevrili olduğunu varsayabilirsin.

Örnekler

Örnek 1

```
Girdi : grid = [
          ["1","1","1","1","0"],
          ["1","1","0","1","0"],
          ["1","1","0","0","0"],
          ["0","0","0","0","0"]
        ]
Çıktı : 1
```

Örnek 2

```
Girdi : grid = [
          ["1","1","0","0","0"],
          ["1","1","0","0","0"],
          ["0","0","1","0","0"],
          ["0","0","0","1","1"]
        ]
Çıktı : 3
```

Kısıtlar

• m == grid.length
• n == grid[i].length
• 1 <= m, n <= 300
• grid[i][j] '0' ya da '1'dir.

90. Surrounded Regions

Çevrelenmiş Bölgeler | Seviye: Orta | LeetCode #130
Orijinal soru: leetcode.com/problems/surrounded-regions/

Soru

'X' ve **'O'** harflerini içeren **m x n** boyutunda bir **board** matrisi veriliyor. **Çevrelenmiş (surrounded)** bölgeleri ele geçir (capture):

• **Bağlantı (Connect):** Bir hücre yatay veya dikey olarak bitişik hücrelere bağlıdır.
• **Bölge (Region):** Bir bölge oluşturmak için her **'O'** hücresini bağla.
• **Çevreleme (Surround):** Bölgeyi **'X'** hücreleriyle bağlayabiliyorsan ve bölgedeki hiçbir hücre tahtanın kenarında değilse, bölge **'X'** hücreleriyle çevrilmiştir.

Çevrelenmiş bir bölgeyi ele geçirmek için, orijinal tahta üzerinde yerinde tüm **'O'**'ları **'X'** ile değiştir. Hiçbir şey döndürmen gerekmez.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : board = [
          ["X","X","X","X"],
          ["X","O","O","X"],
          ["X","X","O","X"],
          ["X","O","X","X"]
        ]
Çıktı : [
          ["X","X","X","X"],
          ["X","X","X","X"],
          ["X","X","X","X"],
          ["X","O","X","X"]
        ]
```

Yukarıdaki şekilde alttaki bölge, tahtanın kenarında olduğu ve çevrelenemeyeceği için ele geçirilmez.

Örnek 2

```
Girdi : board = [["X"]]
Çıktı : [["X"]]
```

Kısıtlar

• m == board.length
• n == board[i].length
• 1 <= m, n <= 200
• board[i][j] 'X' ya da 'O'dur.

91. Clone Graph

Grafı Klonla | Seviye: Orta | LeetCode #133
Orijinal soru: leetcode.com/problems/clone-graph/

Soru

Bağlantılı, yönsüz (undirected) bir graftaki bir düğümün referansı veriliyor. Grafın bir **derin kopyasını (deep copy, klon)** döndür.

Graftaki her düğüm bir değer (**int**) ve komşularının bir listesini (**List[Node]**) içerir.

```
class Node { public int val; public List<Node> neighbors; }
```

Test durumu biçimi: Basitlik için her düğümün değeri, düğümün indeksiyle (1'den başlayan) aynıdır. Örneğin ilk düğümün **val == 1**, ikinci düğümün **val == 2** olur ve böyle devam eder. Graf, test durumunda bir **komşuluk listesiyle (adjacency list)** gösterilir.

Komşuluk listesi, sonlu bir grafı göstermek için kullanılan sırasız listelerin koleksiyonudur. Her liste, graftaki bir düğümün komşuları kümesini tanımlar.

Verilen düğüm her zaman ilk düğüm, yani **val = 1** olan düğümdür. Verilen düğümün kopyasını, klonlanmış grafa bir referans olarak döndürmelisin.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : adjList = [[2,4],[1,3],[2,4],[1,3]]
Çıktı : [[2,4],[1,3],[2,4],[1,3]]
```

Graftta 4 düğüm var. 1. düğümün (val = 1) komşuları 2. düğüm (val = 2) ve 4. düğümdür (val = 4). 2. düğümün (val = 2) komşuları 1. düğüm (val = 1) ve 3. düğümdür (val = 3). 3. düğümün (val = 3) komşuları 2. düğüm (val = 2) ve 4. düğümdür (val = 4). 4. düğümün (val = 4) komşuları 1. düğüm (val = 1) ve 3. düğümdür (val = 3).

Örnek 2

```
Girdi : adjList = [[]]
Çıktı : [[]]
```

Girdinin bir boş liste içerdiğine dikkat et. Graf yalnızca val = 1 olan tek bir düğümden oluşur ve komşusu yoktur.

Örnek 3

```
Girdi : adjList = []
Çıktı : []
```

Bu boş bir graftır; hiç düğümü yoktur.

Kısıtlar

- Graftaki düğüm sayısı [0, 100] aralığındadır.
- 1 <= Node.val <= 100
- Node.val her düğüm için benzersizdir.
- Grafta tekrar eden kenar ve kendine döngü (self-loop) yoktur.
- Graf bağlantılıdır ve verilen düğümden başlayarak tüm düğümlere ulaşılabilir.

92. Evaluate Division

Bölme İşlemini Değerlendir | Seviye: Orta | LeetCode #399
Orijinal soru: leetcode.com/problems/evaluate-division/

Soru

Bir **equations** değişken çifti dizisi ve bir **values** gerçel sayı dizisi veriliyor; **equations[i] = [A_i, B_i]** ve **values[i]**, **A_i / B_i = values[i]** denklemini temsil eder. Her **A_i** veya **B_i**, tek bir değişkeni temsil eden bir dizgidir.

Ayrıca bazı sorgular (**queries**) veriliyor; **queries[j] = [C_j, D_j]**, **C_j / D_j = ?** sonucunu bulman gereken **j**. sorguyu temsil eder.

Tüm sorguların cevaplarını döndür. Tek bir cevap belirlenemiyorsa **-1.0** döndür.

Not: Girdi her zaman geçerlidir. Sorguların değerlendirilmesinin sıfıra bölmeyle sonuçlanmayacağını ve çelişki olmadığını varsayabilirsin.

Not: Denklemler listesinde geçmeyen değişkenler tanımsızdır; bu yüzden onlar için cevap belirlenemez.

Örnekler

Örnek 1

```
Girdi : equations = [["a","b"],["b","c"]],
        values = [2.0,3.0],
        queries = [["a","c"],["b","a"],["a","e"],["a","a"],["x","x"]]
Çıktı : [6.00000,0.50000,-1.00000,1.00000,-1.00000]
```

Verilenler: a / b = 2.0, b / c = 3.0 Sorgular: a / c = ?, b / a = ?, a / e = ?, a / a = ?, x / x = ? Sonuç: [6.0, 0.5, -1.0, 1.0, -1.0] Not: x tanımsızdır => -1.0

Örnek 2

```
Girdi : equations = [["a","b"],["b","c"],["bc","cd"]],
        values = [1.5,2.5,5.0],
        queries = [["a","c"],["c","b"],["bc","cd"],["cd","bc"]]
Çıktı : [3.75000,0.40000,5.00000,0.20000]
```

Örnek 3

```
Girdi : equations = [["a","b"]],
        values = [0.5],
        queries = [["a","b"],["b","a"],["a","c"],["x","y"]]
Çıktı : [0.50000,2.00000,-1.00000,-1.00000]
```

Kısıtlar

- 1 <= equations.length <= 20 ve equations[i].length == 2
- 1 <= A_i.length, B_i.length <= 5
- values.length == equations.length ve 0.0 < values[i] <= 20.0
- 1 <= queries.length <= 20 ve queries[i].length == 2
- 1 <= C_j.length, D_j.length <= 5
- A_i, B_i, C_j, D_j küçük İngilizce harflerden ve rakamlardan oluşur.

93. Course Schedule

Ders Programı | Seviye: Orta | LeetCode #207
Orijinal soru: leetcode.com/problems/course-schedule/

Soru

Almak zorunda olduğun toplam **numCourses** ders var; dersler **0**'dan **numCourses - 1**'e kadar etiketlenmiş. Sana bir **prerequisites** dizisi veriliyor; **prerequisites[i] = [a_i, b_i]**, **a_i** dersini almak istiyorsan önce **b_i** dersini almak zorunda olduğunu gösterir.

Örneğin **[0, 1]** çifti, 0. dersi almak için önce 1. dersi almak gerektiğini gösterir.

Tüm dersleri bitirebiliyorsan **true**, aksi halde **false** döndür.

Örnekler

Örnek 1

```
Girdi : numCourses = 2, prerequisites = [[1,0]]
Çıktı : true
```

Alınacak toplam 2 ders var. 1. dersi almak için 0. dersi bitirmiş olmalısın. Bu yüzden mümkün.

Örnek 2

```
Girdi : numCourses = 2, prerequisites = [[1,0],[0,1]]
Çıktı : false
```

Alınacak toplam 2 ders var. 1. dersi almak için 0. dersi, 0. dersi almak için de 1. dersi bitirmiş olmalısın. Bu yüzden imkânsız.

Kısıtlar

- 1 <= numCourses <= 2000
- 0 <= prerequisites.length <= 5000
- prerequisites[i].length == 2
- 0 <= a_i, b_i < numCourses
- Tüm prerequisites[i] çiftleri benzersizdir.

94. Course Schedule II

Ders Programı II | Seviye: Orta | LeetCode #210
Orijinal soru: leetcode.com/problems/course-schedule-ii/

Soru

Almak zorunda olduğun toplam **numCourses** ders var; dersler **0**'dan **numCourses - 1**'e kadar etiketlenmiş. Sana bir **prerequisites** dizisi veriliyor; **prerequisites[i] = [a_i, b_i]**, **a_i** dersini almak istiyorsan önce **b_i** dersini almak zorunda olduğunu gösterir.

Örneğin **[0, 1]** çifti, 0. dersi almak için önce 1. dersi almak gerektiğini gösterir.

Tüm dersleri bitirmek için almanız gereken derslerin **sırasını** döndür. Birden fazla geçerli cevap varsa herhangi birini döndürebilirsin. Tüm dersleri bitirmek imkânsızsa **boş bir dizi** döndür.

Örnekler

Örnek 1

```
Girdi : numCourses = 2, prerequisites = [[1,0]]
Çıktı : [0,1]
```

Alınacak toplam 2 ders var. 1. dersi almak için 0. dersi bitirmiş olmalısın. Bu yüzden doğru ders sırası [0,1].

Örnek 2

```
Girdi : numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
Çıktı : [0,2,1,3]
```

Alınacak toplam 4 ders var. 3. dersi almak için hem 1. hem 2. dersi bitirmiş olmalısın. 1. ve 2. dersler 0. ders bitirildikten sonra alınmalı. Doğru ders sıralarından biri [0,1,2,3], bir diğeri [0,2,1,3].

Örnek 3

```
Girdi : numCourses = 1, prerequisites = []
Çıktı : [0]
```

Kısıtlar

- 1 <= numCourses <= 2000
- 0 <= prerequisites.length <= numCourses * (numCourses - 1)
- prerequisites[i].length == 2
- 0 <= a_i, b_i < numCourses
- a_i != b_i
- Tüm [a_i, b_i] çiftleri birbirinden farklıdır.

95. Snakes and Ladders

Yılanlar ve Merdivenler | Seviye: Orta | LeetCode #909
Orijinal soru: leetcode.com/problems/snakes-and-ladders/

Soru

n x n boyutunda bir tamsayı matrisi **board** veriliyor. Hücreler, tahtanın sol altından (yani **board[n - 1][0]**) başlayarak her satırda yönü değiştiren **Boustrophedon** (öküz sürme) tarzında **1**'den **n^2**'ye kadar etiketlenmiştir. Tahtanın **1** numaralı karesinde başlıyorsun. Her hamlede, mevcut kare **curr** iken şunları yap:

• **[curr + 1, min(curr + 6, n^2)]** aralığında etiketi olan bir hedef kare **next** seç. Bu seçim, standart 6 yüzlü bir zar atışının sonucunu simüle eder: yani tahtanın boyutundan bağımsız olarak en fazla 6 hedef vardır.
• **next** karesinde yılan ya da merdiven varsa, o yılanın ya da merdivenin hedefine gitmek zorundasın. Aksi halde **next** karesine gidersin.
• Oyun, **n^2** karesine ulaştığında biter.

r satırı ve **c** sütunundaki bir tahta karesinde, **board[r][c] != -1** ise yılan ya da merdiven vardır. O yılanın ya da merdivenin hedefi **board[r][c]**'dir. 1 ve **n^2** numaralı kareler hiçbir yılan ya da merdivenin başlangıç noktası değildir.

Her zar atışında en fazla bir kez yılan ya da merdiven kullandığına dikkat et. Bir yılanın ya da merdivenin hedefi başka bir yılanın ya da merdivenin başlangıcıysa, sonraki yılanı ya da merdiveni takip etmezsin. Örneğin tahta **[[-1,4],[-1,3]]** ise ve ilk hamlede hedef karen 2 ise, merdiveni takip ederek 3. kareye gidersin ama 4. kareye giden sonraki merdiveni takip etmezsin.

n^2 karesine ulaşmak için gereken **en az zar atışı sayısını** döndür. Kareye ulaşmak mümkün değilse **-1** döndür.

Orijinalde tahta görseli de var (bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : board = [[-1,-1,-1,-1,-1,-1],[-1,-1,-1,-1,-1,-1],
          [-1,-1,-1,-1,-1,-1],[-1,35,-1,-1,13,-1],
          [-1,-1,-1,-1,-1,-1],[-1,15,-1,-1,-1,-1]]
Çıktı : 4
```

Başlangıçta 1. karedesin. 2. kareye gitmeye karar verirsin ve merdivenle 15. kareye gitmek zorundasın. Sonra 17. kareye gitmeye karar verirsin ve yılanla 13. kareye inmek zorundasın. Sonra 14. kareye gitmeye karar verirsin ve merdivenle 35. kareye çıkmak zorundasın. Sonra 36. kareye giderek oyunu bitirirsin. Bu, son kareye ulaşmak için mümkün olan en az hamle sayısıdır; bu yüzden 4 döndür.

Örnek 2

```
Girdi : board = [[-1,-1],[-1,3]]
Çıktı : 1
```

Kısıtlar

• n == board.length == board[i].length
• 2 <= n <= 20
• board[i][j] ya -1'dir ya da [1, n^2] aralığındadır.
• 1 ve n^2 etiketli karelerin hiçbiri yılan ya da merdivenin başlangıç noktası değildir.

96. Minimum Genetic Mutation

Minimum Genetik Mutasyon | Seviye: Orta | LeetCode #433
Orijinal soru: leetcode.com/problems/minimum-genetic-mutation/

Soru

Bir gen dizgisi, **'A'**, **'C'**, **'G'** ve **'T'** karakterlerinden seçilen 8 karakter uzunluğunda bir dizgiyle temsil edilebilir.

startGene gen dizgisinden **endGene** gen dizgisine bir mutasyonu araştırmamız gerektiğini düşün; burada bir mutasyon, gen dizgisinde tek bir karakterin değişmesi olarak tanımlanır. Örneğin “AACCGGTT” --> “AACCGGTA” bir mutasyondur.

Ayrıca geçerli tüm gen mutasyonlarını kaydeden bir gen bankası **bank** da var. Bir genin geçerli bir gen dizgisi olması için **bank** içinde bulunması gerekir.

startGene ve **endGene** gen dizgileri ile **bank** gen bankası verildiğinde, **startGene**'den **endGene**'e mutasyon için gereken **minimum mutasyon sayısını** döndür. Böyle bir mutasyon yoksa **-1** döndür.

Başlangıç noktasının geçerli kabul edildiğine, bu yüzden **bank** içinde yer almayabileceğine dikkat et.

Örnekler

Örnek 1

```
Girdi : startGene = "AACCGGTT", endGene = "AACCGGTA", bank = ["AACCGGTA"]
Çıktı : 1
```

Örnek 2

```
Girdi : startGene = "AACCGGTT", endGene = "AAACGGTA",
        bank = ["AACCGGTA","AACCGCTA","AAACGGTA"]
Çıktı : 2
```

Kısıtlar

• 0 <= bank.length <= 10
• startGene.length == endGene.length == bank[i].length == 8
• startGene, endGene ve bank[i] yalnızca ['A', 'C', 'G', 'T'] karakterlerinden oluşur.

97. Word Ladder

Kelime Merdiveni | Seviye: Zor | LeetCode #127
Orijinal soru: leetcode.com/problems/word-ladder/

Soru

wordList sözlüğünü kullanarak **beginWord** kelimesinden **endWord** kelimesine bir **dönüşüm dizisi** (transformation sequence), şu koşulları sağlayan **beginWord -> s_1 -> s_2 -> ... -> s_k** kelime dizisidir:

• Ardışık her kelime çifti tek bir harfle farklıdır.
• 1 <= i <= k için her **s_i**, **wordList** içindedir. **beginWord**'ün **wordList** içinde olması gerekmediğine dikkat et.
• **s_k == endWord**

İki kelime **beginWord** ve **endWord** ile bir **wordList** sözlüğü verildiğinde, **beginWord**'den **endWord**'e en kısa dönüşüm dizisindeki **kelime sayısını** döndür. Böyle bir dizi yoksa **0** döndür.

Örnekler

Örnek 1

```
Girdi : beginWord = "hit", endWord = "cog",
        wordList = ["hot","dot","dog","lot","log","cog"]
Çıktı : 5
```

En kısa dönüşüm dizilerinden biri “hit” -> “hot” -> “dot” -> “dog” -> “cog”'dur ve 5 kelime uzunluğundadır.

Örnek 2

```
Girdi : beginWord = "hit", endWord = "cog",
        wordList = ["hot","dot","dog","lot","log"]
Çıktı : 0
```

endWord olan “cog”, **wordList** içinde olmadığı için geçerli bir dönüşüm dizisi yoktur.

Kısıtlar

• 1 <= beginWord.length <= 10
• endWord.length == beginWord.length
• 1 <= wordList.length <= 5000
• wordList[i].length == beginWord.length
• beginWord, endWord ve wordList[i] küçük İngilizce harflerden oluşur.
• beginWord != endWord
• wordList içindeki tüm kelimeler benzersizdir.

98. Implement Trie (Prefix Tree)

Trie Gerçekle (Önek Ağacı) | Seviye: Orta | LeetCode #208
Orijinal soru: leetcode.com/problems/implement-trie-prefix-tree/

Soru

Trie (“try” diye okunur) ya da **önek ağacı (prefix tree)**, bir dizgi veri kümesindeki anahtarları verimli biçimde saklamak ve geri almak için kullanılan bir ağaç veri yapısıdır. Bu veri yapısının otomatik tamamlama ve yazım denetleyicisi gibi çeşitli uygulamaları vardır.

Trie sınıfını gerçekle:

• **Trie()**: trie nesnesini başlatır.
• **void insert(String word)**: **word** dizgisini trie'ye ekler.
• **boolean search(String word)**: **word** dizgisi trie'de varsa (yani daha önce eklenmişse) **true**, yoksa **false** döndürür.
• **boolean startsWith(String prefix)**: **prefix** önekine sahip, daha önce eklenmiş bir dizgi varsa **true**, yoksa **false** döndürür.

Örnekler

Örnek 1

```
Girdi :
["Trie","insert","search","search","startsWith","insert","search"]
        [[],["apple"],["apple"],["app"],["app"],["app"],["app"]]
Çıktı :
        [null,null,true,false,true,null,true]
```

• **Trie()** ile trie oluşturulur.
• **insert(“apple”)**
• **search(“apple”)**: true döner.
• **search(“app”)**: false döner.
• **startsWith(“app”)**: true döner.
• **insert(“app”)**
• **search(“app”)**: true döner.

Kısıtlar

• 1 <= word.length, prefix.length <= 2000
• word ve prefix yalnızca küçük İngilizce harflerden oluşur.
• **insert**, **search** ve **startsWith** için toplamda en fazla $3 * 10^4$ çağrı yapılacaktır.

99. Design Add and Search Words Data Structure

Kelime Ekleme ve Arama Veri Yapısı Tasarla | Seviye: Orta | LeetCode #211
Orijinal soru: leetcode.com/problems/design-add-and-search-words-data-structure/

Soru

Yeni kelimeler eklemeyi ve bir dizginin daha önce eklenmiş herhangi bir dizgiyle eşleşip eşleşmediğini bulmayı destekleyen bir veri yapısı tasarla.

WordDictionary sınıfını gerçekle:

• **WordDictionary()**: nesneyi başlatır.
• **void addWord(word)**: **word**'ü veri yapısına ekler; daha sonra eşleştirilebilir.
• **bool search(word)**: Veri yapısında **word** ile eşleşen herhangi bir dizgi varsa **true**, yoksa **false** döndürür. **word**, nokta **'.'** karakterleri içerebilir; noktalar herhangi bir harfle eşleşebilir.

Örnekler

Örnek 1

```
Girdi :
["WordDictionary","addWord","addWord","addWord","search",
         "search","search","search"]
        [[],["bad"],["dad"],["mad"],["pad"],["bad"],[".ad"],["b.."]]
Çıktı :
        [null,null,null,null,false,true,true,true]
```

• **addWord(“bad”)**, **addWord(“dad”)**, **addWord(“mad”)**
• **search(“pad”)**: false döner.
• **search(“bad”)**: true döner.
• **search(“.ad”)**: true döner.
• **search(“b..”)**: true döner.

Kısıtlar

• 1 <= word.length <= 25
• addWord içindeki word küçük İngilizce harflerden oluşur.
• search içindeki word '.' ya da küçük İngilizce harflerden oluşur.
• search sorgularındaki word içinde en fazla 2 nokta olacaktır.
• addWord ve search için en fazla 10^4 çağrı yapılacaktır.

100. Word Search II

Kelime Arama II | Seviye: Zor | LeetCode #212
Orijinal soru: leetcode.com/problems/word-search-ii/

Soru

m x n boyutunda bir karakter tahtası **board** ve bir dizgi listesi **words** veriliyor. Tahtada bulunan **tüm kelimeleri** döndür.

Her kelime, ardışık bitişik hücrelerdeki harflerden oluşturulmalıdır; burada bitişik hücreler yatay veya dikey komşu hücrelerdir. Aynı harf hücresi bir kelimede birden fazla kez kullanılamaz.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : board = [
          ["o","a","a","n"],
          ["e","t","a","e"],
          ["i","h","k","r"],
          ["i","f","l","v"]
        ],
        words = ["oath","pea","eat","rain"]
Çıktı : ["eat","oath"]
```

Örnek 2

```
Girdi : board = [["a","b"],["c","d"]], words = ["abcb"]
Çıktı : []
```

Kısıtlar

- m == board.length
- n == board[i].length
- 1 <= m, n <= 12
- board[i][j] küçük bir İngilizce harftir.
- 1 <= words.length <= $3 * 10^4$
- 1 <= words[i].length <= 10
- words[i] küçük İngilizce harflerden oluşur.
- words içindeki tüm dizgiler benzersizdir.

101. Letter Combinations of a Phone Number

Telefon Numarasının Harf Kombinasyonları | Seviye: Orta | LeetCode #17
Orijinal soru: leetcode.com/problems/letter-combinations-of-a-phone-number/

Soru

2-9 arası (ikisi dahil) rakamlar içeren bir dizgi veriliyor. Bu sayının temsil edebileceği tüm olası harf kombinasyonlarını döndür. Cevabı herhangi bir sırada döndürebilirsin.

Rakamların harflerle eşlemesi (tıpkı telefon tuşlarındaki gibi) aşağıda verilmiştir. **1**'in hiçbir harfe karşılık gelmediğine dikkat et.

```
2: abc    3: def
4: ghi    5: jkl    6: mno
7: pqrs   8: tuv    9: wxyz
```

Örnekler

Örnek 1

```
Girdi : digits = "23"
Çıktı : ["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

Örnek 2

```
Girdi : digits = ""
Çıktı : []
```

Örnek 3

```
Girdi : digits = "2"
Çıktı : ["a","b","c"]
```

Kısıtlar

• 0 <= digits.length <= 4
• digits[i], ['2', '9'] aralığında bir rakamdır.

102. Combinations

Kombinasyonlar | Seviye: Orta | LeetCode #77
Orijinal soru: leetcode.com/problems/combinations/

Soru

İki tamsayı **n** ve **k** veriliyor. **[1, n]** aralığından seçilen **k** sayıdan oluşan tüm olası kombinasyonları döndür.

Cevabı herhangi bir sırada döndürebilirsin.

Örnekler

Örnek 1

```
Girdi : n = 4, k = 2
Çıktı : [[1,2],[1,3],[1,4],[2,3],[2,4],[3,4]]
```

4 üzerinden 2 = 6 toplam kombinasyon vardır. Kombinasyonların sırasız olduğuna dikkat et; yani [1,2] ve [2,1] aynı kombinasyon kabul edilir.

Örnek 2

```
Girdi : n = 1, k = 1
Çıktı : [[1]]
```

1 üzerinden 1 = 1 toplam kombinasyon vardır.

Kısıtlar

• 1 <= n <= 20
• 1 <= k <= n

103. Permutations

Permütasyonlar | Seviye: Orta | LeetCode #46
Orijinal soru: leetcode.com/problems/permutations/

Soru

Birbirinden farklı tamsayılardan oluşan bir **nums** dizisi veriliyor. Tüm olası **permütasyonları** döndür. Cevabı herhangi bir sırada döndürebilirsin.

Örnekler

Örnek 1

```
Girdi : nums = [1,2,3]
Çıktı : [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Örnek 2

```
Girdi : nums = [0,1]
Çıktı : [[0,1],[1,0]]
```

Örnek 3

```
Girdi : nums = [1]
Çıktı : [[1]]
```

Kısıtlar

• 1 <= nums.length <= 6
• -10 <= nums[i] <= 10
• nums'ın tüm tamsayıları benzersizdir.

104. Combination Sum

Kombinasyon Toplamı | Seviye: Orta | LeetCode #39
Orijinal soru: leetcode.com/problems/combination-sum/

Soru

Birbirinden farklı tamsayılardan oluşan bir **candidates** dizisi ve bir hedef tamsayı **target** veriliyor. Seçilen sayıların toplamı **target**'a eşit olan, **candidates**'ten oluşturulmuş tüm **benzersiz kombinasyonların** listesini döndür. Kombinasyonları herhangi bir sırada döndürebilirsin.

candidates içindeki aynı sayı **sınırsız sayıda** kez seçilebilir. Seçilen sayılardan en az birinin sıklığı (frekansı) farklıysa iki kombinasyon benzersizdir.

Test durumları, verilen girdi için toplamı **target**'a eşit olan benzersiz kombinasyon sayısının 150'den az olacağı şekilde oluşturulmuştur.

Örnekler

Örnek 1

```
Girdi : candidates = [2,3,6,7], target = 7
Çıktı : [[2,2,3],[7]]
```

2 ve 3 adaydır ve 2 + 2 + 3 = 7. 2'nin birden fazla kez kullanılabildiğine dikkat et.
7 de bir adaydır ve 7 = 7.
Bunlar tek iki kombinasyondur.

Örnek 2

```
Girdi : candidates = [2,3,5], target = 8
Çıktı : [[2,2,2,2],[2,3,3],[3,5]]
```

Örnek 3

```
Girdi : candidates = [2], target = 1
Çıktı : []
```

Kısıtlar

- 1 <= candidates.length <= 30
- 2 <= candidates[i] <= 40
- candidates'in tüm elemanları birbirinden farklıdır.
- 1 <= target <= 40

105. N-Queens II

N-Vezir II | Seviye: Zor | LeetCode #52
Orijinal soru: leetcode.com/problems/n-queens-ii/

Soru

n-vezir (n-queens) bulmacası, **n x n** bir satranç tahtasına **n** veziri, hiçbir ikisi birbirine saldırmayacak şekilde yerleştirme problemidir.

Bir tamsayı **n** verildiğinde, n-vezir bulmacasının **farklı çözümlerinin sayısını** döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : n = 4
Çıktı : 2
```

4-vezir bulmacasının görselde gösterildiği gibi iki farklı çözümü vardır.

Örnek 2

```
Girdi : n = 1
Çıktı : 1
```

Kısıtlar

• 1 <= n <= 9

106. Generate Parentheses

Parantez Üret | Seviye: Orta | LeetCode #22
Orijinal soru: leetcode.com/problems/generate-parentheses/

Soru

n çift parantez verildiğinde, doğru biçimlendirilmiş (well-formed) parantezlerin tüm kombinasyonlarını üreten bir fonksiyon yaz.

Örnekler

Örnek 1

```
Girdi : n = 3
Çıktı : ["((()))","(()())","(())()","()(())","()()()"]
```

Örnek 2

```
Girdi : n = 1
Çıktı : ["()"]
```

Kısıtlar

• 1 <= n <= 8

107. Word Search

Kelime Arama | Seviye: Orta | LeetCode #79
Orijinal soru: leetcode.com/problems/word-search/

Soru

m x n boyutunda bir karakter ızgarası **board** ve bir dizgi **word** veriliyor. **word** ızgarada varsa **true** döndür.

Kelime, ardışık bitişik hücrelerdeki harflerden oluşturulabilir; burada bitişik hücreler yatay veya dikey komşu hücrelerdir. Aynı harf hücresi birden fazla kez kullanılamaz.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]],
        word = "ABCCED"
Çıktı : true
```

Örnek 2

```
Girdi : board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]],
        word = "SEE"
Çıktı : true
```

Örnek 3

```
Girdi : board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]],
        word = "ABCB"
Çıktı : false
```

Kısıtlar

• m == board.length
• n = board[i].length
• 1 <= m, n <= 6
• 1 <= word.length <= 15
• board ve word yalnızca küçük ve büyük İngilizce harflerden oluşur.

Ek soru (Follow up)

Daha büyük bir tahtada çözümünü hızlandırmak için arama budamayı (search pruning) kullanabilir misin?

108. Convert Sorted Array to Binary Search Tree

Sıralı Diziyi İkili Arama Ağacına Çevir | Seviye: Kolay | LeetCode #108
Orijinal soru: leetcode.com/problems/convert-sorted-array-to-binary-search-tree/

Soru

Elemanları artan sırada sıralanmış bir tamsayı dizisi **nums** veriliyor. Bunu **yükseklik dengeli (height-balanced)** bir ikili arama ağacına dönüştür.

(Yükseklik dengeli ikili ağaç, her düğümün iki alt ağacının derinliklerinin birbirinden en fazla bir farklı olduğu ikili ağaçtır.)

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : nums = [-10,-3,0,5,9]
Çıktı : [0,-3,9,-10,null,5]
```

[0,-10,5,null,-3,null,9] de kabul edilir.

Örnek 2

```
Girdi : nums = [1,3]
Çıktı : [3,1]
```

[1,null,3] ve [3,1] ikisi de yükseklik dengeli BST'lerdir.

Kısıtlar

• $1 <= nums.length <= 10^4$
• $-10^4 <= nums[i] <= 10^4$
• nums kesin artan sırada sıralıdır.

109. Sort List

Listeyi Sırala | Seviye: Orta | LeetCode #148
Orijinal soru: leetcode.com/problems/sort-list/

Soru

Bir bağlı listenin başı **head** veriliyor. Listeyi artan sırada sıralayıp döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : head = [4,2,1,3]
Çıktı : [1,2,3,4]
```

Örnek 2

```
Girdi : head = [-1,5,3,4,0]
Çıktı : [-1,0,3,4,5]
```

Örnek 3

```
Girdi : head = []
Çıktı : []
```

Kısıtlar

• Listedeki düğüm sayısı $[0, 5 * 10^4]$ aralığındadır.
• $-10^5 <= Node.val <= 10^5$

Ek soru (Follow up)

Bağlı listeyi **O(n log n)** zamanda ve **O(1)** bellekle (yani sabit alanla) sıralayabilir misin?

110. Construct Quad Tree

Dörtlü Ağaç (Quad Tree) Oluştur | Seviye: Orta | LeetCode #427
Orijinal soru: leetcode.com/problems/construct-quad-tree/

Soru

Yalnızca 0'lardan ve 1'lerden oluşan **n * n** boyutunda bir **grid** matrisi veriliyor. **grid**'i bir **Dörtlü Ağaç (Quad-Tree)** ile temsil etmek istiyoruz. **grid**'i temsil eden Quad-Tree'nin kökünü döndür.

Quad-Tree, her iç düğümün tam olarak dört çocuğa sahip olduğu bir ağaç veri yapısıdır. Ayrıca her düğümün iki özelliği vardır:

• **val**: Düğüm 1'lerden oluşan bir ızgarayı temsil ediyorsa **True**, 0'lardan oluşan bir ızgarayı temsil ediyorsa **False**. **isLeaf** değeri **False** iken **val**'e **True** ya da **False** atayabileceğine dikkat et; cevapta ikisi de kabul edilir.
• **isLeaf**: Düğüm ağaçtaki bir yaprak düğümse **True**, dört çocuğu varsa **False**.

```
class Node {
    public boolean val;
    public boolean isLeaf;
    public Node topLeft;
    public Node topRight;
    public Node bottomLeft;
    public Node bottomRight;
}
```

Bir Quad-Tree'yi iki boyutlu bir alandan şu adımlarla oluşturabiliriz:

1. Mevcut ızgarada tüm değerler aynıysa (yani hepsi 1 ya da hepsi 0), **isLeaf**'i **True** yap, **val**'i ızgaranın değerine ayarla, dört çocuğu **Null** yap ve dur.
2. Mevcut ızgarada farklı değerler varsa, **isLeaf**'i **False** yap, **val**'i herhangi bir değere ayarla ve mevcut ızgarayı görseldeki gibi dört alt ızgaraya böl.
3. Her çocuk için uygun alt ızgarayla özyinelemeli (recursive) olarak tekrarla.

Quad-Tree biçimi: Problemi çözmek için bu bölümü okumana gerek yok. Yalnızca buradaki çıktı biçimini anlamak istersen. Çıktı, Quad-Tree'nin seviye sırası (level order) dolaşımıyla serileştirilmiş halini gösterir; **null**, altında düğüm bulunmayan bir yol sonlandırıcısını belirtir. İkili ağacın serileştirilmesine çok benzer. Tek fark, düğümün **[isLeaf, val]** listesi olarak temsil edilmesidir. **isLeaf** ya da **val** değeri **True** ise listede 1 olarak, **False** ise 0 olarak gösteririz.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : grid = [[0,1],[1,0]]
Çıktı : [[0,1],[1,0],[1,1],[1,1],[1,0]]
```

Bu örneğin açıklaması görselde gösterilmiştir. Görselde Quad-Tree'yi temsil ederken 0'ın False, 1'in True anlamına geldiğine dikkat et.

Örnek 2

```
Girdi : grid = [[1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],
               [1,1,1,1,1,1,1,1],[1,1,1,1,1,1,1,1],
               [1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0],
               [1,1,1,1,0,0,0,0],[1,1,1,1,0,0,0,0]]
Çıktı : [[0,1],[1,1],[0,1],[1,1],[1,0],null,null,null,null,
        [1,0],[1,0],[1,1],[1,1]]
```

Izgaradaki tüm değerler aynı değil. Izgarayı dört alt ızgaraya böleriz. Sol üst, sol alt ve sağ alt alt ızgaraların her birinde değerler aynıdır. Sağ üstte ise farklı değerler vardır; bu yüzden onu da, her birinde değerler aynı olan 4 alt ızgaraya böleriz. Açıklama görselde gösterilmiştir.

Kısıtlar

• n == grid.length == grid[i].length
• n == 2^x, burada 0 <= x <= 6

111. Merge k Sorted Lists

k Sıralı Listeyi Birleştir | Seviye: Zor | LeetCode #23
Orijinal soru: leetcode.com/problems/merge-k-sorted-lists/

Soru

k adet bağlı listenin dizisi **lists** veriliyor; her bağlı liste artan sırada sıralıdır.

Tüm bağlı listeleri tek bir sıralı bağlı listede birleştir ve döndür.

Örnekler

Örnek 1

```
Girdi : lists = [[1,4,5],[1,3,4],[2,6]]
Çıktı : [1,1,2,3,4,4,5,6]
```

Bağlı listeler: [1->4->5, 1->3->4, 2->6]. Bunları tek bir sıralı bağlı listede birleştirince: 1->1->2->3->4->4->5->6

Örnek 2

```
Girdi : lists = []
Çıktı : []
```

Örnek 3

```
Girdi : lists = [[]]
Çıktı : []
```

Kısıtlar

- k == lists.length
- 0 <= k <= 10^4
- 0 <= lists[i].length <= 500
- -10^4 <= lists[i][j] <= 10^4
- lists[i] artan sırada sıralıdır.
- lists[i].length değerlerinin toplamı 10^4'ü aşmayacaktır.

112. Maximum Subarray

Maksimum Alt Dizi | Seviye: Orta | LeetCode #53
Orijinal soru: leetcode.com/problems/maximum-subarray/

Soru

Bir tamsayı dizisi **nums** veriliyor. Toplamı en büyük olan alt diziyi (subarray) bul ve bu toplamı döndür.

Örnekler

Örnek 1

```
Girdi : nums = [-2,1,-3,4,-1,2,1,-5,4]
Çıktı : 6
```

[4,-1,2,1] alt dizisi 6 ile en büyük toplama sahiptir.

Örnek 2

```
Girdi : nums = [1]
Çıktı : 1
```

[1] alt dizisi 1 ile en büyük toplama sahiptir.

Örnek 3

```
Girdi : nums = [5,4,-1,7,8]
Çıktı : 23
```

[5,4,-1,7,8] alt dizisi 23 ile en büyük toplama sahiptir.

Kısıtlar

- $1 <= nums.length <= 10^5$
- $-10^4 <= nums[i] <= 10^4$

Ek soru (Follow up)

O(n) çözümü bulduysan, daha ince bir yaklaşım olan **böl ve yönet (divide and conquer)** yaklaşımıyla başka bir çözüm yazmayı dene.

113. Maximum Sum Circular Subarray

Dairesel Alt Dizinin Maksimum Toplamı | Seviye: Orta | LeetCode #918
Orijinal soru: leetcode.com/problems/maximum-sum-circular-subarray/

Soru

Uzunluğu **n** olan dairesel bir tamsayı dizisi **nums** veriliyor. **nums**'ın **boş olmayan bir alt dizisinin** maksimum olası toplamını döndür.

Dairesel dizi, dizinin sonunun dizinin başına bağlandığı anlamına gelir. Resmi olarak **nums[i]**'nin sonraki elemanı **nums[(i + 1) % n]**, **nums[i]**'nin önceki elemanı ise **nums[(i - 1 + n) % n]**'dir.

Bir alt dizi, sabit bir **nums[i]** elemanını **en fazla bir kez** içerebilir. Resmi olarak, **nums[i], nums[i + 1], ..., nums[j]** alt dizisi için **i <= k1, k2 <= j** ve **k1 % n == k2 % n** olacak şekilde hiçbir **k1, k2** çifti yoktur.

Örnekler

Örnek 1

```
Girdi : nums = [1,-2,3,-2]
Çıktı : 3
```

[3] alt dizisi 3 ile maksimum toplama sahiptir.

Örnek 2

```
Girdi : nums = [5,-3,5]
Çıktı : 10
```

[5,5] alt dizisi 5 + 5 = 10 ile maksimum toplama sahiptir.

Örnek 3

```
Girdi : nums = [-3,-2,-3]
Çıktı : -2
```

[-2] alt dizisi -2 ile maksimum toplama sahiptir.

Kısıtlar

- n == nums.length
- $1 <= n <= 3 * 10^4$
- $-3 * 10^4 <= nums[i] <= 3 * 10^4$

114. Search Insert Position

Ekleme Konumunu Ara | Seviye: Kolay | LeetCode #35
Orijinal soru: leetcode.com/problems/search-insert-position/

Soru

Birbirinden farklı tamsayılardan oluşan sıralı bir dizi ve bir hedef değer veriliyor. Hedef bulunursa **indeksini** döndür. Bulunamazsa, sıralı düzeni bozmadan eklenmesi gereken **indeksi** döndür.

O(log n) çalışma zamanı karmaşıklığına sahip bir algoritma yazmalısın.

Örnekler

Örnek 1

```
Girdi : nums = [1,3,5,6], target = 5
Çıktı : 2
```

Örnek 2

```
Girdi : nums = [1,3,5,6], target = 2
Çıktı : 1
```

Örnek 3

```
Girdi : nums = [1,3,5,6], target = 7
Çıktı : 4
```

Kısıtlar

• 1 <= nums.length <= 10^4
• -10^4 <= nums[i] <= 10^4
• nums artan sırada sıralıdır ve elemanları benzersizdir.
• -10^4 <= target <= 10^4

115. Search a 2D Matrix

2 Boyutlu Matriste Ara | Seviye: Orta | LeetCode #74
Orijinal soru: leetcode.com/problems/search-a-2d-matrix/

Soru

m x n boyutunda bir tamsayı matrisi **matrix** veriliyor ve şu iki özelliğe sahip:

• Her satır azalmayan sırada sıralıdır.
• Her satırın ilk tamsayısı, bir önceki satırın son tamsayısından büyüktür.

Bir tamsayı **target** veriliyor. **target**, **matrix** içindeyse **true**, değilse **false** döndür.

Çözümün **O(log(m * n))** zaman karmaşıklığında olmalıdır.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Çıktı : true
```

Örnek 2

```
Girdi : matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Çıktı : false
```

Kısıtlar

• m == matrix.length
• n == matrix[i].length
• 1 <= m, n <= 100
• -10^4 <= matrix[i][j], target <= 10^4

116. Find Peak Element

Tepe Elemanı Bul | Seviye: Orta | LeetCode #162
Orijinal soru: leetcode.com/problems/find-peak-element/

Soru

Tepe elemanı (peak element), komşularından kesinlikle büyük olan elemandır.

0 tabanlı indeksli bir tamsayı dizisi **nums** veriliyor. Bir tepe elemanı bul ve **indeksini** döndür. Dizide birden fazla tepe varsa herhangi birinin indeksini döndürebilirsin.

nums[-1] = nums[n] = -∞ olduğunu varsayabilirsin. Yani bir eleman, dizinin dışındaki hayali -∞ değerli elemanlarla da komşu kabul edilir.

O(log n) zamanda çalışan bir algoritma yazmalısın.

Örnekler

Örnek 1

```
Girdi : nums = [1,2,3,1]
Çıktı : 2
```

3 bir tepe elemanıdır ve fonksiyonun onun indeksi olan 2'yi döndürmesi gerekir.

Örnek 2

```
Girdi : nums = [1,2,1,3,5,6,4]
Çıktı : 5
```

Fonksiyon ya 1. indeksi (tepe elemanı 2) ya da 5. indeksi (tepe elemanı 6) döndürebilir.

Kısıtlar

• 1 <= nums.length <= 1000
• -2^{31} <= nums[i] <= $2^{31} - 1$
• Her geçerli i için nums[i] != nums[i + 1].

117. Search in Rotated Sorted Array

Döndürülmüş Sıralı Dizide Ara | Seviye: Orta | LeetCode #33
Orijinal soru: leetcode.com/problems/search-in-rotated-sorted-array/

Soru

nums tamsayı dizisi, (birbirinden **farklı** değerlerle) artan sırada sıralıdır.

Fonksiyona verilmeden önce **nums**, bilinmeyen bir **k** pivot indeksi (**1 <= k < nums.length**) etrafında **döndürülür**; sonuçta dizi **[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]** olur (0 tabanlı indeksli). Örneğin **[0,1,2,4,5,6,7]** dizisi 3. indeks etrafında döndürülürse **[4,5,6,7,0,1,2]** olabilir.

Döndürmeden **sonraki nums** dizisi ve bir tamsayı **target** veriliyor. **target**, **nums** içindeyse indeksini, değilse **-1** döndür.

O(log n) çalışma zamanı karmaşıklığına sahip bir algoritma yazmalısın.

Örnekler

Örnek 1

```
Girdi : nums = [4,5,6,7,0,1,2], target = 0
Çıktı : 4
```

Örnek 2

```
Girdi : nums = [4,5,6,7,0,1,2], target = 3
Çıktı : -1
```

Örnek 3

```
Girdi : nums = [1], target = 0
Çıktı : -1
```

Kısıtlar

• 1 <= nums.length <= 5000
• -10^4 <= nums[i] <= 10^4
• nums'ın tüm değerleri benzersizdir.
• nums artan sırada sıralıdır ve muhtemelen döndürülmüştür.
• -10^4 <= target <= 10^4

118. Find First and Last Position of Element in Sorted Array

Sıralı Dizide Elemanın İlk ve Son Konumunu Bul | Seviye: Orta | LeetCode #34
Orijinal soru: leetcode.com/problems/find-first-and-last-position-of-element-in-sorted-array/

Soru

Azalmayan sırada sıralanmış bir tamsayı dizisi **nums** ve bir hedef değer **target** veriliyor. Verilen **target** değerinin başlangıç ve bitiş konumunu bul.

target dizide bulunmuyorsa **[-1, -1]** döndür.

O(log n) çalışma zamanı karmaşıklığına sahip bir algoritma yazmalısın.

Örnekler

Örnek 1

```
Girdi : nums = [5,7,7,8,8,10], target = 8
Çıktı : [3,4]
```

Örnek 2

```
Girdi : nums = [5,7,7,8,8,10], target = 6
Çıktı : [-1,-1]
```

Örnek 3

```
Girdi : nums = [], target = 0
Çıktı : [-1,-1]
```

Kısıtlar

• 0 <= nums.length <= 10^5
• -10^9 <= nums[i] <= 10^9
• nums azalmayan sırada sıralıdır.
• -10^9 <= target <= 10^9

119. Find Minimum in Rotated Sorted Array

Döndürülmüş Sıralı Dizide Minimumu Bul | Seviye: Orta | LeetCode #153
Orijinal soru: leetcode.com/problems/find-minimum-in-rotated-sorted-array/

Soru

Uzunluğu **n** olan, artan sırada sıralı bir dizi, **1 ile n kez** arasında döndürülmüş. Örneğin **[0,1,2,4,5,6,7]** dizisi:

- 4 kez döndürülürse **[4,5,6,7,0,1,2]** olabilir.
- 7 kez döndürülürse **[0,1,2,4,5,6,7]** olabilir.

[a[0], a[1], a[2], ..., a[n-1]] dizisinin **1 kez döndürülmesi**, **[a[n-1], a[0], a[1], a[2], ..., a[n-2]]** dizisini verir.

Benzersiz elemanlardan oluşan, sıralı ve döndürülmüş bir **nums** dizisi veriliyor. Bu dizinin **minimum elemanını** döndür.

O(log n) zamanda çalışan bir algoritma yazmalısın.

Örnekler

Örnek 1

```
Girdi : nums = [3,4,5,1,2]
Çıktı : 1
```

Orijinal dizi [1,2,3,4,5]'tir ve 3 kez döndürülmüştür.

Örnek 2

```
Girdi : nums = [4,5,6,7,0,1,2]
Çıktı : 0
```

Orijinal dizi [0,1,2,4,5,6,7]'dir ve 4 kez döndürülmüştür.

Örnek 3

```
Girdi : nums = [11,13,15,17]
Çıktı : 11
```

Orijinal dizi [11,13,15,17]'dir ve 4 kez döndürülmüştür.

Kısıtlar

- n == nums.length
- 1 <= n <= 5000
- -5000 <= nums[i] <= 5000
- nums'ın tüm tamsayıları benzersizdir.
- nums sıralıdır ve 1 ile n kez arasında döndürülmüştür.

120. Median of Two Sorted Arrays

İki Sıralı Dizinin Medyanı | Seviye: Zor | LeetCode #4
Orijinal soru: leetcode.com/problems/median-of-two-sorted-arrays/

Soru

Sırasıyla **m** ve **n** boyutlarında, sıralı iki dizi **nums1** ve **nums2** veriliyor.

İki sıralı dizinin **medyanını** döndür.

Toplam çalışma zamanı karmaşıklığı **O(log (m+n))** olmalıdır.

Örnekler

Örnek 1

```
Girdi : nums1 = [1,3], nums2 = [2]
Çıktı : 2.00000
```

Birleştirilmiş dizi [1,2,3] olur ve medyan 2'dir.

Örnek 2

```
Girdi : nums1 = [1,2], nums2 = [3,4]
Çıktı : 2.50000
```

Birleştirilmiş dizi [1,2,3,4] olur ve medyan (2 + 3) / 2 = 2.5'tir.

Kısıtlar

• nums1.length == m
• nums2.length == n
• 0 <= m <= 1000
• 0 <= n <= 1000
• 1 <= m + n <= 2000
• -10^6 <= nums1[i], nums2[i] <= 10^6

121. Kth Largest Element in an Array

Dizideki K. En Büyük Eleman | Seviye: Orta | LeetCode #215
Orijinal soru: leetcode.com/problems/kth-largest-element-in-an-array/

Soru

Bir tamsayı dizisi **nums** ve bir tamsayı **k** veriliyor. Dizideki **k**. en büyük elemanı döndür.

Bunun, sıralı düzendeki **k**. en büyük eleman olduğuna, **k**. farklı eleman olmadığına dikkat et.

Sıralama yapmadan çözebilir misin?

Örnekler

Örnek 1

```
Girdi : nums = [3,2,1,5,6,4], k = 2
Çıktı : 5
```

Örnek 2

```
Girdi : nums = [3,2,3,1,2,4,5,5,6], k = 4
Çıktı : 4
```

Kısıtlar

• 1 <= k <= nums.length <= 10^5
• -10^4 <= nums[i] <= 10^4

122. IPO

Halka Arz (IPO) | Seviye: Zor | LeetCode #502
Orijinal soru: leetcode.com/problems/ipo/

Soru

LeetCode, Halka Arz'dan (**IPO**) önce sermayesini artırmak için sermayesini en üst düzeye çıkarmak istiyor. Kaynakları sınırlı olduğu için IPO'dan önce en fazla **k** farklı proje tamamlayabilir. Sermayeyi en üst düzeye çıkarmak için yardımcı ol.

Sana **n** proje veriliyor; **i**. projenin saf kârı **profits[i]** ve projeyi başlatmak için gereken minimum sermaye **capital[i]**'dir.

Başlangıçta **w** kadar sermayen var. Bir projeyi bitirdiğinde kârını toplam sermayene eklersin.

Verilen projeler arasından **en fazla k farklı proje** seçerek sermayeni en üst düzeye çıkar ve nihai maksimize edilmiş sermayeyi döndür.

Cevabın 32 bitlik işaretli tamsayıya sığacağı garanti edilmiştir.

Örnekler

Örnek 1

```
Girdi : k = 2, w = 0, profits = [1,2,3], capital = [0,1,1]
Çıktı : 4
```

Başlangıç sermayen 0 olduğu için yalnızca 0. projeyi başlatabilirsin. Projeyi bitirince 1 kâr elde edersin ve sermayen 1 olur. Sermayen 1 iken 1. veya 2. projeyi başlatabilirsin. En fazla 2 proje seçebildiğin için maksimum sermayeye ulaşmak adına 2. projeyi seçmelisin. Nihai maksimize edilmiş sermaye 0 + 1 + 3 = 4.

Örnek 2

```
Girdi : k = 3, w = 0, profits = [1,2,3], capital = [0,1,2]
Çıktı : 6
```

Kısıtlar

- $1 <= k <= 10^5$
- $0 <= w <= 10^9$
- n == profits.length
- n == capital.length
- $1 <= n <= 10^5$
- $0 <= profits[i] <= 10^4$
- $0 <= capital[i] <= 10^9$

123. Find K Pairs with Smallest Sums

En Küçük Toplamlı K Çifti Bul | Seviye: Orta | LeetCode #373
Orijinal soru: leetcode.com/problems/find-k-pairs-with-smallest-sums/

Soru

Azalmayan sırada sıralı iki tamsayı dizisi **nums1** ve **nums2** ile bir tamsayı **k** veriliyor.

(u, v) çifti, **u**'nun **nums1**'den, **v**'nin **nums2**'den bir eleman olduğu çifti tanımlar.

En küçük toplamlara sahip **k** çifti, yani **(u_1, v_1), (u_2, v_2), ..., (u_k, v_k)** çiftlerini döndür.

Örnekler

Örnek 1

```
Girdi : nums1 = [1,7,11], nums2 = [2,4,6], k = 3
Çıktı : [[1,2],[1,4],[1,6]]
```

Dizideki ilk 3 çift şu sırayla döndürülür:
[1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]

Örnek 2

```
Girdi : nums1 = [1,1,2], nums2 = [1,2,3], k = 2
Çıktı : [[1,1],[1,1]]
```

Dizideki ilk 2 çift şu sırayla döndürülür: [1,1],[1,1],[1,2],[2,1],[1,2],[2,2],[1,3],[1,3],[2,3]

Kısıtlar

- 1 <= nums1.length, nums2.length <= 10^5
- -10^9 <= nums1[i], nums2[i] <= 10^9
- nums1 ve nums2 azalmayan sırada sıralıdır.
- 1 <= k <= 10^4
- k <= nums1.length * nums2.length

124. Find Median from Data Stream

Veri Akışından Medyan Bul | Seviye: Zor | LeetCode #295
Orijinal soru: leetcode.com/problems/find-median-from-data-stream/

Soru

Medyan, sıralı bir tamsayı listesindeki ortadaki değerdir. Listenin boyutu çiftse tek bir orta değer yoktur; medyan, iki orta değerin ortalamasıdır.

• Örneğin **arr = [2,3,4]** için medyan **3**'tür.
• Örneğin **arr = [2,3]** için medyan **(2 + 3) / 2 = 2.5**'tir.

MedianFinder sınıfını gerçekle:

• **MedianFinder()**: **MedianFinder** nesnesini başlatır.
• **void addNum(int num)**: Veri akışındaki **num** tamsayısını veri yapısına ekler.
• **double findMedian()**: Şimdiye kadarki tüm elemanların medyanını döndürür.
Gerçek cevabın 10^{-5} yakınındaki cevaplar kabul edilir.

Örnekler

Örnek 1

```
Girdi :
["MedianFinder","addNum","addNum","findMedian","addNum","findMedian"]
        [[],[1],[2],[],[3],[]]
Çıktı :
        [null,null,null,1.5,null,2.0]
```

MedianFinder nesnesi oluşturulur. **addNum(1)** → dizi [1]. **addNum(2)** → dizi [1, 2]. **findMedian()** → 1.5 döner ((1 + 2) / 2). **addNum(3)** → dizi [1, 2, 3]. **findMedian()** → 2.0 döner.

Kısıtlar

• -10^5 <= num <= 10^5
• **findMedian** çağrıldığında veri yapısında en az bir eleman olacaktır.
• **addNum** ve **findMedian** için toplamda en fazla $5 * 10^4$ çağrı yapılacaktır.

Ek soru (Follow up)

• Veri akışındaki tüm tamsayılar **[0, 100]** aralığındaysa çözümünü nasıl optimize ederdin?
• Veri akışındaki sayıların **%99**'u **[0, 100]** aralığındaysa çözümünü nasıl optimize ederdin?

125. Add Binary

İkili Sayıları Topla | Seviye: Kolay | LeetCode #67
Orijinal soru: leetcode.com/problems/add-binary/

Soru

İki ikili dizgi (binary string) **a** ve **b** veriliyor. Toplamlarını bir ikili dizgi olarak döndür.

Örnekler

Örnek 1

```
Girdi : a = "11", b = "1"
Çıktı : "100"
```

Örnek 2

```
Girdi : a = "1010", b = "1011"
Çıktı : "10101"
```

Kısıtlar

- 1 <= a.length, b.length <= 10^4
- a ve b yalnızca '0' veya '1' karakterlerinden oluşur.
- Her dizge, “0” dışında başında sıfır içermez.

126. Reverse Bits

Bitleri Ters Çevir | Seviye: Kolay | LeetCode #190
Orijinal soru: leetcode.com/problems/reverse-bits/

Soru

Verilen 32 bitlik **işaretsiz (unsigned)** bir tamsayının bitlerini ters çevir.

Not:

• Java gibi bazı dillerde işaretsiz tamsayı türü yoktur. Bu durumda hem girdi hem çıktı işaretli tamsayı türü olarak verilir ve bu, sonucu etkilememelidir; çünkü iç ikili gösterim aynıdır.
• Java'da derleyici işaretli tamsayıları **2'ye tümleyen (2's complement)** gösterimiyle temsil eder. Bu yüzden Örnek 2'de girdi işaretli tamsayı **-3**, çıktı ise işaretli tamsayı **-1073741825**'tir.

Örnekler

Örnek 1

```
Girdi : n = 00000010100101000001111010011100
Çıktı : 964176192 (00111001011110000010100101000000)
```

Girdi ikili dizgisi 00000010100101000001111010011100, işaretsiz tamsayı olarak 43261596'yı temsil eder; bu yüzden 964176192 döndürülür. Bunun ikili gösterimi 00111001011110000010100101000000'dır.

Örnek 2

```
Girdi : n = 11111111111111111111111111111101
Çıktı : 3221225471 (10111111111111111111111111111111)
```

Girdi ikili dizgisi 11111111111111111111111111111101, işaretsiz tamsayı olarak 4294967293'ü temsil eder; bu yüzden 3221225471 döndürülür. Bunun ikili gösterimi 10111111111111111111111111111111'dir.

Kısıtlar

• Girdi, uzunluğu 32 olan bir ikili dizgidir.

Ek soru (Follow up)

Bu fonksiyon çok sayıda kez çağrılırsa nasıl optimize ederdin?

127. Number of 1 Bits

1 Bitlerinin Sayısı | Seviye: Kolay | LeetCode #191
Orijinal soru: leetcode.com/problems/number-of-1-bits/

Soru

İşaretsiz bir tamsayı **n** veriliyor. İkili gösteriminde **1** değerine sahip bit sayısını (**Hamming ağırlığı** olarak da bilinir) döndür.

Örnekler

Örnek 1

```
Girdi : n = 11
Çıktı : 3
```

Girdi ikili dizgisi 1011'dir ve toplam üç tane 1 biti vardır.

Örnek 2

```
Girdi : n = 128
Çıktı : 1
```

Girdi ikili dizgisi 10000000'dır ve toplam bir tane 1 biti vardır.

Örnek 3

```
Girdi : n = 2147483645
Çıktı : 30
```

Girdi ikili dizgisi 11111111111111111111111111111101'dir ve toplam otuz tane 1 biti vardır.

Kısıtlar

• $1 <= n <= 2^{31} - 1$

Ek soru (Follow up)

Bu fonksiyon çok sayıda kez çağrılırsa nasıl optimize ederdin?

128. Single Number

Tek Sayı | Seviye: Kolay | LeetCode #136
Orijinal soru: leetcode.com/problems/single-number/

Soru

Boş olmayan bir tamsayı dizisi **nums** veriliyor. Yalnızca bir eleman bir kez görünür, diğer her eleman iki kez görünür. Yalnızca bir kez görünen o elemanı bul.

Doğrusal (linear) çalışma zamanı karmaşıklığına ve yalnızca **sabit ekstra alana** sahip bir çözüm gerçekleştirmelisin.

Örnekler

Örnek 1

```
Girdi : nums = [2,2,1]
Çıktı : 1
```

Örnek 2

```
Girdi : nums = [4,1,2,1,2]
Çıktı : 4
```

Örnek 3

```
Girdi : nums = [1]
Çıktı : 1
```

Kısıtlar

• $1 <= nums.length <= 3 * 10^4$
• $-3 * 10^4 <= nums[i] <= 3 * 10^4$
• Yalnızca bir kez görünen bir eleman dışında, dizideki her eleman iki kez görünür.

129. Single Number II

Tek Sayı II | Seviye: Orta | LeetCode #137
Orijinal soru: leetcode.com/problems/single-number-ii/

Soru

Bir tamsayı dizisi **nums** veriliyor; burada bir eleman tam olarak **bir kez**, diğer her eleman tam olarak **üç kez** görünür. Yalnızca bir kez görünen elemanı bul.

Doğrusal (linear) çalışma zamanı karmaşıklığına ve yalnızca **sabit ekstra alana** sahip bir çözüm gerçekleştirmelisin.

Örnekler

Örnek 1

```
Girdi : nums = [2,2,3,2]
Çıktı : 3
```

Örnek 2

```
Girdi : nums = [0,1,0,1,0,1,99]
Çıktı : 99
```

Kısıtlar

• 1 <= nums.length <= $3 * 10^4$
• -2^{31} <= nums[i] <= $2^{31} - 1$
• nums içindeki her eleman tam üç kez görünür; yalnızca bir eleman tam bir kez görünür.

130. Bitwise AND of Numbers Range

Sayı Aralığının Bitsel AND'i | Seviye: Orta | LeetCode #201
Orijinal soru: leetcode.com/problems/bitwise-and-of-numbers-range/

Soru

İki tamsayı **left** ve **right** veriliyor. **[left, right]** aralığındaki (uçlar dahil) tüm sayıların **bitsel AND** sonucunu döndür.

Örnekler

Örnek 1

```
Girdi : left = 5, right = 7
Çıktı : 4
```

Örnek 2

```
Girdi : left = 0, right = 0
Çıktı : 0
```

Örnek 3

```
Girdi : left = 1, right = 2147483647
Çıktı : 0
```

Kısıtlar

• $0 <= left <= right <= 2^{31} - 1$

131. Palindrome Number

Palindrom Sayı | Seviye: Kolay | LeetCode #9
Orijinal soru: leetcode.com/problems/palindrome-number/

Soru

Bir tamsayı **x** palindromsa **true**, değilse **false** döndür.

Palindrom, soldan sağa ve sağdan sola okunduğunda aynı olan sayıdır. Örneğin 121 palindromdur, 123 değildir.

Örnekler

Örnek 1

```
Girdi : x = 121
Çıktı : true
```

Soldan sağa 121, sağdan sola 121 okunur.

Örnek 2

```
Girdi : x = -121
Çıktı : false
```

Soldan sağa -121 okunur. Sağdan sola ise 121- olur. Bu yüzden palindrom değildir.

Örnek 3

```
Girdi : x = 10
Çıktı : false
```

Sağdan sola 01 okunur. Bu yüzden palindrom değildir.

Kısıtlar

• $-2^{31} <= x <= 2^{31} - 1$

Ek soru (Follow up)

Tamsayıyı dizgiye (string) çevirmeden çözebilir misin?

132. Plus One

Bir Ekle | Seviye: Kolay | LeetCode #66
Orijinal soru: leetcode.com/problems/plus-one/

Soru

Büyük bir tamsayıyı temsil eden **digits** adlı bir tamsayı dizisi veriliyor; **digits[i]**, tamsayının **i**. rakamıdır. Rakamlar en anlamlıdan en az anlamlıya doğru soldan sağa sıralanmıştır. Büyük tamsayı başında sıfır içermez.

Büyük tamsayıyı **bir artır** ve ortaya çıkan rakam dizisini döndür.

Örnekler

Örnek 1

```
Girdi : digits = [1,2,3]
Çıktı : [1,2,4]
```

Dizi 123 sayısını temsil eder. 123 + 1 = 124 olduğundan sonuç [1,2,4] olur.

Örnek 2

```
Girdi : digits = [4,3,2,1]
Çıktı : [4,3,2,2]
```

Dizi 4321 sayısını temsil eder. 4321 + 1 = 4322 olduğundan sonuç [4,3,2,2] olur.

Örnek 3

```
Girdi : digits = [9]
Çıktı : [1,0]
```

Dizi 9 sayısını temsil eder. 9 + 1 = 10 olduğundan sonuç [1,0] olur.

Kısıtlar

- 1 <= digits.length <= 100
- 0 <= digits[i] <= 9
- digits başında sıfır içermez.

133. Factorial Trailing Zeroes

Faktöriyelin Sondaki Sıfırları | Seviye: Orta | LeetCode #172
Orijinal soru: leetcode.com/problems/factorial-trailing-zeroes/

Soru

Bir tamsayı **n** veriliyor. **n!** sayısındaki **sondaki sıfırların** sayısını döndür.

n! = n * (n - 1) * (n - 2) * ... * 3 * 2 * 1 olduğuna dikkat et.

Örnekler

Örnek 1

```
Girdi : n = 3
Çıktı : 0
```

3! = 6, sondaki sıfır yok.

Örnek 2

```
Girdi : n = 5
Çıktı : 1
```

5! = 120, bir tane sondaki sıfır var.

Örnek 3

```
Girdi : n = 0
Çıktı : 0
```

Kısıtlar

• $0 <= n <= 10^4$

Ek soru (Follow up)

Logaritmik zaman karmaşıklığında bir çözüm yazabilir misin?

134. Sqrt(x)

Karekök | Seviye: Kolay | LeetCode #69
Orijinal soru: leetcode.com/problems/sqrtx/

Soru

Negatif olmayan bir tamsayı **x** veriliyor. **x**'in karekökünü, en yakın tamsayıya **aşağı yuvarlanmış** halde döndür. Döndürülen tamsayı da **negatif olmamalıdır**.

Yerleşik üs alma fonksiyonlarını veya operatörlerini **kullanamazsın**.

• Örneğin C++'ta **pow(x, 0.5)** ya da Python'da **x ** 0.5** kullanma.

Örnekler

Örnek 1

```
Girdi : x = 4
Çıktı : 2
```

4'ün karekökü 2'dir, bu yüzden 2 döndürürüz.

Örnek 2

```
Girdi : x = 8
Çıktı : 2
```

8'in karekökü 2.82842...'dir ve aşağı yuvarlanınca 2 olur; bu yüzden 2 döndürürüz.

Kısıtlar

• $0 <= x <= 2^{31} - 1$

135. Pow(x, n)

Üs Alma | Seviye: Orta | LeetCode #50
Orijinal soru: leetcode.com/problems/powx-n/

Soru

x sayısının **n**. kuvvetini hesaplayan **pow(x, n)** fonksiyonunu gerçekle (yani $\mathbf{x^n}$).

Örnekler

Örnek 1

```
Girdi : x = 2.00000, n = 10
Çıktı : 1024.00000
```

Örnek 2

```
Girdi : x = 2.10000, n = 3
Çıktı : 9.26100
```

Örnek 3

```
Girdi : x = 2.00000, n = -2
Çıktı : 0.25000
```

$2^{-2} = 1/2^2 = 1/4 = 0.25$

Kısıtlar

- $-100.0 < x < 100.0$
- $-2^{31} <= n <= 2^{31} - 1$
- n bir tamsayıdır.
- Ya x sıfır değildir ya da $n > 0$'dır.
- $-10^4 <= x^n <= 10^4$

136. Max Points on a Line

Bir Doğru Üzerindeki Maksimum Nokta | Seviye: Zor | LeetCode #149
Orijinal soru: leetcode.com/problems/max-points-on-a-line/

Soru

X-Y düzlemindeki bir noktayı temsil eden **points** dizisi veriliyor; **points[i] = [x_i, y_i]**. Aynı **düz doğru** üzerinde bulunan **maksimum nokta sayısını** döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : points = [[1,1],[2,2],[3,3]]
Çıktı : 3
```

Örnek 2

```
Girdi : points = [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]
Çıktı : 4
```

Kısıtlar

- 1 <= points.length <= 300
- points[i].length == 2
- -10^4 <= x_i, y_i <= 10^4
- points içindeki tüm noktalar benzersizdir.

137. Climbing Stairs

Merdiven Tırmanma | Seviye: Kolay | LeetCode #70
Orijinal soru: leetcode.com/problems/climbing-stairs/

Soru

Bir merdiveni tırmanıyorsun. Tepeye ulaşmak için **n** basamak var.

Her seferinde **1** ya da **2** basamak tırmanabilirsin. Tepeye kaç **farklı yolla** çıkabilirsin?

Örnekler

Örnek 1

```
Girdi : n = 2
Çıktı : 2
```

Tepeye çıkmanın iki yolu var:

1. 1 adım + 1 adım
2. 2 adım

Örnek 2

```
Girdi : n = 3
Çıktı : 3
```

Tepeye çıkmanın üç yolu var:

1. 1 adım + 1 adım + 1 adım
2. 1 adım + 2 adım
3. 2 adım + 1 adım

Kısıtlar

• 1 <= n <= 45

138. House Robber

Ev Soyguncusu | Seviye: Orta | LeetCode #198
Orijinal soru: leetcode.com/problems/house-robber/

Soru

Sen, bir sokaktaki evleri soymayı planlayan profesyonel bir hırsızsın. Her evde belli miktarda para saklı. Seni durduran tek kısıt, bitişik evlerin güvenlik sistemlerinin birbirine bağlı olmasıdır: **aynı gece iki bitişik ev soyulursa otomatik olarak polise haber verilir**.

Her evdeki para miktarını temsil eden bir tamsayı dizisi **nums** veriliyor. Polise haber vermeden bu gece soyabileceğin **maksimum para miktarını** döndür.

Örnekler

Örnek 1

```
Girdi : nums = [1,2,3,1]
Çıktı : 4
```

1. evi (para = 1) soy, sonra 3. evi (para = 3) soy. Toplam soyulan miktar = 1 + 3 = 4.

Örnek 2

```
Girdi : nums = [2,7,9,3,1]
Çıktı : 12
```

1. evi (para = 2), 3. evi (para = 9) ve 5. evi (para = 1) soy. Toplam soyulan miktar = 2 + 9 + 1 = 12.

Kısıtlar

• 1 <= nums.length <= 100
• 0 <= nums[i] <= 400

139. Word Break

Kelime Bölme | Seviye: Orta | LeetCode #139
Orijinal soru: leetcode.com/problems/word-break/

Soru

Bir **s** dizgisi ve bir dizgi sözlüğü **wordDict** veriliyor. **s**, sözlükte bulunan bir veya daha fazla kelimenin boşlukla ayrılmış bir dizisine bölünebiliyorsa **true** döndür.

Dikkat: Sözlükteki aynı kelime, bölmede birden fazla kez kullanılabilir.

Örnekler

Örnek 1

```
Girdi : s = "leetcode", wordDict = ["leet","code"]
Çıktı : true
```

“leetcode”, “leet code” olarak bölünebilir.

Örnek 2

```
Girdi : s = "applepenapple", wordDict = ["apple","pen"]
Çıktı : true
```

“applepenapple”, “apple pen apple” olarak bölünebilir. “apple” kelimesinin yeniden kullanılabildiğine dikkat et.

Örnek 3

```
Girdi : s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Çıktı : false
```

Kısıtlar

• 1 <= s.length <= 300
• 1 <= wordDict.length <= 1000
• 1 <= wordDict[i].length <= 20
• s ve wordDict[i] küçük İngilizce harflerden oluşur.
• wordDict içindeki tüm dizgiler benzersizdir.

140. Coin Change

Para Bozma | Seviye: Orta | LeetCode #322
Orijinal soru: leetcode.com/problems/coin-change/

Soru

Farklı değerlerdeki madeni paraları temsil eden bir tamsayı dizisi **coins** ve toplam para miktarını temsil eden bir tamsayı **amount** veriliyor.

Bu miktarı oluşturmak için gereken **en az madeni para sayısını** döndür. Bu miktar, madeni paraların hiçbir kombinasyonuyla oluşturulamıyorsa **-1** döndür.

Her madeni paradan **sonsuz sayıda** olduğunu varsayabilirsin.

Örnekler

Örnek 1

```
Girdi : coins = [1,2,5], amount = 11
Çıktı : 3
```
11 = 5 + 5 + 1

Örnek 2

```
Girdi : coins = [2], amount = 3
Çıktı : -1
```

Örnek 3

```
Girdi : coins = [1], amount = 0
Çıktı : 0
```

Kısıtlar

• 1 <= coins.length <= 12
• 1 <= coins[i] <= 2^{31} - 1
• 0 <= amount <= 10^4

141. Longest Increasing Subsequence

En Uzun Artan Alt Dizi | Seviye: Orta | LeetCode #300
Orijinal soru: leetcode.com/problems/longest-increasing-subsequence/

Soru

Bir tamsayı dizisi **nums** veriliyor. En uzun **kesin artan alt dizinin** (subsequence) uzunluğunu döndür.

Örnekler

Örnek 1

```
Girdi : nums = [10,9,2,5,3,7,101,18]
Çıktı : 4
```

En uzun artan alt dizi [2,3,7,101]'dir; bu yüzden uzunluk 4'tür.

Örnek 2

```
Girdi : nums = [0,1,0,3,2,3]
Çıktı : 4
```

Örnek 3

```
Girdi : nums = [7,7,7,7,7,7,7]
Çıktı : 1
```

Kısıtlar

- 1 <= nums.length <= 2500
- -10^4 <= nums[i] <= 10^4

Ek soru (Follow up)

O(n log(n)) zaman karmaşıklığında çalışan bir çözüm bulabilir misin?

142. Triangle

Üçgen | Seviye: Orta | LeetCode #120
Orijinal soru: leetcode.com/problems/triangle/

Soru

Bir **triangle** dizisi veriliyor. **Tepeden tabana en küçük yol toplamını** döndür.

Her adımda, altındaki satırdaki **bitişik** bir sayıya geçebilirsin. Daha açık söylemek gerekirse, mevcut satırda **i** indeksindeysen, bir sonraki satırda **i** indeksine ya da **i + 1** indeksine geçebilirsin.

Örnekler

Örnek 1

```
Girdi : triangle = [[2],[3,4],[6,5,7],[4,1,8,3]]
Çıktı : 11
```

Verilen üçgen şöyle gösterilir:

```
   2
  3 4
 6 5 7
4 1 8 3
```

Tepeden tabana minimum yol toplamı 2 + 3 + 5 + 1 = 11'dir.

Örnek 2

```
Girdi : triangle = [[-10]]
Çıktı : -10
```

Kısıtlar

- 1 <= triangle.length <= 200
- triangle[0].length == 1
- triangle[i].length == triangle[i - 1].length + 1
- -10^4 <= triangle[i][j] <= 10^4

Ek soru (Follow up)

Yalnızca **O(n)** ekstra alan kullanarak bunu yapabilir misin? (Burada **n**, üçgendeki toplam satır sayısıdır.)

143. Minimum Path Sum

Minimum Yol Toplamı | Seviye: Orta | LeetCode #64
Orijinal soru: leetcode.com/problems/minimum-path-sum/

Soru

Negatif olmayan sayılarla dolu **m x n** boyutunda bir **grid** veriliyor. Sol üst köşeden sağ alt köşeye giden ve yol boyunca tüm sayıların toplamını **en aza indiren** bir yol bul.

Not: Her seferinde yalnızca aşağı ya da sağa hareket edebilirsin.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : grid = [[1,3,1],[1,5,1],[4,2,1]]
Çıktı : 7
```

1 → 3 → 1 → 1 → 1 yolu toplamı en aza indirir.

Örnek 2

```
Girdi : grid = [[1,2,3],[4,5,6]]
Çıktı : 12
```

Kısıtlar

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 200
- 0 <= grid[i][j] <= 200

144. Unique Paths II

Benzersiz Yollar II | Seviye: Orta | LeetCode #63
Orijinal soru: leetcode.com/problems/unique-paths-ii/

Soru

m x n boyutunda bir tamsayı dizisi **obstacleGrid** veriliyor. Robot başlangıçta sol üst köşede (**grid[0][0]**) duruyor. Robot, sağ alt köşeye (**grid[m-1][n-1]**) gitmeye çalışıyor. Robot her seferinde yalnızca **aşağı** ya da **sağa** hareket edebilir.

Izgarada engeller ve boş alanlar işaretlenmiştir; engel ve boş alan sırasıyla **1** ve **0** ile gösterilir. Robotun izlediği yol, engel içeren hiçbir kareyi **içeremez**.

Sağ alt köşeye ulaşan olası **benzersiz yolların sayısını** döndür.

Test durumları, cevabın **2 * 10^9**'dan küçük veya eşit olacağı şekilde oluşturulmuştur.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]
Çıktı : 2
```

3x3 ızgaranın tam ortasında bir engel var. Sol üst köşeden sağ alt köşeye ulaşmanın 2 yolu var:
1. Sağ → Sağ → Aşağı → Aşağı
2. Aşağı → Aşağı → Sağ → Sağ

Örnek 2

```
Girdi : obstacleGrid = [[0,1],[0,0]]
Çıktı : 1
```

Kısıtlar

• m == obstacleGrid.length
• n == obstacleGrid[i].length
• 1 <= m, n <= 100
• obstacleGrid[i][j] 0 ya da 1'dir.

145. Longest Palindromic Substring

En Uzun Palindromik Alt Dizgi | Seviye: Orta | LeetCode #5
Orijinal soru: leetcode.com/problems/longest-palindromic-substring/

Soru

Bir **s** dizgisi veriliyor. **s** içindeki en uzun **palindromik alt dizgiyi** (substring) döndür.

Örnekler

Örnek 1

```
Girdi : s = "babad"
Çıktı : "bab"
```

“aba” da geçerli bir cevaptır.

Örnek 2

```
Girdi : s = "cbbd"
Çıktı : "bb"
```

Kısıtlar

• 1 <= s.length <= 1000
• s yalnızca rakamlardan ve İngilizce harflerden oluşur.

146. Interleaving String

İç İçe Geçmiş Dizgi | Seviye: Orta | LeetCode #97
Orijinal soru: leetcode.com/problems/interleaving-string/

Soru

s1, **s2** ve **s3** dizgileri veriliyor. **s3**'ün, **s1** ve **s2**'nin **iç içe geçmesiyle (interleaving)** oluşup oluşmadığını bul.

s ve **t** dizgilerinin iç içe geçmesi şöyle tanımlanır: **s** ve **t** sırasıyla **n** ve **m** adet **boş olmayan** alt dizgiye bölünür, öyle ki:

• **$s = s_1 + s_2 + ... + s_n$**
• **$t = t_1 + t_2 + ... + t_m$**
• **$|n - m| <= 1$**
• İç içe geçmiş dizgi, **$s_1 + t_1 + s_2 + t_2 + s_3 + t_3 + ...$** ya da **$t_1 + s_1 + t_2 + s_2 + t_3 + s_3 + ...$** biçimindedir.

Not: a + b, **a** ve **b** dizgilerinin birleştirilmesini (concatenation) ifade eder.

Örnekler

Örnek 1

```
Girdi : s1 = "aabcc", s2 = "dbbca", s3 = "aadbbcbcac"
Çıktı : true
```

s1 = “aa” + “bc” + “c” ve s2 = “dbbc” + “a” olarak bölünebilir. Bunları iç içe geçirmek “aa” + “dbbc” + “bc” + “a” + “c” = “aadbbcbcac” verir. s3 elde edilebildiği için true döndürürüz.

Örnek 2

```
Girdi : s1 = "aabcc", s2 = "dbbca", s3 = "aadbbbaccc"
Çıktı : false
```

s3'ü elde etmenin bir yolu yoktur.

Örnek 3

```
Girdi : s1 = "", s2 = "", s3 = ""
Çıktı : true
```

Kısıtlar

• 0 <= s1.length, s2.length <= 100
• 0 <= s3.length <= 200
• s1, s2 ve s3 küçük İngilizce harflerden oluşur.

Ek soru (Follow up)

Yalnızca **O(s2.length)** ekstra bellek alanı kullanarak bu soruyu çözebilir misin?

147. Edit Distance

Düzenleme Mesafesi | Seviye: Orta | LeetCode #72
Orijinal soru: leetcode.com/problems/edit-distance/

Soru

İki dizgi **word1** ve **word2** veriliyor. **word1**'i **word2**'ye dönüştürmek için gereken **minimum işlem sayısını** döndür.

Bir kelime üzerinde şu üç işleme izin verilir:

• Bir karakter ekle
• Bir karakteri sil
• Bir karakteri başka bir karakterle değiştir

Örnekler

Örnek 1

```
Girdi : word1 = "horse", word2 = "ros"
Çıktı : 3
```

horse → rorse (‘h’ yerine ‘r’)
rorse → rose (‘r’ silinir)
rose → ros (‘e’ silinir)

Örnek 2

```
Girdi : word1 = "intention", word2 = "execution"
Çıktı : 5
```

intention → inention (‘t’ silinir)
inention → enention (‘i’ yerine ‘e’)
enention → exention (‘n’ yerine ‘x’)
exention → exection (‘n’ yerine ‘c’)
exection → execution (‘u’ eklenir)

Kısıtlar

• 0 <= word1.length, word2.length <= 500
• word1 ve word2 küçük İngilizce harflerden oluşur.

148. Best Time to Buy and Sell Stock III

Hisse Senedini Al-Sat İçin En İyi Zaman III | Seviye: Zor | LeetCode #123
Orijinal soru: leetcode.com/problems/best-time-to-buy-and-sell-stock-iii/

Soru

Bir **prices** dizisi veriliyor; **prices[i]**, belirli bir hisse senedinin **i**. gündeki fiyatıdır.

Maksimum kârı bul. **En fazla iki işlem** tamamlayabilirsin.

Not: Aynı anda birden fazla işlemde bulunamazsın (yani yeniden almadan önce hisseyi satmalısın).

Örnekler

Örnek 1

```
Girdi : prices = [3,3,5,0,0,3,1,4]
Çıktı : 6
```

4. gün al (fiyat = 0) ve 6. gün sat (fiyat = 3), kâr = 3 - 0 = 3. Sonra 7. gün al (fiyat = 1) ve 8. gün sat (fiyat = 4), kâr = 4 - 1 = 3.

Örnek 2

```
Girdi : prices = [1,2,3,4,5]
Çıktı : 4
```

1. gün al (fiyat = 1) ve 5. gün sat (fiyat = 5), kâr = 5 - 1 = 4. Ardışık günlerde art arda alım yapıp sonra satamazsın; çünkü aynı anda birden fazla işlemde bulunmuş olursun. Yeniden almadan önce satmalısın.

Örnek 3

```
Girdi : prices = [7,6,4,3,1]
Çıktı : 0
```

Bu durumda hiçbir işlem yapılmaz; yani maksimum kâr = 0.

Kısıtlar

• 1 <= prices.length <= 10^5
• 0 <= prices[i] <= 10^5

149. Best Time to Buy and Sell Stock IV

Hisse Senedini Al-Sat İçin En İyi Zaman IV | Seviye: Zor | LeetCode #188
Orijinal soru: leetcode.com/problems/best-time-to-buy-and-sell-stock-iv/

Soru

Bir tamsayı dizisi **prices** ve bir tamsayı **k** veriliyor; **prices[i]**, belirli bir hisse senedinin **i**. gündeki fiyatıdır.

Maksimum kârı bulmak için **en fazla k işlem** yapabilirsin. Elde edebileceğin maksimum kârı döndür.

Not: Aynı anda birden fazla işlemde bulunamazsın (yani yeniden almadan önce hisseyi satmalısın).

Örnekler

Örnek 1

```
Girdi : k = 2, prices = [2,4,1]
Çıktı : 2
```

1. gün al (fiyat = 2) ve 2. gün sat (fiyat = 4), kâr = 4 - 2 = 2.

Örnek 2

```
Girdi : k = 2, prices = [3,2,6,5,0,3]
Çıktı : 7
```

2. gün al (fiyat = 2) ve 3. gün sat (fiyat = 6), kâr = 6 - 2 = 4. Sonra 5. gün al (fiyat = 0) ve 6. gün sat (fiyat = 3), kâr = 3 - 0 = 3.

Kısıtlar

• 1 <= k <= 100
• 1 <= prices.length <= 1000
• 0 <= prices[i] <= 1000

150. Maximal Square

En Büyük Kare | Seviye: Orta | LeetCode #221
Orijinal soru: leetcode.com/problems/maximal-square/

Soru

'**0**' ve '**1**' ile doldurulmuş, **m x n** boyutunda bir ikili **matrix** veriliyor. Yalnızca **1**'lerden oluşan en büyük kareyi bul ve **alanını** döndür.

Orijinal soruda örnekleri gösteren görseller var (yukarıdaki bağlantıya bak).

Örnekler

Örnek 1

```
Girdi : matrix = [["1","0","1","0","0"],["1","0","1","1","1"],
        ["1","1","1","1","1"],["1","0","0","1","0"]]
Çıktı : 4
```

Örnek 2

```
Girdi : matrix = [["0","1"],["1","0"]]
Çıktı : 1
```

Örnek 3

```
Girdi : matrix = [["0"]]
Çıktı : 0
```

Kısıtlar

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 300
- matrix[i][j] '0' ya da '1'dir.